

laSalle

UNIVERSITAT RAMON LLULL

Escola Tècnica Superior d'Enginyeria La Salle

Treball Final de Grau

Grau en Enginyeria Multimèdia

Drummin In The Studio (D.I.T.S.):

Disseny i desenvolupament d'una aplicació
musical en realitat virtual i integració MIDI

Alumne
Tárek Díaz Carissimi

Professor Ponent
Jordi Roig Vilaseca

Título del trabajo

ACTA DE L'EXAMEN DEL TREBALL FI DE CARRERA

Reunit el Tribunal qualificador en el dia de la data, l'alumne

D. Tárek Díaz Carissimi

va exposar el seu Treball de Fi de Carrera, el qual va tractar sobre el tema següent:

Acabada l'exposició i contestades per part de l'alumne les objeccions formulades pels Srs. membres del tribunal, aquest valorà l'esmentat Treball amb la qualificació de

Barcelona,

VOCAL DEL TRIBUNAL

VOCAL DEL TRIBUNAL

PRESIDENT DEL TRIBUNAL

Resum

La bateria és l'instrument base de la música actual, es pot dir que es el "cor" de la banda, sent la responsable de mantenir el tempo i marcar el ritme que guia la resta d'instruments. Tot i la seva importància i l'atractiu que té a l'hora de tocar-la, la seva pràctica quotidiana presenta diverses dificultats: requereix un espai físic considerable, produeix un nivell sonor elevat que condiciona als espais compartits i ofereix poca flexibilitat de personalització, ja que ampliar o modificar el kit implica que es disposi de més espai o una despesa econòmica addicional.

Aquest projecte proposa la realitat virtual (VR) com a alternativa per superar aquestes limitacions i oferir un entorn de pràctica accessible, silenciós i totalment personalitzable que a més ofereixi tot un sistema per a que l'instrument funcioni per a la producció musical. Per fer això, s'ha dissenyat i desenvolupat una sala virtual de pràctica amb Unity i Meta Quest 3.

El sistema inclou la configuració lliure de una bateria modular, un mode d'interpretació amb baquetes virtuals i detecció precisa de cops, així com la possibilitat d'importar cançons MIDI per assajar-les en temps real. També té un sistema de MIDI bidireccional (*in / out*), que permet utilitzar instruments externs, com una bateria electrònica real, com a entrada del sistema, o bé enviar el que s'està tocant dins del programa a un DAW per a la seva gravació. A més de sons propis, control dinàmic de volum i mescla, i un *WindowViewer* que projecta les diferents pantalles de l'ordinador dins la VR per poder, per exemple, veure partitures, reproduir cançons de fons o controlar el DAW sense abandonar l'entorn de realitat virtual.

Posant tot això en comú, aquest projecte no és només una simulació de bateria, sinó un espai complet per practicar, aprendre i crear música dins la realitat virtual.

Resumen

La batería es el instrumento base de la música actual, se puede decir que es el “corazón” de la banda, siendo la responsable de mantener el tempo y marcar el ritmo que guía al resto de instrumentos. A pesar de su importancia y el atractivo que tiene a la hora de tocarla, su práctica cotidiana presenta diversas dificultades: requiere un espacio físico considerable, produce un nivel sonoro elevado que condiciona a los espacios compartidos y ofrece poca flexibilidad de personalización, ya que ampliar o modificar el kit implica disponer de más espacio o un gasto económico adicional.

Este proyecto propone la realidad virtual (VR) como alternativa para superar estas limitaciones y ofrecer un entorno de práctica accesible, silencioso y totalmente personalizable que, además, proporcione un sistema para que el instrumento funcione en la producción musical. Para ello, se ha diseñado y desarrollado una sala virtual de práctica con Unity y Meta Quest 3.

El sistema incluye la configuración libre de una batería modular, un modo de interpretación con baquetas virtuales y detección precisa de golpes, así como la posibilidad de importar canciones MIDI para ensayarlas en tiempo real. También dispone de un sistema MIDI bidireccional (*in / out*), que permite utilizar instrumentos externos, como una batería electrónica real, como entrada del sistema, o bien enviar lo que se toca dentro del programa a un DAW para su grabación. Además de sonidos propios, control dinámico de volumen y mezcla, y un WindowViewer que proyecta las distintas pantallas del ordenador dentro de la VR para poder, por ejemplo, ver partituras, reproducir canciones de fondo o controlar el DAW sin abandonar el entorno de realidad virtual.

En conjunto, este proyecto no es solo una simulación de batería, sino un espacio completo para practicar, aprender y crear música dentro de la realidad virtual.

Abstract

The drums are the foundational instrument of modern music. It can be considered the “heart” of the band, being responsible for keeping the tempo and marking the rhythm that guides the rest of the instruments. Despite its importance and the appeal it has when played the daily practice of it presents several challenges: it requires considerable physical space, produces a high sound level that limits shared environments, and offers little flexibility for customization, since expanding or modifying the kit requires more space or additional financial investment.

This project proposes virtual reality (VR) as an alternative to overcome these limitations and provide a practice environment that is accessible, silent, and fully customizable, while also offering a complete system for the instrument to be used in music production. To achieve this, a virtual practice room has been designed and developed with Unity and Meta Quest 3.

The system includes a freely configurable modular drum kit with a performance mode that uses virtual drumsticks and precise hit detection, as well as the ability to import MIDI songs for real-time practice. It also features a bidirectional MIDI system (in / out), allowing the use of external instruments, such as a real electronic drum kit, as input to the system, or sending what is being played within the program to a DAW for recording. In addition, it includes built-in sounds, dynamic volume and mixing control, and a WindowViewer that projects the computer’s screens inside VR, enabling users to view sheet music, play background tracks, or control the DAW without leaving the virtual environment.

Altogether, this project is not just a drum simulation, but a complete space to practice, learn, and create music within virtual reality.

Agraïments

A la meva família i als meus amics que m'han acompanyat durant tot el procés i han estat un suport constant en els moments bons i en els més difícils. Sense ells, aquest projecte mai es podria haver creat d'aquesta manera.

Però sobre tot, aquest treball el dedico especialment al meu avi, que ha estat un dels meus grans suports al llarg de la carrera i qui ha tingut una gran part de la motivació per a poder acabar-la finalment. Malauradament no ha pogut veure com arribo al final d'aquesta etapa, però el seu exemple, els seus consells i el seu record m'han acompanyat i m'acompanyaran sempre.

Título del trabajo

Contingut

1	Introducció	13
1.1	Context.....	13
1.2	Motivació	17
1.3	Objectius	18
1.4	Estat de l'art.....	19
1.4.1	BeatSaber	19
1.4.2	ParadiddleVR	20
1.4.3	RagnaRock	21
1.4.4	KORG Gadget VR.....	22
1.4.5	MaestroVR	23
2	Context tecnològic	24
2.1	Aplicacions en realitat virtual	24
2.2	Unity.....	26
2.2.1	Introducció a Unity	26
2.2.2	Elements bàsics de Unity.....	27
2.2.3	Unity per a VR.....	35
2.2.4	XR Interaction Toolkit	36
2.2.5	Altres eines utilitzades.....	38
2.3	MIDI	39
2.3.1	Introducció al protocol MIDI	39
2.3.2	MIDI i DAWs.....	40
2.3.3	DryWetMidi	41
3	Contingut de l'aplicació.....	42
4	Metodologia	46
5	Desenvolupament	47
5.1	Idees inicials i prototip.....	47
5.2	Arquitectura general.....	48
5.3	Instruments i interaccions	49
5.3.1	Detecció de cops en VR	51
5.3.2	Entrada MIDI.....	54
5.3.3	Entrada per teclat.....	55
5.3.4	Resposta sonora	56

5.3.5	Animació i partícules	58
5.3.6	Menú de l'instrument.....	60
5.3.7	Cas especial: hi-hat.....	62
5.3.8	Mapa de notes MIDI dels instruments	65
5.4	Funcions implementades.....	66
5.4.1	Spawn dels instruments	66
5.4.2	Drumsets	68
5.4.3	MIDI in/out	74
5.4.4	Cançons MIDI.....	78
5.4.5	Captura de finestres	87
5.4.6	Control de volum i mescla	89
5.5	User Interface	91
5.6	Accessibilitat i usabilitat	95
5.6.1	Ajust de l'alçada	96
5.6.2	Ajust de les baquetes	97
5.6.3	ResetView	98
5.6.4	Menú ambidextre.....	99
5.7	Selecció dels àudios i preparació	100
5.8	Modelatge i ambientació	102
5.9	Rendiment i optimització.....	108
6	Problemes i solucions.....	109
7	Conclusions i futur.....	111
7.1	Conclusions	111
7.2	Línies de futur	112
7.3	Costos.....	113
8	Referències.....	114
9	Il·lustracions	116

Acrònims

API: *Application Programming Interface*

AR: *Augmented reality*

CPU: *Central Processing Unit*

DAW: *Digital Audio Workstation*

LUFS: *Loudness Units relative to Full Scale*

MIDI: *Musical Instrument Digital Interface*

MR: *Mixed Reality*

SDK: *Software Development Kit*

TFG: Treball de Final de Grau

UI: *User Interface*

VR: *Virtual Reality*

XR: *Extended Reality*

Título del trabajo

1 Introducció

1.1 Context

Avui en dia, la música forma una part fonamental de la nostra societat. La podem trobar en festivals, concerts, a les botigues, en anuncis, videojocs, pel·lícules o simplement a través dels nostres telèfons mòbils. Amb el creixement de la digitalització i les plataformes de *streaming*, la música s'ha convertit en un producte global i que acompanya les persones en el seu dia a dia. Ja no només escoltem música de manera passiva, sinó que també la podem produir, compartir i experimentar de formes que fa pocs anys enrere semblaven impensables. [1]



Il·lustració 1. Representació visual de la indústria musical. Font:

<https://www.royaltyexchange.com/blog/influence-of-streaming-services-on-global-music-consumption>

Darrere de cada peça musical hi ha els instruments que la formen, els responsables de donar-li vida. Entre tots ells, tenim a la bateria (Il·lustració 3). Aquest instrument ocupa un lloc central que sovint es converteix en la base i el cor que marca el tempo i el ritme sobre el qual s'estructura una cançó i permet que la resta d'instruments es mantinguin units en una sola peça.



Il·lustració 2. La bateria i les seves parts Font: <https://www.drumeo.com/beat/parts-of-a-drum-set/>

Al mateix temps, la bateria té un atractiu especial per la seva senzillesa i la immediatesa amb què es percep el ritme. És un instrument amb el que es fàcil connectar, fins i tot per a persones sense coneixements musicals, ja que només escoltant-la és possible seguir-ne el compàs i sentir el batec que sosté la cançó. Això la converteix en un instrument intuïtiu i molt gratificant per començar a endinsar-se en la pràctica musical.

Sent així la bateria tant atractiva per tocar, aprendre i practicar amb una bateria, ja sigui tant acústica com electrònica, té una sèrie de limitacions reals:

- L'espai físic del que s'ha de disposar per tindre lloc per a l'instrument.
- El volum que genera és elevat i resulta poc compatible amb entorns domèstics o compartits.
- El cost tant la compra inicial com l'ampliació del kit suposen una despesa econòmica considerable.
- La poca flexibilitat real de personalització ja que modificar o ampliar el kit implica que es disposi de més espai o una despesa econòmica addicional.

Aquestes dificultats fan que moltes persones trobin una gran barrera per a començar o, tot i l'interès, no puguin practicar l'instrument de manera regular.

Un exemple es aquesta publicació de *Reddit* [2] on un usuari comenta, com es pot veure a la Il·lustració 3, com ell ha intentat insonoritzar i adaptar la seva bateria per a no molestar al seus veïns però encara així es massa soroll per als seus veïns arribant a el punt de que a vegades arriba a odiar a l'instrument.



Il·lustració 3. Bateria acústica insonoritzada

Font: https://www.reddit.com/r/drums/comments/8ynxly/still_too_loud_for_my_neighbors_i_hate_this/

Per altra banda, les tecnologies immersives com son la realitat estesa o XR, que engloba a la realitat augmentada o AR, la realitat virtual o VR, i a la realitat mixta o XR, han obert en els darrers anys noves vies i alternatives en molts àmbits. En concret, la VR s'ha consolidat com una eina capaç de generar experiències totalment immersives, on l'usuari pot submergir-se en un entorn digital i interactuar amb objectes i espais de manera natural. Aquesta capacitat de recrear sensacions físiques en un entorn virtual ha estat aprofitada en camps diversos com els videojocs, la simulació mèdica el disseny industrial, l'educació o també, com es en aquest cas, en l'àmbit artístic i creatiu. [3]



Il·lustració 4. Implementació de la VR en laboratori de Ciències de la Salut. Font: <https://simzine.news/es/experience-es/sim-voices-es/utilizar-la-vr-en-el-aula-o-laboratorio-de-ciencias-de-la-salud-con-un-gran-numero-de-estudiantes/>

En el cas concret de la música, la VR ha obert la porta a noves formes de practicar, aprendre i experimentar. Un dels primers èxits van ser títols com *Beat Saber* o *Synth Riders*, que tot i no simular instruments reals, convertien el ritme i el moviment corporal en una experiència lúdica i immersiva.



Il·lustració 5. Fotograma del joc Beat Saber. Font: <https://www.meta.com/es-es/experiences/beat-saber/2448060205267927/>

Amb el temps han aparegut aplicacions més específiques dedicades a la pràctica d'un instrument. Un bon exemple és *Paradiddle*, en el que es pot simular una bateria modular i completament personalitzable. També hi ha jocs com *Ragnarock* que combinen la mecànica d'un joc musical amb la interpretació de ritmes de bateria en VR.



Il·lustració 6. Fotograma d'una cover feta en *ParadiddleVR*. Font: <https://paradiddleapp.com/>

També s'han creat altres aplicacions musicals molt interessants que no només es centren en tocar un instrument, com *Maestro VR*, que permet dirigir una orquestra, o *Korg Gadget VR*, orientada a la producció i creació musical amb sintetitzadors i estacions de treball virtuals.



Il·lustració 7. Exemple del joc *KORG Gadget VR* amb espai de producció musical. Font: https://store.steampowered.com/app/2298070/KORG_Gadget_VR/?l=spanish

La realitat és que la majoria d'aquestes aplicacions se centren exclusivament en un aspecte o en l'entreteniment o en la simulació de l'instrument o creació de música. El repte d'aquest projecte és donar un gir de rosca més i integrar-ho tot en una mateixa experiència: un espai on sigui possible practicar, aprendre i crear música d'una manera flexible i immersiva.

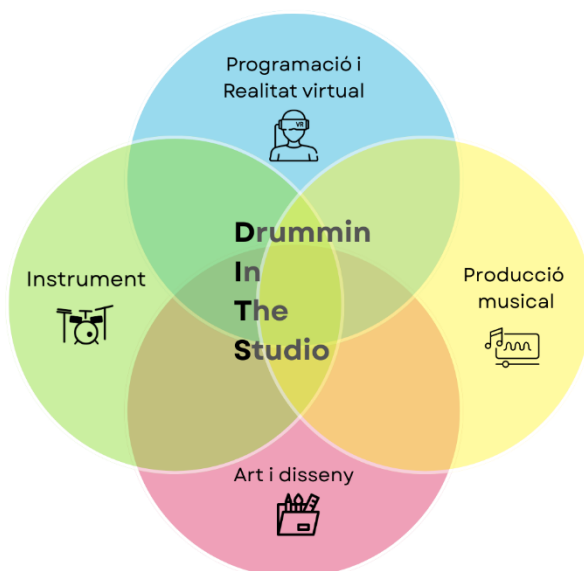
1.2 Motivació

La meua principal motivació per a realitzar aquest TFG és la d'afrontar el repte de dissenyar i desenvolupar una aplicació en realitat virtual, un camp que he pogut donar els meus primers passos en la meua formació en Enginyeria Multimèdia - Menció en Videojocs. Aquest projecte m'ha donat l'oportunitat d'aplicar de manera pràctica molts dels coneixements adquirits al llarg de la carrera, com poden ser el disseny de projectes i videojocs, la programació, el modelatge 3D o el disseny gràfic, en un sol projecte.

Una altra gran motivació ha sigut el poder enfocar el meu projecte cap a la part musical i en especial en l'instrument que es la bateria, que es una gran passió i afició meua. Tot i que tampoc toco la bateria de manera professional, he pogut comprovar de primera mà les dificultats que es tenen a l'hora de la seva pràctica regular: l'espai que requereix, el soroll que genera i el seu cost. Aquests inconvenients han servit per realment voler buscar i trobar solucions reals i m'han permès fer-ho d'una forma molt més personal.

La motivació també ve del meu interès per la producció musical, per això el projecte no es limita a només simular una bateria en VR, sinó que també té la possibilitat d'importar cançons MIDI per practicar-les, connectar instruments externs com una bateria electrònica real i enviar els esdeveniments MIDI a un DAW. Aquesta connexió entre la pràctica i el món de la producció musical ha estat un mix perfecte per a la creació del projecte.

Per últim, la possibilitat de desenvolupar jo mateix tot l'apartat artístic del projecte ha sigut una bona forma de motivar-me a fer el projecte, gràcies als coneixements adquirits en disseny i modelatge 3D, he pogut crear tot l'art necessari per al projecte.



Il·lustració 8. Diagrama de Venn de les motivacions per al projecte

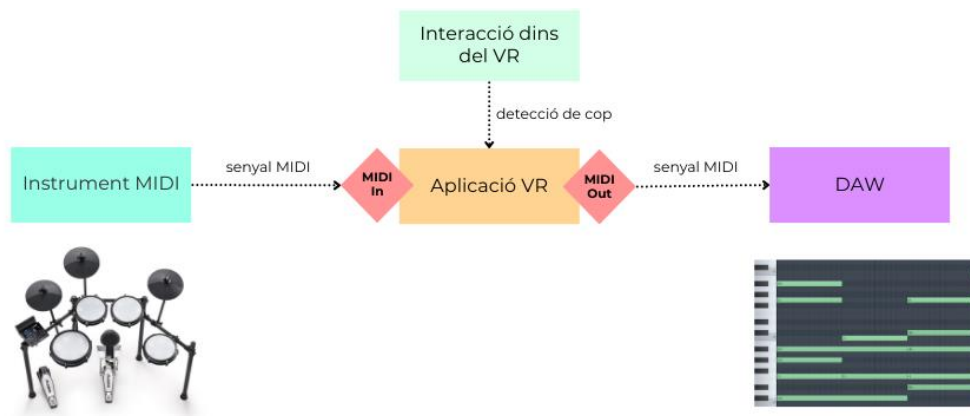
1.3 Objectius

L'objectiu principal d'aquest projecte és crear una aplicació en realitat virtual, utilitzat Unity com a motor, que funcioni com una sala de pràctica musical completa. Llavors, l'aplicació està basada la idea de crear una simulació d'una bateria modular personalitzable, amb integració de MIDI d'entrada i sortida i que permet importar i tocar cançons en temps real, de manera que es pugui connectar la pràctica de l'instrument, la interpretació musical i, fins i tot, la producció de música.

No només es vol centrar en un tipus concret de públic segons el seu coneixement de l'instrument, sinó que es busca que sigui una aplicació a l'abast de tot tipus de persones, tant experimentades en la bateria com noves en aquest món. Per això es vol crear una aplicació intuïtiva i amb un sistema que permeti tindre bateries ja configurades llestes per tocar i també la opció de poder guardar configuracions pròpies.

Com que també la intenció es que sigui una experiència immersiva i el més realista possible tota la interacció amb aquest kit es vol fer a través de baquetes virtuals que utilitzen un sistema de detecció capaç de captar la intensitat, la velocitat i la zona d'impacte. Això permet obtenir una resposta dinàmica i la més propera a la sensació real de tocar una bateria.

Un altre dels objectius mes rellevants és el de que l'aplicació tingui integració MIDI bidireccional, es a dir, d'entrada i de sortida. Així qualsevol instrument MIDI extern, com per exemple una bateria electrònica, pot ser utilitzat com a entrada dins del VR i al mateix temps, la bateria virtual pot funcionar com a instrument de sortida, enviant les notes a través d'un port MIDI a un DAW. Això permet que no només sigui una aplicació simplement lúdica sinó que es pot utilitzar com un medi de producció musical.



Il·lustració 9. Diagrama de com funciona el sistema MIDI bidireccional

També es busca afavorir l'aprenentatge i la pràctica lliure del instrument per això es vol comptar amb un reproductor MIDI que permet importar i tocar qualsevol cançó en aquest format i això sumat a la integració d'un sistema que pugui projectar partitures, reproduir música de fons o controlar un DAW sense sortir de l'espai virtual són objectius que formen part del projecte

Finalment, s'ha volgut crear una aplicació on el rendiment i l'apartat visual i artístic puguin coexistir i per això s'ha escollit la estètica *low poly*, tant per la seva gran personalitat visual com per la seva capacitat d'optimització en dispositius de realitat virtual com les Meta Quest 3.

1.4 Estat de l'art

1.4.1 BeatSaber

Beat Saber és un joc musical rítmic en VR llançat l'any 2018 per Beat Games. Aquest joc va ser un dels primers jocs musicals en realitat virtual que es va fer conegut, i va suposar un canvi important i un impuls en la popularització de la realitat virtual com a tal. [4]

El joc consisteix a tallar blocs de colors que apareixen en funció del ritme de la música, utilitzant dos sabres virtuals que representen les mans del jugador. La mecànica és senzilla i intuïtiva, cada bloc que apareix indica la direcció en què ha de ser tallat i el color correspon a la mà amb què s'ha de fer el moviment.

Beat Saber no només va destacar per la seva jugabilitat, sinó també pel seu enorme impacte cultural i comercial. En menys d'un any va superar el milió de còpies venudes i va rebre premis com el “Best VR/AR Game” als *Game Awards 2019*. El seu èxit es va veure reforçat per la comunitat de jugadors, que va crear *mods* i cançons personalitzades, convertint el títol en una plataforma viva i en constant expansió.

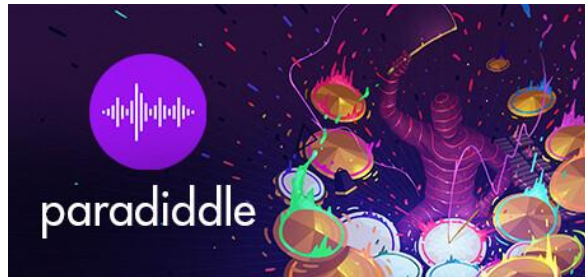
A més, el joc es va reconèixer com una forma d'exercici físic, ja que requereix moviments continus i intensos, motiu pel qual es considera també una aplicació de fitness VR. Tot plegat, Beat Saber es va consolidar com un punt d'inflexió que va establir les bases per al disseny de futurs jocs de ritme immersius en realitat virtual. [4]



Il·lustració 10. Fotograma de gameplay del joc Beat Saber. Font: https://store.steampowered.com/app/620980/Beat_Saber/

1.4.2 ParadiddleVR

ParadiddleVR és una simulació de bateria en realitat virtual, publicada amb accés anticipat el 2017 per Emre Tanirgan i és, probablement, l'aplicació que més s'assembla al projecte presentat en aquest TFG, ja que simula de manera directa una bateria en realitat virtual. [5]



Il·lustració 11. Banner oficial de ParadiddleVR.

Font: <https://store.steampowered.com/app/685240/Paradiddle>

ParadiddleVR permet col·locar diferents parts del kit de la bateria en un entorn 3D i tocar-les amb baquetes virtuals. Una de les seves característiques principals és la possibilitat de configurar el kit a mida i gravar interpretacions, fet que la converteix en una eina útil tant per a aficionats com per a músics que volen practicar sense una bateria real.



Il·lustració 12. Kit modular del joc ParadiddleVR.

Font: <https://store.steampowered.com/app/685240/Paradiddle>

Un dels punts forts de *ParadiddleVR* és la seva precisió en la detecció dels cops i la incorporació de dinàmiques sonores que varien en funció de la intensitat. A més, ofereix un sistema de reproducció de cançons, ampliant així les opcions de pràctica. Per altra banda, la seva integració amb entorns professionals de producció musical és limitada i, encara que permet exportar interpretacions, no ofereix un control MIDI complet ni eines que facilitin la connexió amb DAWs de forma nativa.

Per això, *ParadiddleVR* és un exemple clau per aquest projecte ja que ha servit de gran inspiració per aquest projecte ja que molt camí ja ha estat explorat tot i que s'utilitzi un motor i programació molt diferents del que s'utilitza en aquest projecte. També es pot veure com es mostra el potencial i l'interès que té una aplicació de bateria en VR, però on també on es poden explorar nous espais de millora en el que aquest projecte vol aconseguir i aprofundir.

1.4.3 RagnaRock

RagnaRock és un joc rítmic en VR, desenvolupat per WanadevStudio i llançat el 2021, que combina música, moviment i un estil visual molt característic. [6]

En *RagnaRock* el jugador adopta el paper d'un capità viking que colpeja tambors gegants per marcar el ritme de cançons de folk i metall nòrdiques. Els cops correctes fan avançar a un vaixell, de manera que la mecànica de joc no només depèn de seguir el ritme, sinó també de superar reptes visuals i obtenir puntuacions.



Il·lustració 13. Fotograma del joc RagnaRock.

Font: <https://store.steampowered.com/app/1345820/Ragnarock/>

La rellevància de RagnaRock és que, tot i no buscar realisme en la simulació de la bateria, aporta un component clar d'immersió i diversió a través del ritme i la percussió. És un bon exemple de com la VR pot utilitzar elements musicals per crear experiències lúdiques que connecten amb l'energia de tocar un instrument, encara que en aquest cas sigui de manera més simbòlica i arcade.

En el context d'aquest TFG, RagnaRock s'inclou com a referència per mostrar la diferència entre un joc musical i una eina de pràctica instrumental. Mentre que el projecte present vol aconseguir una experiència precisa i útil per a la pràctica real de la bateria, RagnaRock posa l'èmfasi en la gamificació i en l'espectacle. Tanmateix, demostra l'atractiu que té el ritme i la percussió dins la realitat virtual, i valida la idea que aquest tipus d'interaccions connecten de manera especial amb els usuaris.

1.4.4 KORG Gadget VR

KORG Gadget VR representa una altra vessant de la música en realitat virtual, llançada el 2023 per la companyia japonesa KORG. Aquesta eina adapta a VR la popular estació de treball *KORG Gadget*, incorporant sintetitzadors, caixes de ritmes i seqüenciadors dins d'un estudi virtual. [7]

Aquesta aplicació no se centra entrada no en la interpretació d'un instrument concret, sinó en la producció musical. L'usuari pot manipular els controls amb les mans i construir composicions dins d'un entorn immersiu que afegeix noves dimensions a la creació musical.



Il·lustració 14. Fotograma de KORG Gadget VR.

Font: https://store.steampowered.com/app/2298070/KORG_Gadget_VR/

L'interès d'aquesta aplicació radica en com la VR pot transformar processos habituals de producció i composició musical, fent-los més visuals i intuïtius. No es tracta de practicar un instrument, sinó de tenir un estudi virtual complet al qual accedir sense necessitat de hardware físic.

En aquest projecte, *KORG Gadget VR* serveix com a punt de comparació per la seva orientació professional i la seva connexió amb la producció musical. Tot i que el present TFG es centra en la bateria, comparteix amb *KORG Gadget VR* la voluntat d'anar més enllà del joc i acostar-se a eines útils per a músics i creadors. La diferència és que, en aquest cas, la proposta combina interpretació i producció dins un mateix entorn, mentre que *KORG Gadget VR* està més focalitzat en la composició.

1.4.5 MaestroVR

MaestroVR és una aplicació de direcció orquestral en VR, desenvolupada per Symphonic Games i publicada el 2020. En aquesta aplicació l'usuari hi assumeix el rol de director i pot controlar una orquestra virtual mitjançant els seus moviments, influint en el tempo, la dinàmica i l'expressivitat de la interpretació. [8]

El valor de MaestroVR és mostrar com la VR pot simular rols musicals que difícilment serien accessibles en la vida real. Ser director d'una orquestra requereix recursos i oportunitats molt específiques, però amb aquesta aplicació qualsevol persona pot experimentar què significa guiar un conjunt musical i sentir el poder expressiu de la direcció.



Il·lustració 15. Fotograma del joc Maestro VR.

Font: https://store.steampowered.com/app/1327920/Maestro_VR/

Aquest exemple s'ha inclòs en l'estat de l'art per ampliar la perspectiva i demostrar la versatilitat de la música en VR. Tot i que és molt diferent del projecte present, ajuda a destacar que les aplicacions musicals immersives poden abastar tant la pràctica instrumental com la producció o la direcció, i que la proposta d'aquest TFG se situa enmig d'aquest ventall, buscant unir la interpretació amb la producció musical dins un mateix entorn.

2 Context tecnològic

2.1 Aplicacions en realitat virtual

Tot i que ja hi va haver intents als anys noranta, com el Virtual Boy de Nintendo (Il·lustració 16) o diversos prototips de cascs de realitat virtual, les limitacions tècniques i tecnològiques d'aquella època van fer que la realitat virtual no pugui prosperar. Aquesta nova tecnologia no va començar a consolidar-se fins a la dècada del 2010 amb l'aparició d'Oculus Rift i posteriorment HTC Vive, PlayStation VR i, com ha sigut més recentment, els dispositius Meta Quest. Aquests avenços van fer possible que la VR sigui molt més accessible per al públic general, amb qualitat i preus molt més assequibles.



Il·lustració 16. Consola Virtual Boy de Nintendo. Font: https://es.wikipedia.org/wiki/Virtual_Boy

A partir de la comercialització d'aquesta tecnologia d'una forma més normalitzada han sorgit aplicacions de tot tipus en la que destaquen els videojocs que són especialment populars i existeixen pràcticament de tots els gèneres: shooters, survival, simuladors, esportius, jocs de rol, per posar alguns exemples. Un dels jocs més valorats ha sigut el joc Half-Life: Alyx que té una història immersiva i completa.



Il·lustració 17. Fotograma del videojoc Half-Life: Alyx. Font: https://store.steampowered.com/app/546560/HalfLife_Alyx

En paral·lel al creixement de la realitat virtual, ha sorgit el concepte de metavers, entès com un entorn digital persistent i compartit on els usuaris poden interactuar mitjançant avatars en espais immersius. Aquest fenomen ha anat més enllà dels videojocs tradicionals per convertir-se en una plataforma social i creativa. Un dels exemples més coneguts és VRChat, una aplicació llançada el 2014 que s'ha fet molt popular per permetre a milers d'usuaris crear i personalitzar els seus propis mons virtuals, assistir a esdeveniments i socialitzar en temps real. VRChat demostra el potencial del metavers com a espai col·lectiu, on la realitat virtual no només serveix per jugar sinó també per establir vincles socials i culturals, obrint la porta a noves opcions d'entreteniment i comunitat. [9] [10]



Il·lustració 18. VRChat. Font: <https://www.businessinsider.com/vrchat-explained-2018-2>

Actualment, la indústria de la VR continua creixent i consolidant-se. Encara que el mercat sigui més reduït que el de les aplicacions tradicionals, cada cop hi ha més estudis i empreses que aposten per aquesta tecnologia. La realitat virtual s'ha convertit en una plataforma amb un gran potencial de futur, no només en l'entreteniment sinó també en àmbits educatius, creatius i professionals.



Il·lustració 19. La realitat virtual aplicada en entorns professionals. Font: <https://centroformacionprofesorado.castillalamancha.es/comunidad/crpf/recurso/realidad-virtual-y-su-aplicacion-en-fp-a-traves/a5a471b6-b629-40cb-80fa-a8e216d256d7>

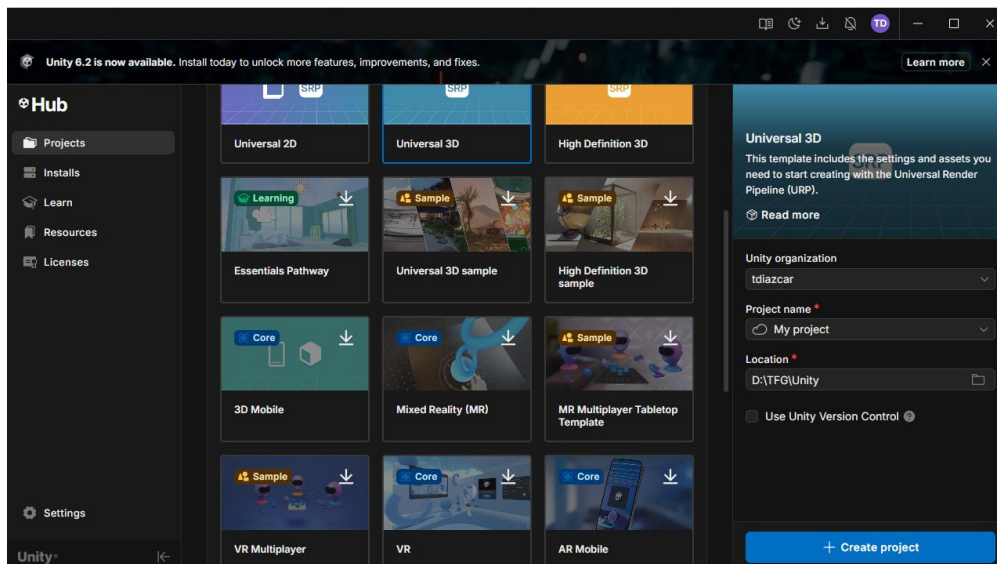
2.2 Unity

2.2.1 Introducció a Unity

Unity és un motor de desenvolupament multiplataforma creat per Unity Technologies, llançat per primera vegada l'any 2005 i disponible tant per Microsoft Windows com per Mac OS i Linux. Aquest motor s'ha convertit en un dels més utilitzats, tant en l'àmbit de l'entreteniment, com poden ser els videojocs, com en altres sectors molt diversos sectors com l'arquitectura, la simulació, l'educació o la realitat virtual.

Com a motor gràfic, Unity integra en un sol entorn totes les funcionalitats necessàries per desenvolupar aplicacions 2D i 3D. Incorpora un motor de renderitzat que permet generar escenes visuals complexes, un motor físic basat en NVIDIA PhysX per simular el comportament real dels objectes i un gran conjunt d'eines per gestionar àudio, interfícies d'usuari, animacions i, fins i tot, intel·ligència artificial. El llenguatge de programació principal és C# que s'ha convertit en estàndard dins de Unity.

Un dels punts destacats de Unity és la seva multiplataforma que permet que els projectes es puguin compilar i siguin compatibles amb les principals plataformes mòbils, de escriptori, consoles i web. Això permet treballar sobre un únic projecte i exportar-lo a múltiples plataformes sense necessitat de reescriure tot el codi. A més té un suport oficial que permet crear joc, aplicacions i solucions industrials evolutives per a la nova era de la realitat estesa. [11]



Il·lustració 20. Aplicació UnityHub amb els seus diferents templates.

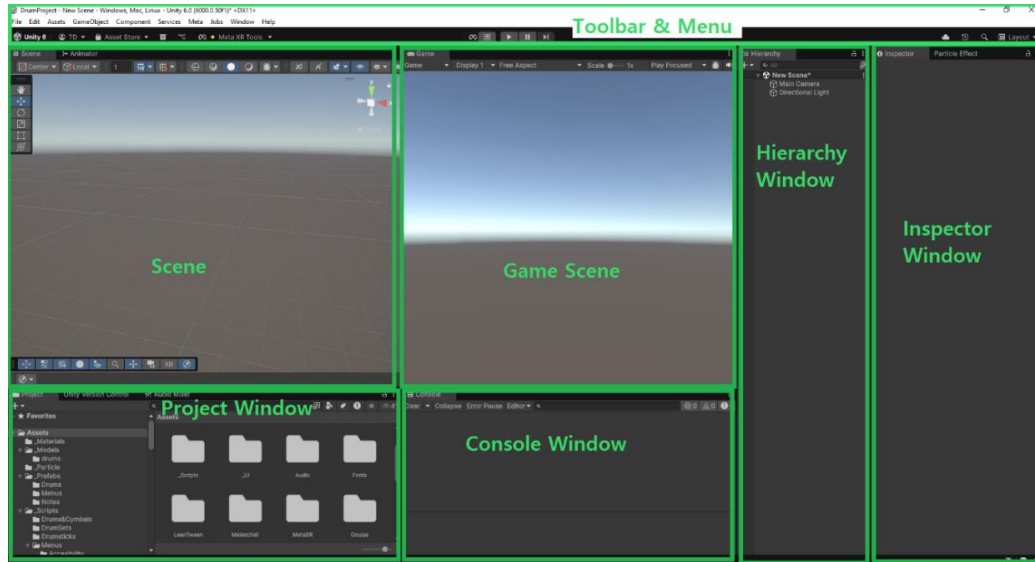
També un punt a favor de Unity es la corba d'aprenentatge, que resulta fàcil i intuïtiva a més de una gran comunitat activa que l'envolta, una documentació oficial gegant i es poden trobar tutorials de tot tipus.

Per aquests motius són per als quals també s'ha decidit utilitzar Unity com a plataforma d'aprenentatge utilitzada per algunes assignatures a la carrera de Enginyeria Multimèdia – Menció en Videojocs i que ha servit d'inspiració per utilitzar-lo en aquest projecte.

2.2.2 Elements bàsics de Unity

2.2.2.1 Interfície

Tal i com es pot observar a la Il·lustració 21. Interfície de Unity i les seves parts. Il·lustració 21 això es el que es troba al obrir un projecte de Unity.



Il·lustració 21. Interfície de Unity i les seves parts.

En la part superior es troba el *Toolbar & Menu*. Des de el menú es on podem tocar tot tipus d'opcions com crear projectes nous, obrir ja existents, entrar a la configuració del projecte, la configuració del build, crear nous Assets, entre altres moltes més. També a la *Toolbar* trobem els botons de *Play* i *Pause* que serveixen per reproduir i pausar el programa que s'està fent en temps real.

La *Scene* és l'escenari on es poden anar col·locant tots els objectes, anomenats *GameObjects* que inclouen càmeres, llums i models i on es poden interactuar amb aquests objectes.

En el *Game Scene* es mostra com veurà l'usuari final quan s'executi l'aplicació a través de veure el punt de vista de la càmera principal que està activa en aquell moment. En el cas de la realitat virtual, quan s'utilitza unes ulleres en la *Game Scene* es on es veu el que veu l'usuari a través del dispositiu.

El *Hierarchy Window* és la llista de tots els objectes que es troben en l'escena. Aquí també es poden afegir, eliminar o organitzar els element.

A l'*Inspector Window* es on es mostra i es permet modificar les propietat de l'objecte seleccionat a la *Hierarchy Window* o a la *Scene*. On també es poden canviar els components com Scripts, materials, transformacions, entre altres.

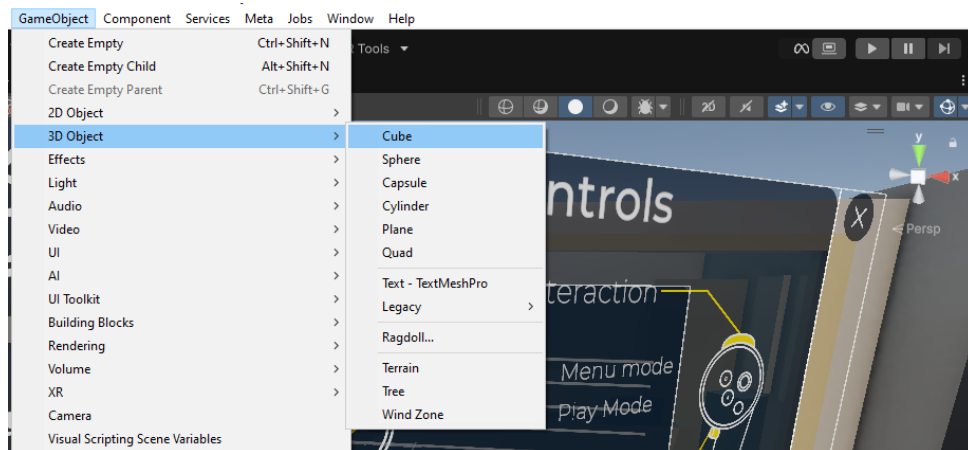
El *Project Window* conté tots els arxius i recursos del projecte com són els models, scripts, àudios, materials, prefabs... Funciona com un explorador d'arxius organitzat en carpetes.

El *Console Window* mostra missatges, advertències i errors que genera el motor o els scripts durant l'execució. És essencial per depurar i fer seguiment del funcionament del projecte. [12]

2.2.2.2 Game Objects i Components

Els *GameObjects* són els objectes fonamentals de Unity que representen personatges, *props* i l'escenari. Per ells mateixos no fan res però funcionen com a contenidors per als *Components*, que implementen la vertadera funcionalitat. [13]

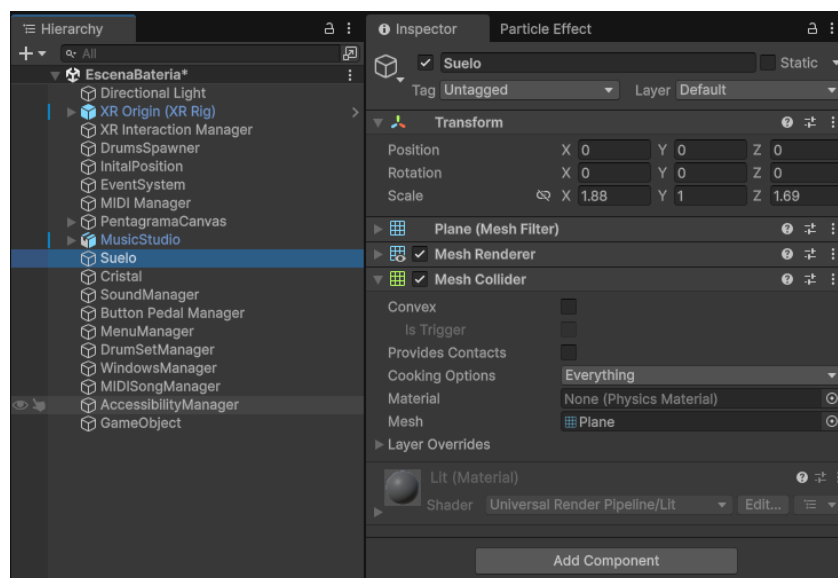
Com es pot observar en la llista de la Il·lustració 22, hi ha gran varietat de *GameObjects*, en aquest exemple es pot veure com s'està seleccionant per crear un cub.



Il·lustració 22. Llistat de GameObjects de Unity.

Els *Components*, per altra banda, defineixen el comportament i característiques dels *GameObjects*, com ara el *Transform* (posició, rotació i escala), un *Rigidbody* (afegeix física), un *Collider* (col·lisions) o scripts personalitzats. Els *GameObjects* també es poden crear sense cap Component, és a dir objectes buits.

A la Il·lustració 23 es pot veure com s'ha seleccionat en el *Hierarchy* el *GameObject* anomenat "Suelo" on es poden veure el seus components en el *Inspector*.

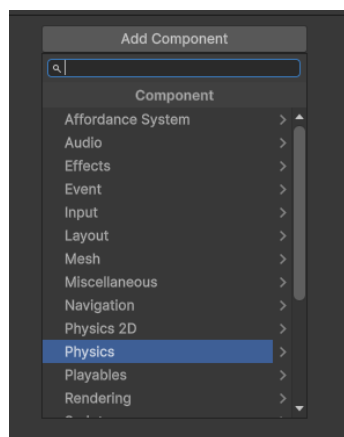


Il·lustració 23. Exemple de un GameObject i els seus components

Un *GameObject* sempre té el component *Transform* adjunt (per representar la posició y la orientació) i no es possible treure això. Els altres components que li donen a l'objecte funcionalitat poden ser agregats des del menú *Component* del editor o des de un script.

Una característica interessant que s'ha utilitzat bastant en aquest projecte ha sigut la possibilitat de activar i desactivar els *GameObjects*, per poder mantenir instanciat cada *GameObject* però que no estiguin activats a la escena. A la Il·lustració 23. Exemple de un *GameObject* i els seus components Il·lustració 23 es pot observar el botó d'activació a la part superior esquerra de l'*Inspector*.

Per agregar nous components es pot utilitzar el *Component Browser*, que pot ser activat amb el botó *Add Component* en el inspector de l'objecte (Il·lustració 24).



Il·lustració 24. *Component Browser* de Unity.

Un dels grans aspectes dels Components son la seva flexibilitat. Quan s'adjunta un Component a un *GameObject*, hi ha diferents valors o *Properties* dins del Component que poden ser ajustats a l'editor mentre es construeix un joc, o bé mitjançant scripts quan el joc s'està executant. Hi ha dos tipus de propietats (*Properties*): *Values* i *References*. [14]

A la Il·lustració 25 es pot observar un component *Script* del projecte anomenat "Hi Hat Hit Detector" on té diferents *Properties* amb *Values* i *References*.



Il·lustració 25. Exemple de un component *Script* amb les seves propietats.

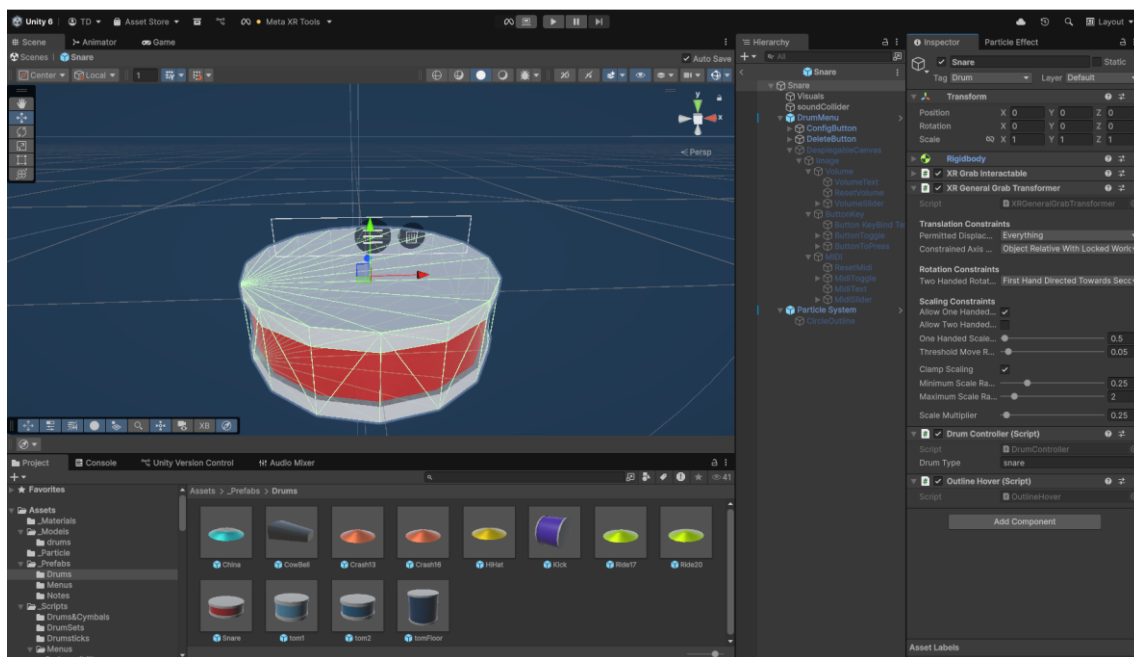
2.2.2.3 Prefabs

Un Prefab és un tipus especial de *GameObject* que es desa com a una plantilla reutilitzable, permet instanciar còpies de un *Game Object* configurat amb tots els seus components que s'han afegit mantenint la configuració.

Qualsevol edició feta a un *prefab* es reflectirà immediatament en totes les instàncies produïdes a partir seu, però també es poden anul·lar components i ajustos per a cada instància individualment. [15]

En aquest projecte, els *prefabs* han estat essencials per gestionar les diferents peces de bateria i els menús de forma individual. Donant la possibilitat de després poder fer còpies d'aquests, ja que s'han d'anar fent instàncies de cada peça per separat, i un cop instanciat el *prefab*, ja poder canviar les seves propietats per separat si l'usuari vol.

En aquesta Il·lustració 26 es pot observar un *prefab* creat per una peça de la bateria d'aquest projecte des de l'editor de *prefabs*. En la part inferior, en el *Project Window*, també es pot observar els altres *prefabs* creats.



Il·lustració 26. Un prefab i la seva configuració desde l'editor.

Per crear un *prefab*, només cal arrossegar el *GameObject* que es desitja convertir en *prefab*, és a dir, el que es vol que serveixi com a plantilla, a la *Project View*, on es troben la resta dels *assets*.

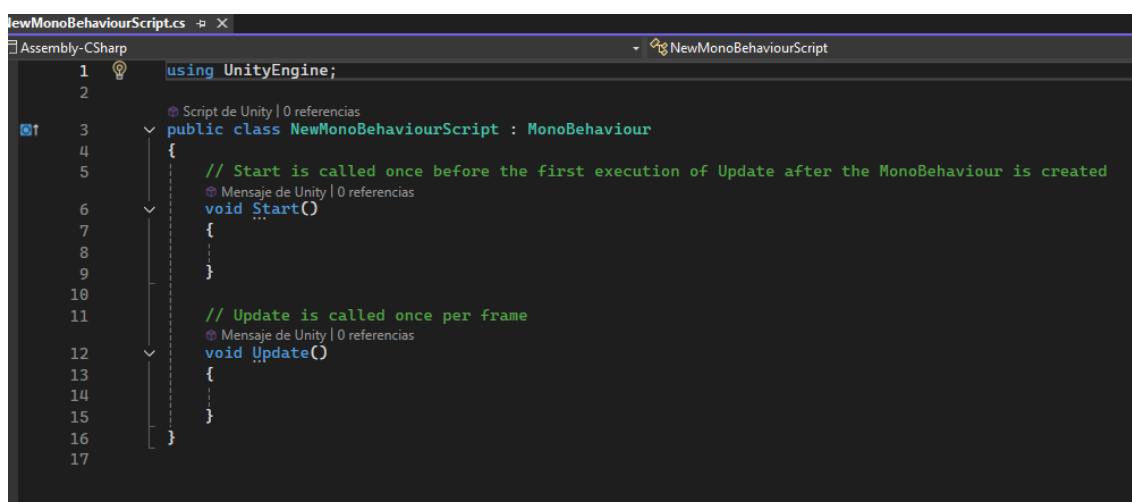
2.2.2.4 Scripts

Com s'ha vist en l'apartat *Game Objects* i *Components*, els *GameObjects* són controlats pels *Components* que estan adjunts. Tot i que els *Components* de Unity són molt versàtils, es necessita anar més enllà per poder donar funcionalitats específiques per a la creació del programa. [16]

Els scripts a Unity permeten programar el comportament dels *GameObjects* mitjançant el llenguatge de programació C#. A través d'ells, es defineixen accions, interaccions i lògica de joc.

A diferència de la majoria d'altres *assets*, els scripts normalment es creen directament dins de Unity. Es pot crear un script nou des del menú *Create* a la part superior esquerra del panell *Project*, o bé des del menú principal. L'script nou es crea dins la carpeta que s'hagi seleccionat al panell *Project*.

Aquests són fragments de codi que es creen al fer un nou script que tenen de base dues funcions: *Start()* i *Update()*. La primera només es crida una sola vegada, quan s'inicia el *GameObject* al que està assignat, la segona es crida a cada *frame*.



```
1 using UnityEngine;
2
3 public class NewMonoBehaviourScript : MonoBehaviour
4 {
5     // Start is called once before the first execution of Update after the MonoBehaviour is created
6     void Start()
7     {
8     }
9
10    // Update is called once per frame
11    void Update()
12    {
13    }
14 }
15
16
17
```

Il·lustració 27. Nou Script de Unity.

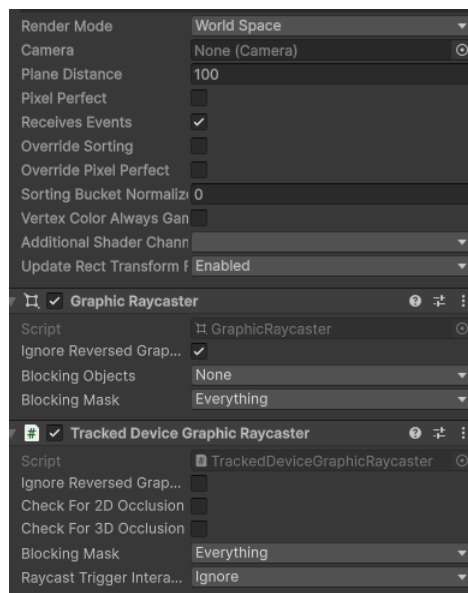
En aquest projecte, s'han desenvolupat scripts específics per gestionar cada una de les parts que s'han implementat.

2.2.2.5 User Interface (UI)

El sistema de UI de Unity permet crear interfícies d'usuari ràpides i intuïtives. Aquest sistema permet crear menús, botons, sliders i altres elements amb què l'usuari pot interactuar. Unity ofereix un sistema complet de UI que es pot integrar tant en aplicacions 2D com 3D, i que és essencial en projectes VR per gestionar funcions de manera intuïtiva. [17]

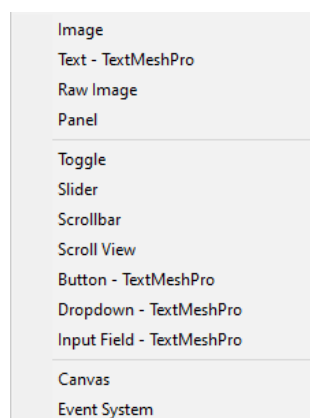
Per a la creació d'una interfície, es necessita crear un *Canvas*. El Canvas és l'àrea on han d'estar tots els elements de la interfície d'usuari (UI).

Per a poder crear una UI que funcioni en un programa de realitat virtual s'ha de configurar de manera concreta. Com es pot observar en la Il·lustració 28, el *canvas* ha d'assignar el *render mode* a *World Space* i afegir els components de *GraphicRaycaster* i *Tracked Device Graphic Raycaster*.



Il·lustració 28. Componentes de un canvas per a VR.

En aquest es el menú amb tots els elements que es poden afegir a un *Canvas* i que l'usuari pugui interactuar per anar navegant pels menús que s'han creat pel projecte (Il·lustració 29).



Il·lustració 29. Menú d'elements de la UI de Unity

Els elements principals utilitzats en la interfície en aquest projecte son:

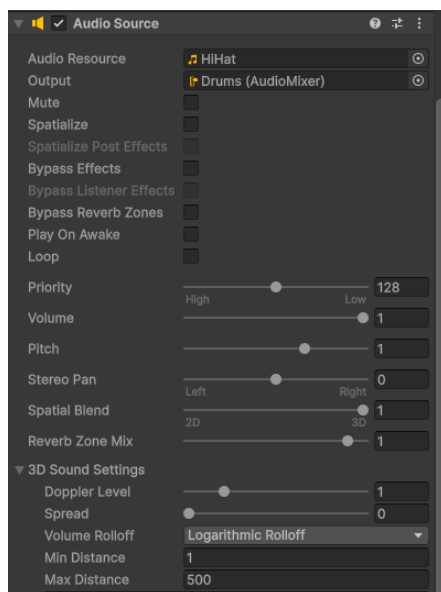
- *Button*, es l'element que respon quan l'usuari hi fa clic, que es pot utilitzar per executar accions a través de l'event *OnClick()*.
- *Toggle*, és un element que funciona com una casella de verificació que es pot marcar o desmarcar.
- *Slider*, és un element que permet seleccionar un valor dins d'un rang continu movent un punter. S'utilitza, per exemple, per ajustar el volum o l'angle de les baquetes. També emet un event *OnValueChanged* al canviar de valor.
- *Dropdown*, permet a l'usuari escollir una opció d'entre una llista.
- *Text – TextMeshPro*, es l'element que s'utilitza per escriure text i que permet personalitzar fonts, mides, colors, alineació, efectes, entre altres opcions.

2.2.2.6 Àudio

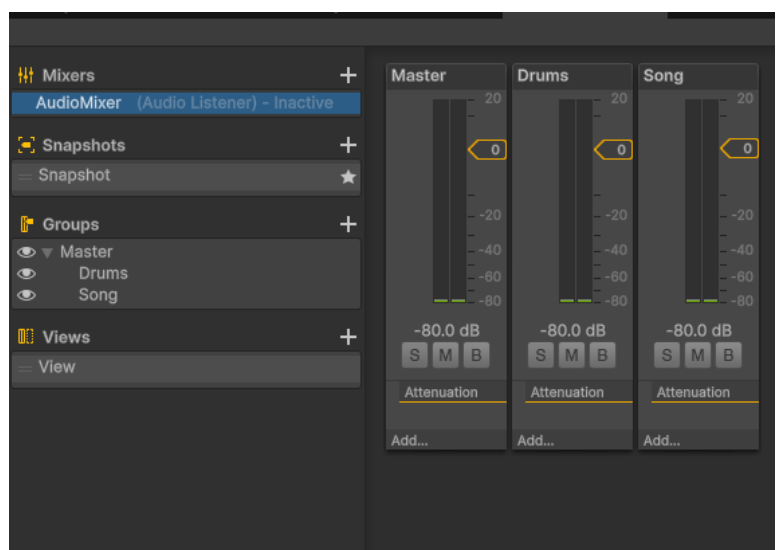
Per a la gestió de l'àudio Unity compta amb tot un sistema que permet reproduir, controlar i organitzar sons dins d'una aplicació interactiva.

Aquest sistema es basa en l'ús de components com els *Audio Source*, que permet reproduir els sons en 2D o 3D dins de l'escena, i eines com l'*Audio Mixer*, que fan possible la mescla de volums i aplicar efectes.

Cada component que emet so dins del projecte, compta amb un *Audio Source* (Il·lustració 31) que conté la configuració individual de reproducció dels sons i que estan centralitzats tots a un *Audio Mixer* (Il·lustració 30) centralitzat que permet gestionar el volum. L'àudio mixer pot crear grups i subgrups de audio que poden ser controlats individualment per poder separar les diferents fonts d'àudio.



Il·lustració 31. Audio Source de Unity.

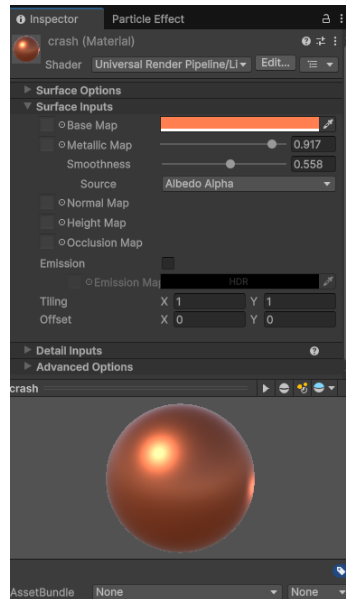


Il·lustració 30. Audio Mixer de Unity.

2.2.2.7 Materials i partícules

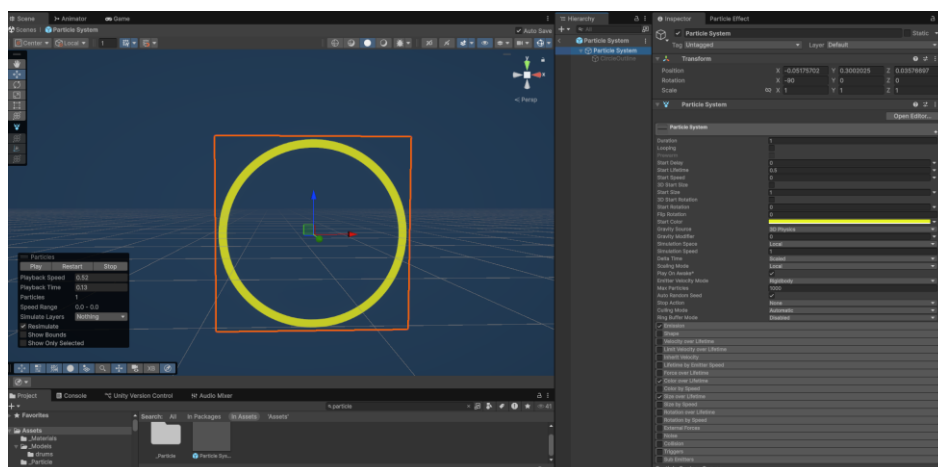
Els Materials a Unity són *assets* que defineixen l'aparença visual de la superfície d'un objecte 3D. Un material combina un *shader* (un programa GPU que determina com es renderitza el material) amb propietats com el color, la textura o el canal de reflectivitat. [19]

Els materials es guarden amb extensió `.mat` i es poden editar a l'Inspector, ajustant valors com textures, colors i configuracions *shader* tal i com es mostra a la Il·lustració 32. Material vist des de l'inspector de Unity.



Il·lustració 32. Material vist des de l'inspector de Unity.

Les partícules, en canvi, són sistemes visuals que permeten crear efectes dinàmics com fum, pols, espurnes o ones de llum. Gràcies al component *Particle System* es poden personalitzar les partícules generades. [20] En el projecte s'utilitzen partícules quan es detecta un cop en un instrument. A la Il·lustració 33 es pot veure l'exemple del *prefab* d'una partícula.

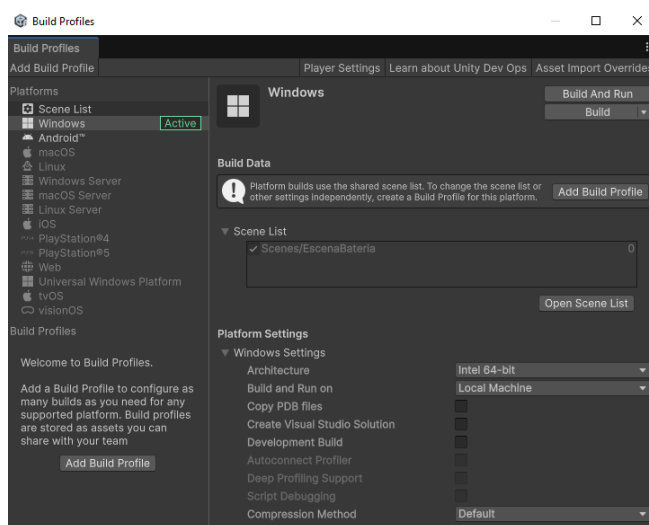


Il·lustració 33. Partícula i el sistema de partícules

2.2.3 Unity per a VR

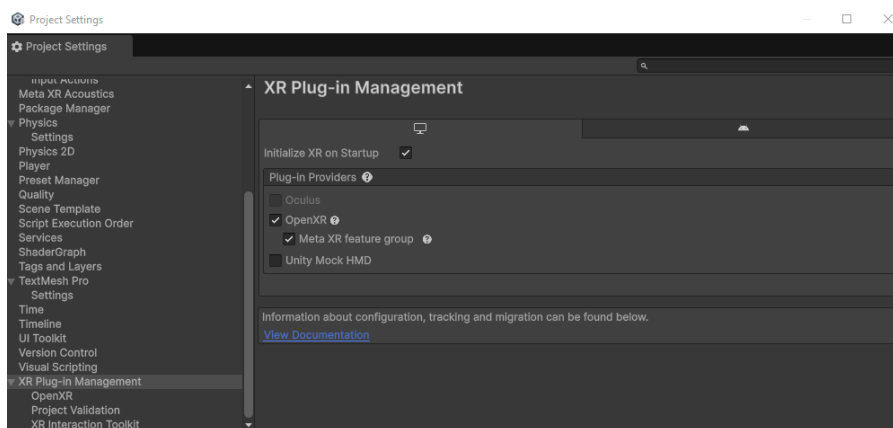
Unity no només és un motor versàtil per a videojocs i aplicacions interactives, sinó que també està àmpliament preparat per al desenvolupament en realitat virtual. La seva arquitectura ja inclou compatibilitat amb la majoria de dispositius VR actuals, com Meta Quest, HTC Vive o PlayStation VR, i permet configurar projectes immersius de manera relativament senzilla. [21]

El primer pas per preparar un projecte de VR a Unity és configurar correctament els *Build Settings*. En aquest camp s'ha de establir la plataforma de destinació i els ajustos de resolució i textures. En el cas d'aquest projecte s'utilitza Windows com a plataforma ja que es necessita tenir connexió amb aplicacions externes, però la majoria de aplicacions en VR es configurarien per fer el *build* en Android.



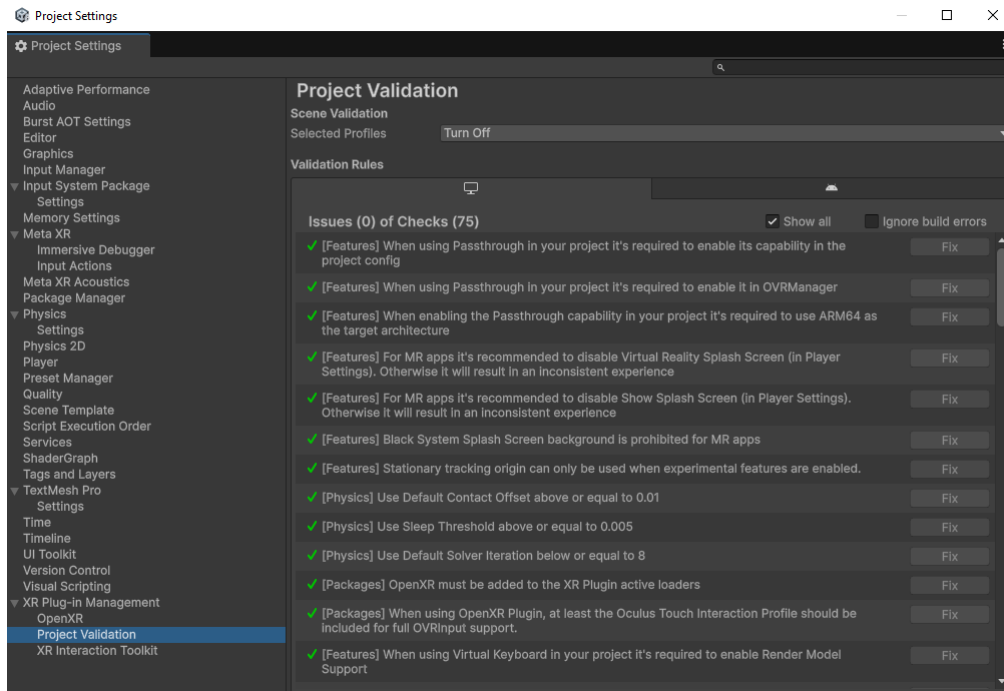
Il·lustració 34. Build Settings de Unity

Un altre punt clau és l'activació del sistema XR Plugin Management i la seva configuració. En aquest projecte s'ha fet servir OpenXR, l'estàndard recomanat actualment per Unity, que garanteix compatibilitat amb la majoria de dispositius VR i ofereix suport complet per al seguiment del cap, dels controladors i l'entrada d'usuari.



Il·lustració 35. XR Plug-in Management

A més, Unity incorpora l'eina de *Project Validation*, que revisa la configuració del projecte i comprova que tots els requisits necessaris per a VR i que estiguin correctament configurats.



II-lustració 36. Pestanya de Project Validation sense errors

Per a la implementació de les interaccions s'ha utilitzat exclusivament l'XR Interaction Toolkit, un paquet oficial de Unity que proporciona components preparats per gestionar la càmera, els controladors i la manipulació d'objectes en VR. [21]

Gràcies a aquesta validació, es pot garantir que el projecte estigui preparat per funcionar de manera òptima a les Meta Quest 3 i que no quedin errors de configuració que podrien afectar l'experiència immersiva.

2.2.4 XR Interaction Toolkit

El XR Interaction Toolkit és un paquet d'Unity que proporciona un sistema d'interacció d'alt nivell i basat en components, especialment dissenyat per crear experiències de VR i AR. Ofereix un marc estructurat per gestionar interaccions tant 3D com amb la interfície d'usuari (UI) a partir dels esdeveniments d'entrada del sistema d'Unity. [22]

Aquest és el paquet que s'utilitza com a base per a la interacció de l'usuari amb l'entorn on els components principals utilitzats en el projecte són:

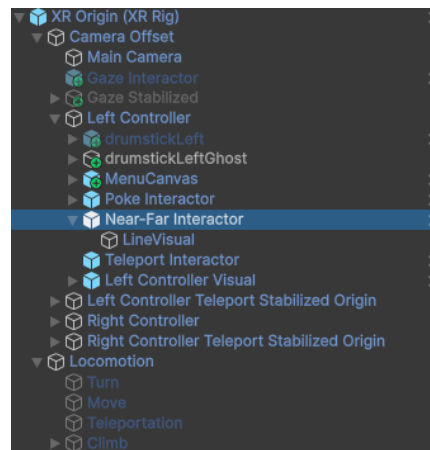
- Interaction Manager
- XR Origin (XR Rig)
- XR Grab Interactables

A continuació, s'explicaran en detall.

Cada *Scene* que utilitza l'XR Interaction Toolkit necessita com a mínim un Interaction Manager per facilitar la interacció entre *Interactors* i *Interactables*. Per defecte, els *Interactors* i *Interactables* fan referència al primer *Interaction Manager* que troben a l'escena si no se n'especifica cap. [22]

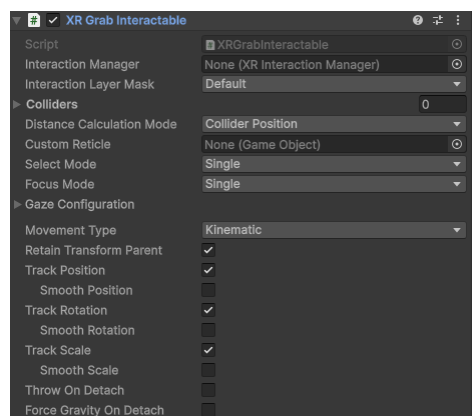
L'XR Origin (abans conegut com XR Rig) és el prefab principal que proporciona l'XR Interaction Toolkit per configurar ràpidament el moviment i les interaccions de l'usuari en VR. Aquest *prefab* conté:

- Main Camera (VR Camera). Representa els ulls de l'usuari. Està connectada al sistema de seguiment de cap (head tracking) i mostra exactament allò que veu la persona amb el casc VR.
- Left/Right Hand Controller. GameObject amb components d'Interactor per simular les mans i detectar interaccions properes (Direct Interactor) o llunyanes (Ray Interactor).
- XR Origin transform. és el punt d'origen que es pot reubicar o reescalar per adaptar l'alçada de l'usuari i el seu espai de joc.



II·l·lustració 37. Prefab XR Origin del XR Interaction Toolkit

El component XR Grab Interactable permet que un objecte dins de l'escena pugui ser agafat, mogut o llançat. Es connecta amb l'XR Interaction Manager per gestionar la relació entre els Interactors i els Interactables. A més, pot seguir físicament el moviment del controlador (si té un Rigidbody) i fins i tot conservar la velocitat quan és alliberat.



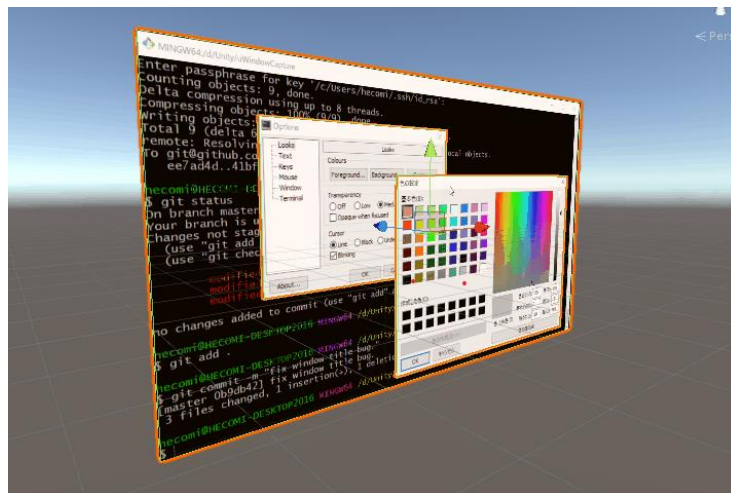
II·l·lustració 38. Component XR Grab Interactable.

2.2.5 Altres eines utilitzades

Per a poder implementar algunes funcions de l'aplicació s'ha fet us d'eines que no venen implementades a Unity com a tal però que han sigut molt útils. Aquestes eines han estat uWindowCapture, LeanTween i DryWetMidi. A continuació es farà una explicació de les dues primeres i a l'apartat 2.3.3 s'explicarà la última.

2.2.5.1 uWindowCapture

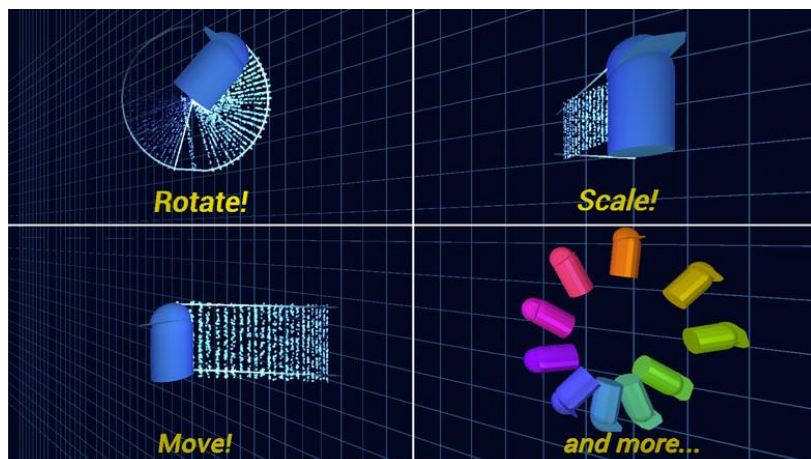
uWindowCapture és un *asset* per a Unity que permet capturar finestres de Windows mitjançant APIs com Windows Graphics Capture, PrintWindow o BitBlt. Aquestes captures es poden utilitzar dins Unity com a Texture2D, habilitant la visualització de finestres del PC directament a l'entorn VR. [23]



Il·lustració 39. uWindowCapture en acció. Font: <https://github.com/hecomi/uWindowCapture>

2.2.5.2 LeanTween

Per a l'animació de diferents elements s'ha utilitzat LeanTween que es un engine de tweening altament eficient per a Unity3D, creat per Dented Pixel. Facilita animar propietats d'objectes com posició, rotació, escala o opacitat de manera senzilla, sense necessitat del sistema de Animator. [24]



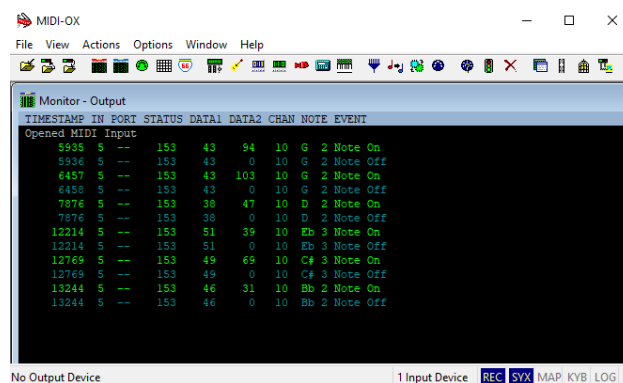
Il·lustració 40. Exemples de animacions que pot fer LeanTween

2.3 MIDI

2.3.1 Introducció al protocol MIDI

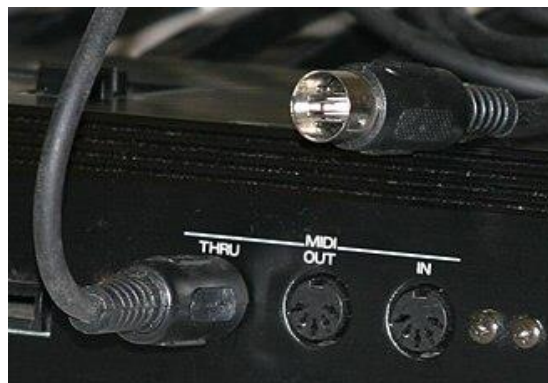
El MIDI, sigles en anglès de *Musical Instrument Digital Interface*, és un estàndard tècnic establert el 1983 per permetre la comunicació entre instruments musicals electrònics, ordinadors i altres dispositius d'àudio. S'usa un protocol digital que transmet esdeveniments musicals: notes, intensitat, sincronització i altres paràmetres. Cada esdeveniment pot incloure informació com el to de la nota, el moment i la velocitat amb què es toca. [25]

A la Il·lustració 41 es pot veure un exemple de *MIDI Events* registrats per una aplicació (MIDI-OX) que captura els esdeveniments MIDI que rep l'ordinador.



Il·lustració 41. MIDI Events registrats per la aplicació MIDI-OX

A través dels ports MIDI, que son canals lògics o físics, es per on es permet fer la transmissió de missatges MIDI entre dispositius. Tradicionalment, els ports eren físics, però avui dia és habitual treballar amb ports virtuals que s'executen dins l'ordinador.



Il·lustració 42. Connectors MIDI físics

Els ports poden ser d'entrada (*in*), sortida (*out*) o de pas (*thru*):

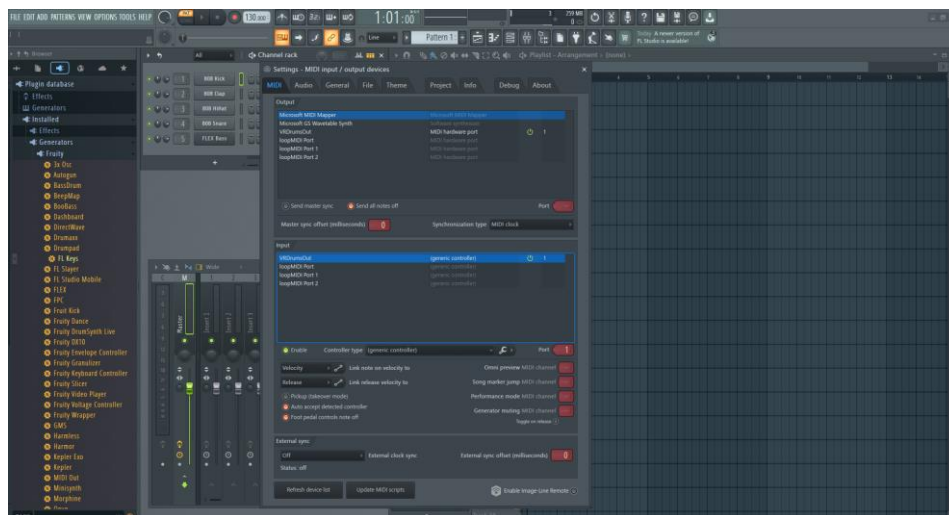
- MIDI IN: rep missatges MIDI (per exemple, d'un teclat o bateria electrònica).
- MIDI OUT: envia missatges MIDI cap a un altre dispositiu o aplicació.
- MIDI THRU: replica els missatges rebuts pel port IN i els envia cap a un altre dispositiu, permetent crear cadenes d'aparells.

2.3.2 MIDI i DAWs

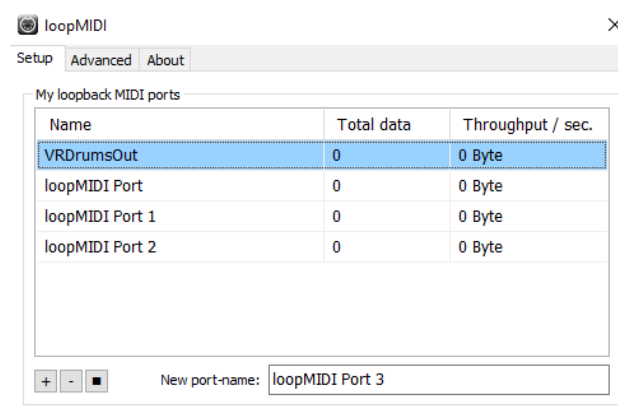
Un DAW (de l'anglès Digital Audio Workstation) és un dispositiu electrònic o un programari utilitzat per enregistrar, editar i produir fitxers d'àudio. Els DAWs existeixen en una gran varietat de configuracions: des d'un sol programa instal·lat en un portàtil, fins a una unitat independent integrada, o bé una configuració altament complexa amb nombrosos components controlats per un ordinador central. [27]

Els DAWs integren control MIDI, com a eina fonamental per tocar instruments virtuals, editar seqüències i automatitzar paràmetres com el volum, efectes, entre altres.

En la Il·lustració 43 es pot veure un exemple de la configuració MIDI d'un DAW, en aquest cas FL Studio 2024. Aquí es pot veure com per la entrada (*input*) utilitzem el port anomenat VRDrumsOut creat amb l'aplicació LoopMIDI com es pot observar en la Il·lustració 44.



Il·lustració 43. Configuració MIDI del DAW FLStudio 2024



Il·lustració 44. Ports MIDI virtuals creats amb loopMIDI

Per això la connexió entre l'aplicació i un DAW serà possible gràcies a un port MIDI virtual en el que s'enviaran *MIDI Events* que sortiran del MIDI Out que estigui configurat en el programa i entraran pel MIDI In del DAW.

2.3.3 DryWetMidi

Per poder fer possible la integració del llenguatge MIDI dins de Unity s'ha utilitzat la llibreria DryWetMidi.



Il·lustració 45. Logo de DryWetMIDI. Font: <https://melanchall.github.io/drywetmidi/>

DryWetMIDI és una biblioteca .NET per treballar amb dades i dispositius MIDI, desenvolupada per Melanchall. Permet llegir, escriure i crear fitxers MIDI (Standard MIDI Files, SMF), gestionar esdeveniments MIDI, manejar dispositius d'entrada i sortida, reproduir i gravar dades MIDI, així com realitzar operacions avançades com quantització, treball amb notes o generar patrons musicals. [28]

Es pot integrar fàcilment descarregant l'asset oficial des de l'Asset Store de Unity o bé afegint manualment la biblioteca al projecte (per exemple, des de GitHub o mitjançant NuGet).

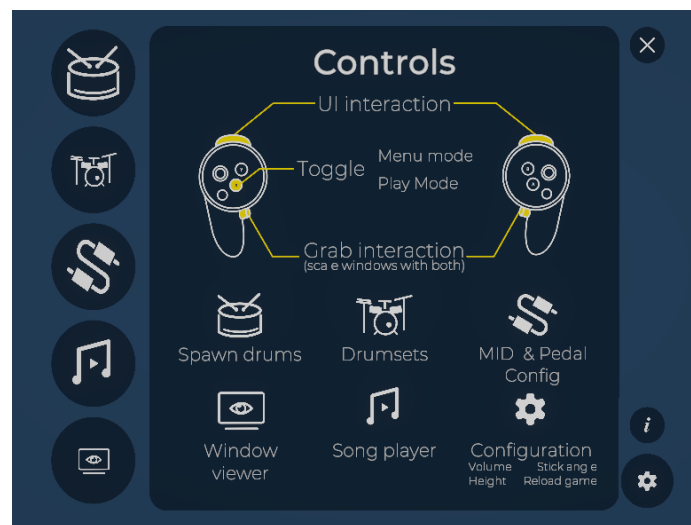
De les opcions que integra DryWetMidi les que més interessants per al projecte son:

- Entrades i sortides MIDI: mitjançant InputDevice i OutputDevice, es poden rebre dades de dispositius externs (com bateria electrònica) o enviar-les al DAW, facilitant una integració bidireccional total
- Lectura de cançons MIDI: permet importar fitxers MIDI i obtenir esdeveniments com note on i note off, sincronitzant la pràctica amb les peces musicals reals.
- Detecció d'esdeveniments en temps real: es poden detectar i respondre a l'inici o final d'una nota durant la reproducció, sincronitzant accions (com visualitzacions o sons) amb el ritme

3 Contingut de l'aplicació

En aquest apartat s'explica d'una manera general quin es el contingut de l'aplicació actualment i fins on s'ha arribat a implementar.

El sistema actual compta amb un menú interactiu en el que s'integren totes les funcions que conté el programa i fa que es pugui tindre una navegació fàcil i intuïtiva a través dels diferents botons que es troben a la esquerra d'aquest. Aquest menú conté un apartat de "informació" on s'expliquen els detalls de com navegar i interactuar amb l'entorn.



Il·lustració 46. Apartat d'informació del menú principal

La primera pantalla del menú és la de d'un *spawner* de peces de bateria que permet instanciar fins a 16 parts diferents, cadascuna amb el seu so personalitzat. Aquestes mostres no són fixes, sinó que segons la intensitat que s'apliqui en el cop, el volum i el so escollit canvia. En alguns instruments, com els plats, també s'han afegit variacions depenent de la zona d'impacte. Per exemple, es diferencia entre el so de la campana (*bell*) i el de la vora del plat.



Il·lustració 47. Pantalla de *spawner* d'instruments

A més d'aquest *spawner* de peces individuals, s'ha creat un sistema de configuració i càrrega de *drumsets* (kits de bateria) complets, que permet escollir entre quatre kits predefinits en el sistema i fins a sis espais personalitzables on l'usuari pot guardar o instanciar les seves pròpies configuracions.



Il·lustració 48. Menú d'*spawn* de *drumkits*

Pel que fa a la pràctica amb cançons, s'ha implementat un sistema de reproducció MIDI amb un panell de control. L'usuari pot moure el temps i la velocitat de reproducció, posar la cançó en pausa, repetir-la en mode bucle i seguir la cançó de manera visual amb un panell visual on es pot veure la cançó com una partitura.



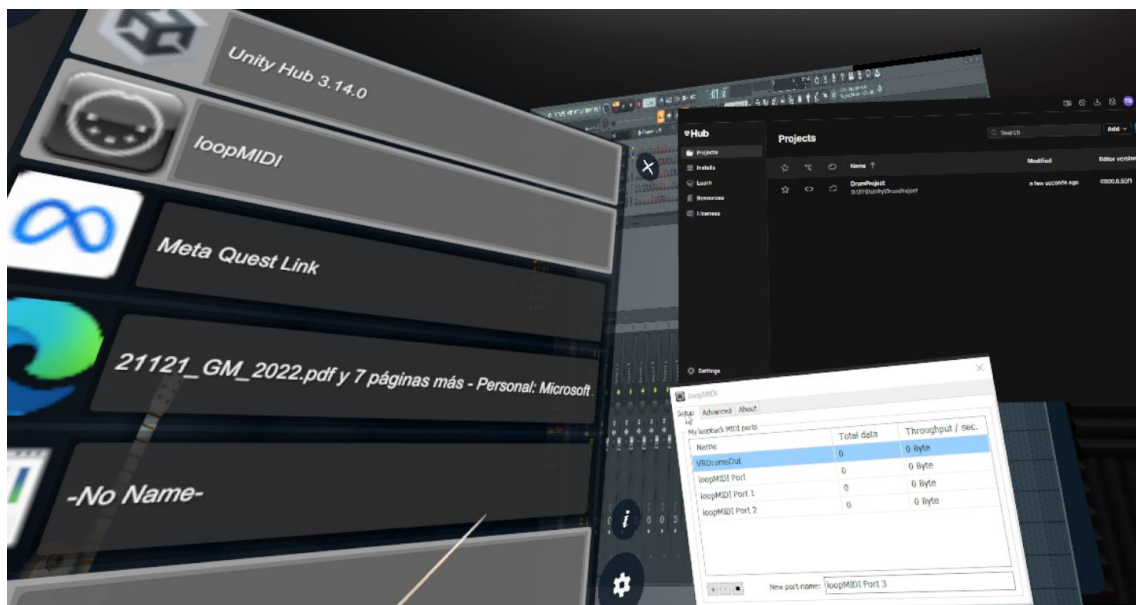
Il·lustració 49. Reproductor de cançons MIDI amb la partitura de fons

En l'àmbit de la integració amb l'exterior del programa, s'ha desenvolupat un sistema complet de MIDI *in/out* molt senzill de configurar. El sistema permet utilitzar instruments externs com a entrada del sistema, per exemple una bateria electrònica real, i, per altra banda, utilitzar la bateria virtual com a instrument MIDI de sortida, enviant les interpretacions a un DAW o a qualsevol altre programa que pugui llegir MIDI.



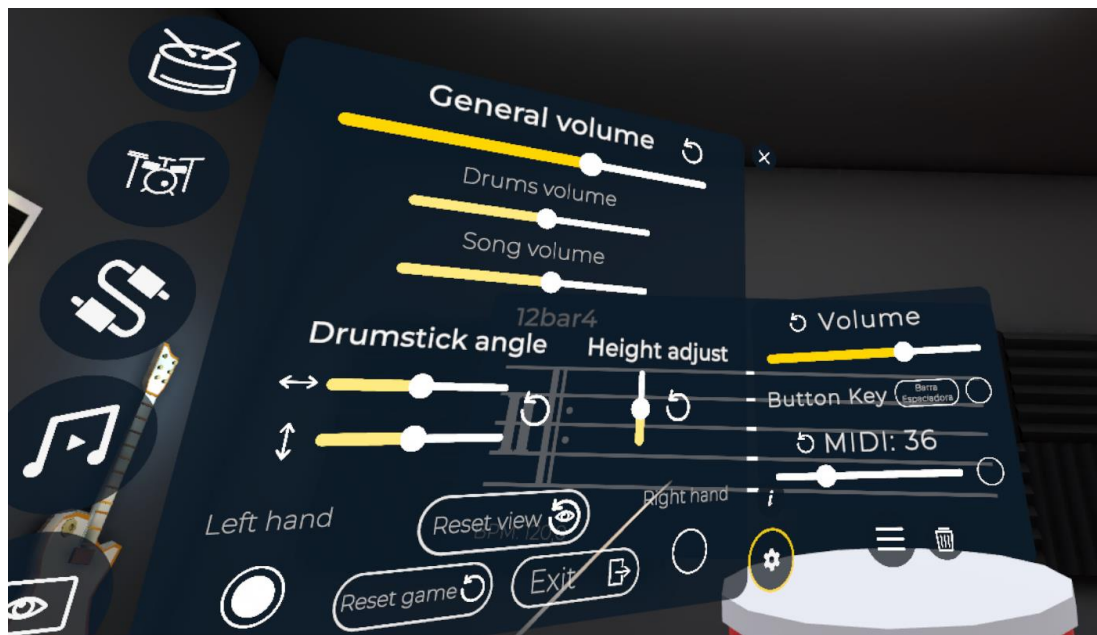
Il·lustració 50. Menú de configuració de MIDI i Pedal

Per poder fer la visualització de les diferents pantalles s'ha fet un sistema de visualització de les diferents aplicacions que es troben actualment en el ordinador on es pot elegir quina pantalla es vol visualitzar i genera una pantalla individual per a cada aplicació on es poden moure per l'espai i canviar d'escala fent que es pugui visualitzar tot des de el propi programa.



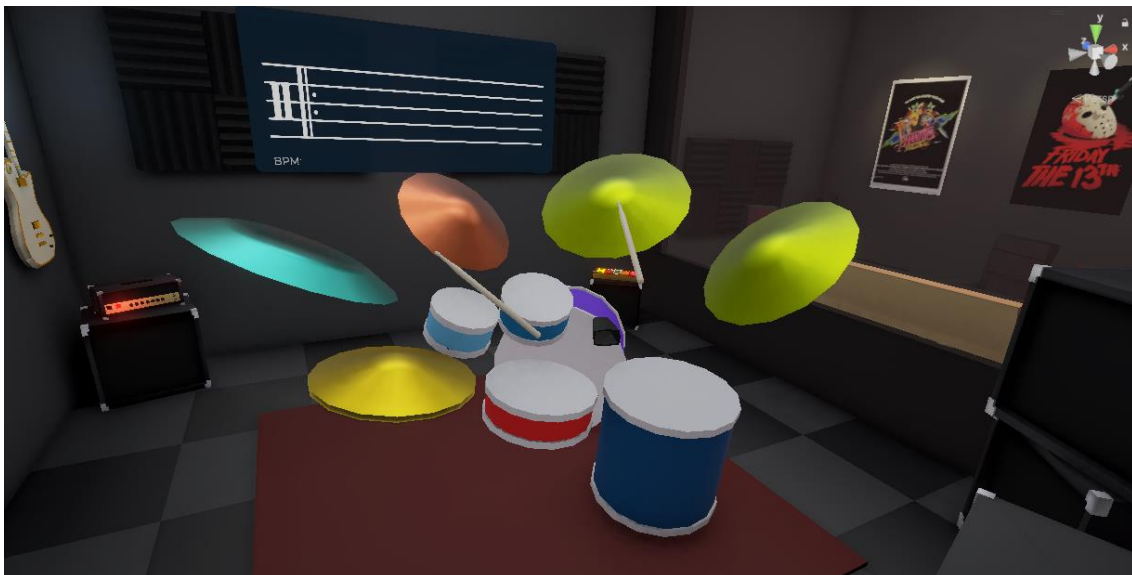
Il·lustració 51. Exemple de el window viewer amb diferents pestanyes obertes

També s'ha implementat tot un sistema de control de volums i que permet configurar les entrades MIDI individuals per a cada peça del kit. L'usuari pot ajustar el volum de cada tambor o plat, així com reassignar el número de nota MIDI associat a cada instrument.



Il·lustració 52. Exemple de la configuració del menú principal i un menú d'un instrument obert

Pel que fa a tota la part visual s'ha creat un espai on passa tot i s'ha fet amb un estil *low poly* i senzill per a poder tenir un bon rendiment i que generi la sensació de simplicitat que s'ha buscat des de un principi.



Il·lustració 53. Escena de el programa

4 Metodologia

Per a la planificació i seguiment de les diferents parts d'aquest treball s'ha optat per una metodologia organitzada i lineal però a la vegada flexible pel fet d'anar experimentant dins del propi projecte i anant creant el camí dins de la pròpia creació d'aquest.

En primer lloc, l'organització personal s'ha basat en un seguiment manual, on s'han anat anotant idees, esquemes i tasques pendents. Aquest sistema senzill ha permès ordenar el treball i tenir un registre de les prioritats en cada fase. També s'ha fet servir GitHub per gestionar el control de versions del codi. Gràcies a aquesta eina s'ha pogut mantenir un historial de canvis, crear punts segurs als quals tornar en cas d'errors i separar funcionalitats en diferents *pushes* durant tot el procés.

El projecte s'ha estructurat en fases iteratives. Inicialment es va desenvolupar un parell de prototips bàsics amb l'objectiu de comprovar la viabilitat de la idea que es tenia de simular una bateria en VR. Al tenir una base inicial, es va treballar sobre el prototip.

D'aquesta manera es com s'han anat afegint les diferents parts com són el sistema de sensibilitat del volum dels cops, la integració inicial de MIDI, la configuració de *drumsets*, el control de volums, el *Window Viewer* i el sistema de reproducció de cançons, com exemples dels grans blocs del programa. Aquest procés pas a pas ha facilitat identificar problemes i resoldre'ls sense comprometre el conjunt total del projecte.

Un element clau ha estat el poder provar el progrés del desenvolupament directament des de les ulleres de realitat virtual Meta Quest 3. Per poder fer això s'ha utilitzat Meta Quest Link, una eina que permet connectar les ulleres Meta Quest a Unity i executar el projecte en temps real dins del casc.

Pel que fa a la part visual, s'ha treballat cap al final del procés. Un cop l'estructura funcional ha arribat a un punt on ja es podia considerar com una base sòlida, s'ha començat a dissenyar i implementar la millora visual. Per a la creació dels recursos s'han utilitzat les següents aplicacions: Adobe Illustrator per al disseny d'ícones i elements gràfics de la interfície, 3ds Max per als primers models 3D, que posteriorment s'han refinat i finalitzat amb Blender.

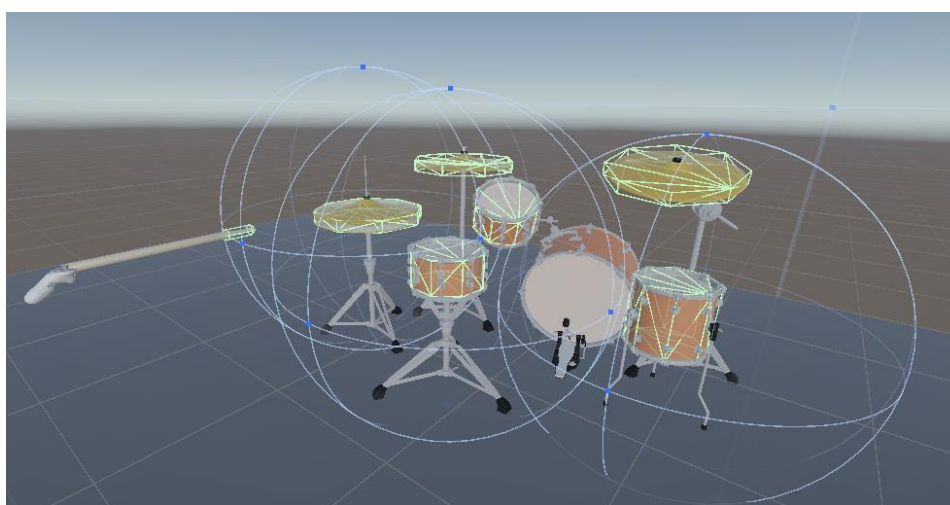
Durant el procés del projecte s'ha comptat amb el feedback de diferents persones que han anat donant idees i punts a millorar a partir de provar el programa durant diferents fases. Donant un *feedback* que s'ha anat recollint per la implementació de les idees del projecte i també per saber quines eines d'accessibilitat afegir en el programa per a que sigui el més ampli i intuïtiu possible.

5 Desenvolupament

5.1 Idees inicials i prototip

La idea inicial d'aquest projecte va sortir d'una proposta de projecte de TFG proposada per un professor que tenia la idea de crear un instrument en realitat virtual que pogués funcionar com a instrument MIDI, va ser l'inici que va donar lloc a la idea del projecte. A partir d'aquesta idea es va pensar tot el potencial que això podria tenir i ampliar-ho fins on ha arribat.

Per poder comprovar que era un projecte plausible es va crear un primer prototip que consistia en modelar una bateria senzilla amb una configuració fixa de caixa, bombo, toms i plats. L'objectiu principal d'aquest primer prototip era comprovar la viabilitat tècnica de la proposta.



Il·lustració 54. Primer prototip de una bateria amb colliders

Per implementar aquesta funcionalitat inicial es va optar per un sistema de detecció basat en *colliders* i les físiques de Unity, tal i com es pot veure a la Il·lustració 54.

Les baquetes tenien el seu propi *collider* que feien que quan una baqueta entrava en contacte amb el *collider* d'un tambor o d'un plat, es feia trigger i es generava el so corresponent gràcies a un script (a la Il·lustració 55 es pot veure el fragment en el que s'activa el so en *OnTriggerEnter*).

```

@ Mensaje de Unity | 0 referencias
private void OnTriggerEnter(Collider other)
{
    if(other.tag == "DrumStickHead")
    {
        source.volume = other.gameObject.GetComponent<TrackSpeed>().speed;
        ActivateSound();
    }
}

3 referencias
private void ActivateSound()
{
    source.Play();
}

```

Il·lustració 55. Fragment de l'script d'activació del so *OnTriggerEnter*

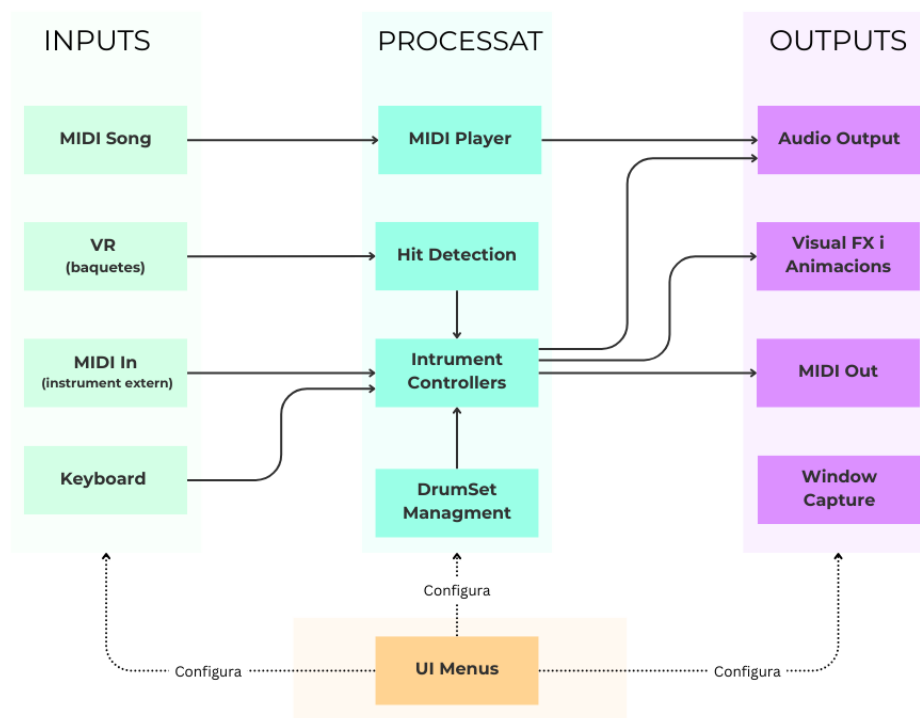
Tot i que el mecanisme permetia una interacció bàsica, van sorgir problemes: es produïen falsos positius, la mesura de la velocitat del cop era imprecisa i resultava difícil ajustar una resposta coherent amb la intensitat i el punt de l'impacte.

Malgrat aquests problemes, aquest primer prototip va ser clau perquè va permetre obtenir una demostració mínima viable que era la de una bateria virtual que reaccionava als cops de les baquetes i reproduïa sons, obrint la porta a seguir explorant noves vies tècniques i a partir d'aquí desenvolupar el projecte.

5.2 Arquitectura general

A partir de les conclusions del primer prototip (5.1), el projecte va evolucionar cap a una idea més completa que era la creació d'una bateria amb un nou sistema de detecció de cops, que l'instrument fos modular i configurable i que totes les funcionalitats es puguin seleccionar i configurar amb un menú principal ampliable.

Per tenir una idea global de com ha estat estructurat finalment el programa s'ha fet aquest diagrama d'alt nivell de l'arquitectura general del sistema (Il·lustració 56).

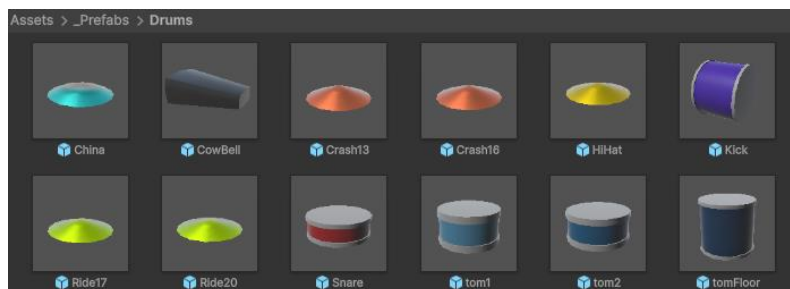


Il·lustració 56. Diagrama d'arquitectura del sistema

Aquest diagrama està dividit en tres grans blocs que són els Inputs, el bloc de processat i els outputs. Els **inputs** són les fonts d'informació que entren al sistema, que passen la informació al bloc de **processat** i que finalment al bloc de **outputs** es poden percebre al programa. També trobem el bloc de **UI Menus** que són tots els menús interactius que actuen com la capa de configuració dels paràmetres, selecció de dispositius o disposició dels instruments.

5.3 Instruments i interaccions

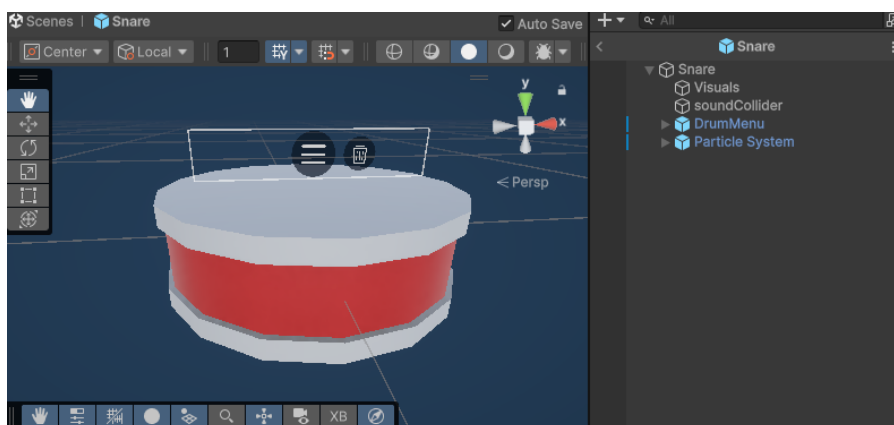
En aquest apartat s'explica com estan construïts els instruments i com es fa la interacció amb ells. Per començar, a la Il·lustració 57 es pot observar els diferents instruments que estan implementats dins del sistema, com hem vist a l'apartat 2.2.2.3, cada instrument està conformat per un *prefab* que dins d'ell conté els diferents components que fan possible el seu funcionament.



Il·lustració 57. Prefabs dels diferents instruments implementats

Els instruments poden ser activats mitjançant cops físics dins de la VR, per entrades de teclat o bé per entrades MIDI externes. Sempre que son activats envien un *MIDI Event* que pot ser enviat pel port *MIDI Out* del sistema.

Per posar el primer exemple d'un instrument utilitzarem l'*Snare*, com un exemple de com son els **tambors**.



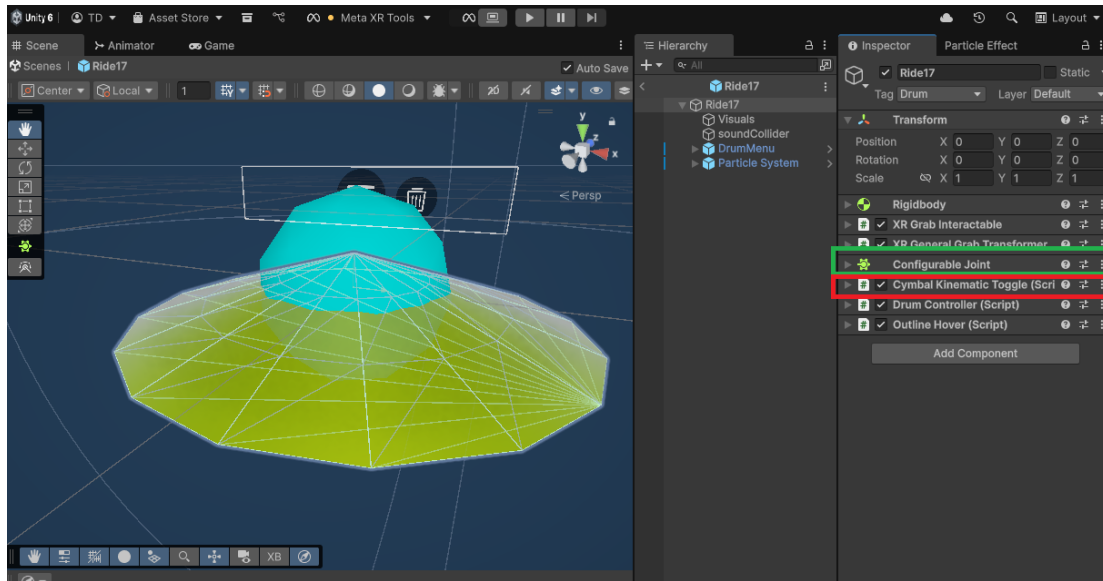
Il·lustració 58. Snare.prefab

Com es pot observar, dins del *prefab Snare*, trobem els següents *GameObjects*:

- Snare: conté els components de XR Interactable (vegeu 2.2.4) per poder ser mogut pels *interactors* i un *DrumController* que té el control de tots els components del instrument.
- Visuals: conté els components de visualització i animacions (vegeu apartat 5.3.5).
- soundCollider: conté tots els components de gestió de cops i sons de l'instrument (vegeu apartats 5.3.1 i 0).
- DrumMenu: menú individual on es pot configurar cada instrument (vegeu apartat 5.3.6).
- ParticleSystem: la partícula activada cada cop que es rep un input. (vegeu apartat 5.3.5)

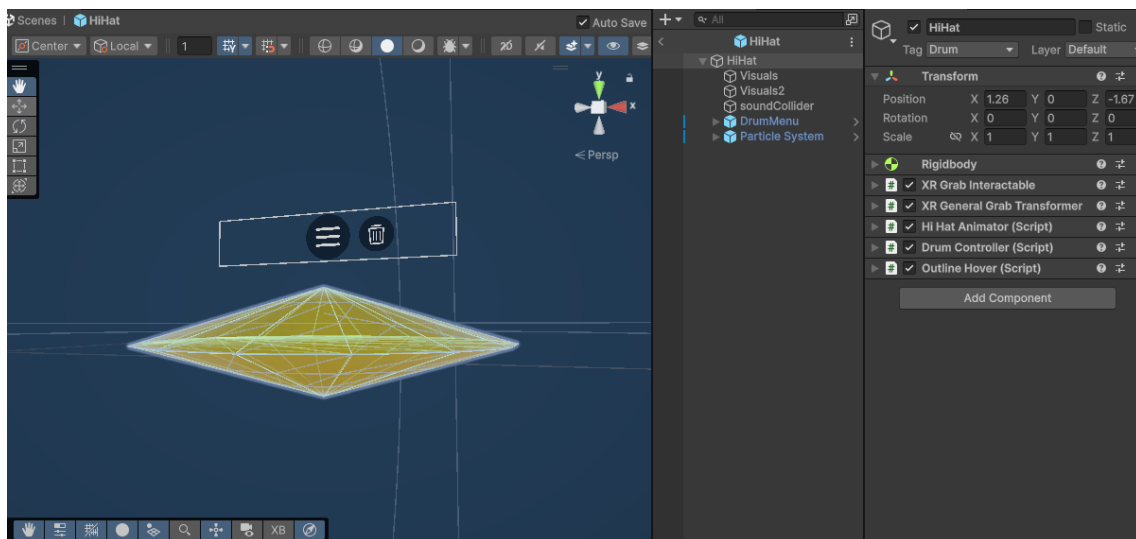
Els **plats** (*Crash, Ride o China*) presenten una complexitat extra pel seu comportament físic ja que reaccionen movent-se quan els colpegen. Això es per que a més dels components que tenen els tambors, els plats tenen un component *ConfigurableJoint* que fa possible aquest efecte gràcies a les físiques de Unity.

Al activar les físiques de Unity per poder utilitzar el *Joint* s'ha hagut de crear un script que permeti canviar de mode *kinematic* (sense físiques) a mode físiques per a que no hi haguessin errors amb el canvi de posició del *Joint* i els *Interactors*. En la Il·lustració 59 es pot observar un *prefab* d'un plat amb el seu *Configurable Joint* (en verd) i el *CymbalKinematicToggle* (en vermell).



Il·lustració 59. Exemple del *prefab* Ride17 amb un *ConfigurableJoint* i *CymbalKinematicToggle*

Per últim, el **hi-hat** és l'instrument amb més lògica específica. El seu *prefab* conté uns components específics per poder tancar, obrir i reproduir els sons específics per a cada estat del instrument. Com es pot observar en la Il·lustració 60 el *prefab* té dos *GameObjects* pels visuals a més de els components específics del instrument. Per més detall vegeu 5.3.7.

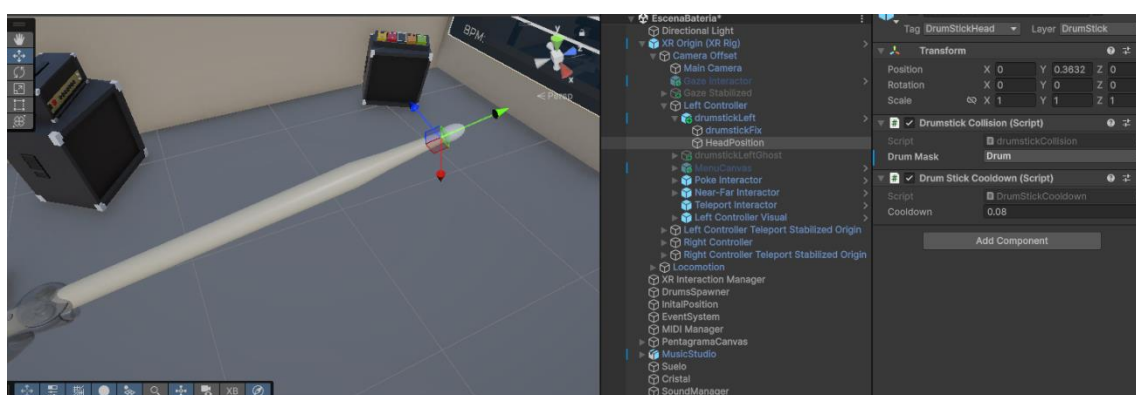


Il·lustració 60. Prefab hi-hat

5.3.1 Detecció de cops en VR

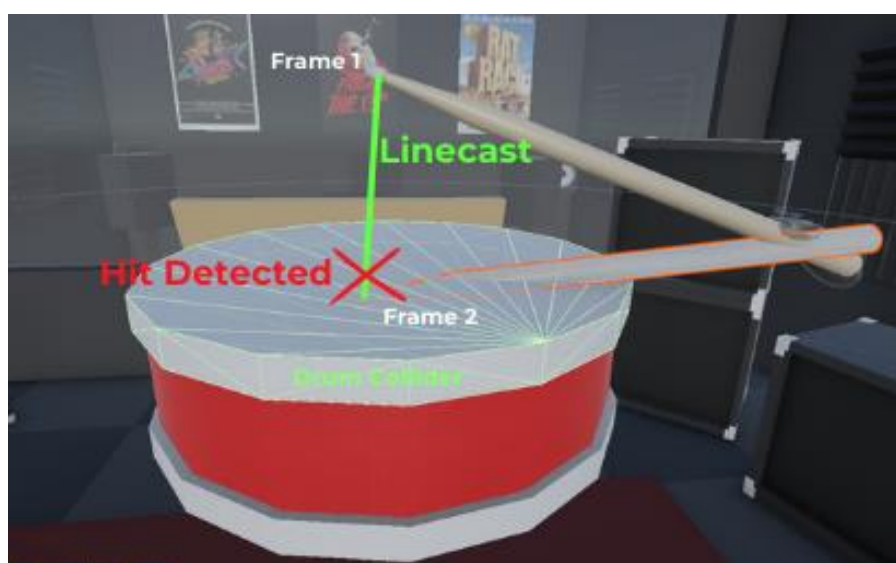
Ara s'explicarà com es fa la detecció dels cops als instruments en l'espai de VR a través de les baquetes virtuals amb una detecció coherent dels impactes i una resposta estable.

Les baquetes virtuals són els *prefabs* que simulen unes baquetes de bateria dins de l'entorn de VR. Aquestes baquetes segueixen el moviment dels controladors al ser col·locades com a filles del *LeftController* i el *Right Controller* del *XR Origin* (vegeu 2.2.4) com es pot observar a la Il·lustració 61. A la punta de cada baqueta hi ha un *GameObject* anomenat *Head Position* que conté dos scripts: *DrumstickCollision* i *DrumStickCooldown*. En aquest punt de la baqueta es on es fa la comprovació de si la baqueta ha impactat amb algun instrument.



Il·lustració 61. Baqueta virtual col·locada en l'escena amb el *HeadPosition* seleccionat

La idea per a que les baquetes puguin detectar el cop es anar comprovant el moviment de la baqueta en cada *frame* i gràcies a el sistema de *Physics.Linecast* (que crea una línia entre dos punts i comprova si ha hagut col·lisió amb algun objecte) comprovar si la baqueta ha travessat a algun instrument tal i com es pot veure a la Il·lustració 62.



Il·lustració 62. Esquema de la trajectòria de la baqueta entre dos frames i la detecció a través de linecast

L'script *DrumStickCollision* és el que s'encarrega d'això. Com es pot observar a la Il·lustració 63, entre *frame* i *frame* (en el *Update()*) comprova si ha travessat algun *collider* amb la *LayerMask* d'un instrument i si es així, calcula la velocitat d'impacte, gràcies a saber la trajectòria que ha fet i el temps que ha trigat, i envia la informació de la velocitat i posició del cop al *hitDetector* del instrument objectiu del impacte.

```
public class DrumstickCollision : MonoBehaviour
{
    private Vector3 lastPosition;
    private bool wasHitting = false;

    [SerializeField] private LayerMask drumMask;

    private DrumstickController drumstickController;
    private DrumStickCooldown cooldown;

    // Mensaje de Unity | 0 referencias
    void Start()
    {
        // Mensaje de Unity | 0 referencias
        void Update()
        {
            if (Physics.Linecast(lastPosition, transform.position, out RaycastHit hit, drumMask))
            {
                if (!wasHitting && cooldown.CanHit())
                {
                    Vector3 velocity = (transform.position - lastPosition) / Time.deltaTime;
                    float speed = velocity.magnitude;

                    DrumController drumController = hit.collider.GetComponentInParent<DrumController>();
                    HitDetector hitDetector = drumController.hitDetector;
                    hitDetector.DrumHit(velocity, speed, hit.point);

                    cooldown.RegisterHit();
                }
                wasHitting = true;
            }
            else
            {
                wasHitting = false;
            }

            lastPosition = transform.position;
        }
    }
}
```

Il·lustració 63. Update de *DrumStickCollision*

L'altre component de les baquetes es el *DrumStickCooldown* (vegeu Il·lustració 64) que té una funció molt senzilla que es la de guardar el temps del darrer impacte i no permetre'n cap de nou fins que hagi passat un interval mínim de temps. D'aquesta manera no es tenen dobles cops no desitjats per errors de seguiment o petits rebots.

```
public class DrumStickCooldown : MonoBehaviour
{
    public float cooldown = 0.08f;
    private float lastHitTime = -999f;

    1 referencia
    public bool CanHit()
    {
        return Time.time - lastHitTime >= cooldown;
    }

    1 referencia
    public void RegisterHit()
    {
        lastHitTime = Time.time;
    }
}
```

Il·lustració 64. Script *DrumStickCooldown*

Els instruments, per la seva part, tenen el component *HitDetector* ja mencionat, específicament un component que deriva d'aquest anomenat *DrumHitDetector* o *HiHatHitDetector* (específic pels *hi-hats*).

Quan un cop es detectat, el *DrumHitDetector* pren la velocitat i el lloc d'impacte per la funció de *DrumHit()* i fa els càlculs que comproven si el cop és vàlid, activa la resposta sonora (vegeu **¡Error! Marcador no definido.¡Error! No se encuentra el origen de la referencia.**), la resposta visual (vegeu 5.3.5) i envia una senyal de *NoteOn* pel *MIDI Out*.

Com es pot observar a la Il·lustració 65, es comprova quin ha sigut l'angle en el que s'ha fet l'impacte (configurat normalment a 45 graus) i si aquest no entra en el rang, fa que el cop no sigui vàlid.

```
public override void DrumHit(Vector3 velocity, float speed, Vector3 hitPoint)
{
    Vector3 normal = (normalReference != null) ? normalReference.up : transform.up;
    float dot = Vector3.Dot(velocity.normalized, -normal);
    if (dot < minDotAngle || speed < minSpeed)
    {
        return;
    }
}
```

Il·lustració 65. Fragment de la funció *DrumHit*

En el cas de que l'instrument sigui un **plat**, es a dir que l'instrument té físiques de moviment, s'aplica també una força sobre el seu *Rigidbody*, de manera que es genera el moviment oscil·latori que reproduceix el comportament d'un plat real. Al fragment de codi de la **¡Error! No se encuentra el origen de la referencia.** es pot veure com s'aplica aquesta força a través de la funció *AddForceAtPosition()*.

```
if (hasPhysics)
{
    GetComponentInParent<Rigidbody>()?.AddForceAtPosition(velocity * 0.1f, hitPoint, ForceMode.Impulse);
}
```

Il·lustració 66. Fragment de *DrumHit()* on s'aplica la força si l'instrument té físiques

Tot això mencionat funciona igual per a tots els instruments menys per al **hi-hat**. A diferència dels altres instruments aquest té un sistema més complex al tindre diferents estats: obert, tancat o tancar ràpidament. Per veure el seu cas especial vegeu apartat 5.3.7.

Així es com es gestiona la detecció dels cops als instruments a través de les baquetes virtuals. La manera de com aquests cops es transformen en resposta sonora i animació de l'instrument s'explica amb més detall als apartats **¡Error! No se encuentra el origen de la referencia.** i 5.3.5 respectivament.

5.3.2 Entrada MIDI

A més dels cops físics amb les baquetes virtuals, els instruments també poden rebre senyals des de dispositius MIDI externs. La gestió d'entrades MIDI externes es fa a través de la llibreria DryWetMIDI (vegeu 2.3.3), que permet detectar quan arriba un *MIDI Event* d'un dispositiu d'entrada i tenir la informació com la nota i la velocitat del cop. Aquesta informació és rep al sistema pel *MidiManager* i distribuïda als instruments corresponents.

En la Il·lustració 67 es pot veure el fragment del *MidiManager* que rep el *MIDI Event* de l'instrument extern i es comprova si es tracta d'una nota d'entrada (*NoteOnEvent*). Si la nota rebuda té una velocitat major que 0 es considera un cop vàlid (una velocitat de 0 equival a un *NoteOff*, és a dir, deixar de tocar la nota). Tot seguit, es crida *OnNoteOnReceived*, que notifica la resta del sistema que hi ha hagut un cop.

```
2 referencias
private void HandleMidiEvent(object sender, MidiEventReceivedEventArgs e)
{
    if (e.Event is NoteOnEvent noteOn && noteOn.Velocity > 0)
    {
        Debug.Log($"NOTE ON: {noteOn.NoteNumber} Velocity: {noteOn.Velocity}");
        OnNoteOnReceived?.Invoke(noteOn);
    }
}
```

Il·lustració 67. *HandleMidiEvent* del script *MidiManager*

Aquest esdeveniment el rep *MidiReceiver* (dins de cada instrument), ja que esta subscrit a aquests esdeveniments al *Start()*, i comprova que la nota que s'ha rebut és la assignada al instrument. Un cop comprovat, envia la informació de velocitat a *DrumHitFromMidi()*.

```
Mensaje de Unity | 0 referencias
void Start()
{
    detector = GetComponent<HitDetector>();
    dispatcher = UnityMainThreadDispatcher.Instance();
    targetMidiNote = defaultMidiNote;

    MidiManager.Instance.OnNoteOnReceived += OnNoteOn;
}

2 referencias
private void OnNoteOn(NoteOnEvent note)
{
    if (note.NoteNumber == targetMidiNote && isMidi)
    {
        dispatcher.Enqueue(() =>
        {
            detector.DrumHitFromMidi(note.Velocity);
        });
    }
}
```

Il·lustració 68. Codi del *MidiReceiver*

Com es pot observar, apareix un component que es el *dispatcher*. En DryWetMidi els *callbacks* de MIDI arriben en un *thread* diferent del *thread* principal i això és un problema perquè la majoria de mètodes només poden executar-se des del *main thread*. Per resoldre-ho, s'utilitza un *UnityMainThreadDispatcher* que envia la crida de MIDI al *main thread*. D'aquesta manera, quan el *dispatcher.Enqueue()* rep el *MIDI Event*, aquesta queda pendent i s'executa de forma segura al següent *frame* dins del *loop* principal de Unity.

La funció *DrumHitFromMidi()* (Il·lustració 69) funciona de manera molt semblant a *DrumHit()* vist a l'apartat anterior, però en comptes de calcular la velocitat a partir de la trajectòria de la baqueta, utilitza la *velocity* del *MIDI Event*.

```
public override void DrumHitFromMidi(int velocity)
{
    float normalized = Mathf.InverseLerp(1f, 127f, velocity);
    float volume = Mathf.Clamp(normalized * maxVolume * volumeMultiplier, 0f, maxVolume);
```

Il·lustració 69. Funció *DrumHitFromMidi*

Finalment, cal mencionar que es pot canviar la nota d'entrada de cada instrument gràcies a canviar el *targetMidiNote* a través del menú de cada instrument, vegeu apartat 5.3.6.

Igual que amb el cas de el sistema de detecció de cops virtuals, el *hi-hat* també té el seu propi *DrumHitFromMidi()* que diferencia els diferents estats que pot tindre, vegeu apartat 5.3.7.

Així es com es gestiona la detecció dels *MIDI Events* de els instruments externs al programa. La manera de com aquestes entrades es transformen en resposta sonora i animació de l'instrument s'explica amb més detall als apartats 5.3.3 i 5.3.4 respectivament.

5.3.3 Entrada per teclat

Una altra manera d'activar els instruments és a través del teclat de l'ordinador o de pedals USB que poden servir per simular un pedal de bateria real sense disposar d'una bateria electrònica.

Cada instrument té la possibilitat de tenir assignada una tecla concreta que quan l'usuari la prem s'activa l'instrument. A la Il·lustració 70 es pot veure com això es possible gràcies al component *KeyTrigger* que quan es prem la tecla assignada i està permès rebre inputs de teclat, crida a la funció *DrumHitFromMidi()*, ja explicada a l'apartat anterior (5.3.2).

```
public void SetAction(InputAction action)
{
    Debug.Log("set actioned ->" + action);
    drumAction = action;

    drumAction.performed += ctx =>
    {
        if (buttonActivation)
        {
            hitDetector.DrumHitFromMidi(100);
        }
    };
}
```

Il·lustració 70. Funció *SetAction* del component *KeyTrigger*

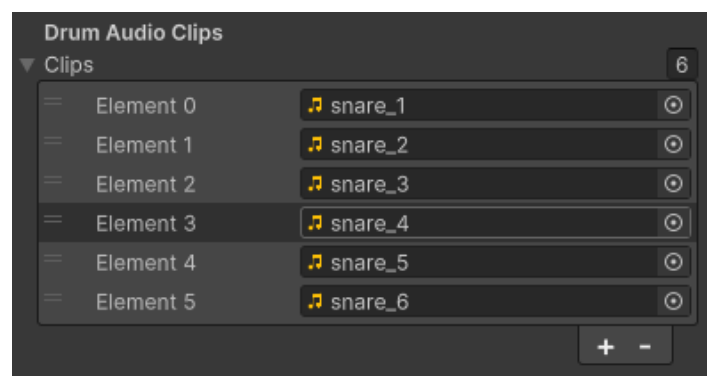
En el cas del *hi-hat*, la gestió de l'entrada per teclat o pedal és especialment important, ja que aquest instrument necessita informació extra determinar el seu estat (obert, tancat o *splash*). Per més informació vegeu 5.3.7.

5.3.4 Resposta sonora

Un cop hem vist com es poden rebre inputs que activen als instruments, aquests han de poder generar el so per a que funcioni com un instrument real. Aquest procés es el mateix per a tots els instruments i formes de entrada, en excepció del *hi-hat*, que com ja s'ha vist es un instrument especial.

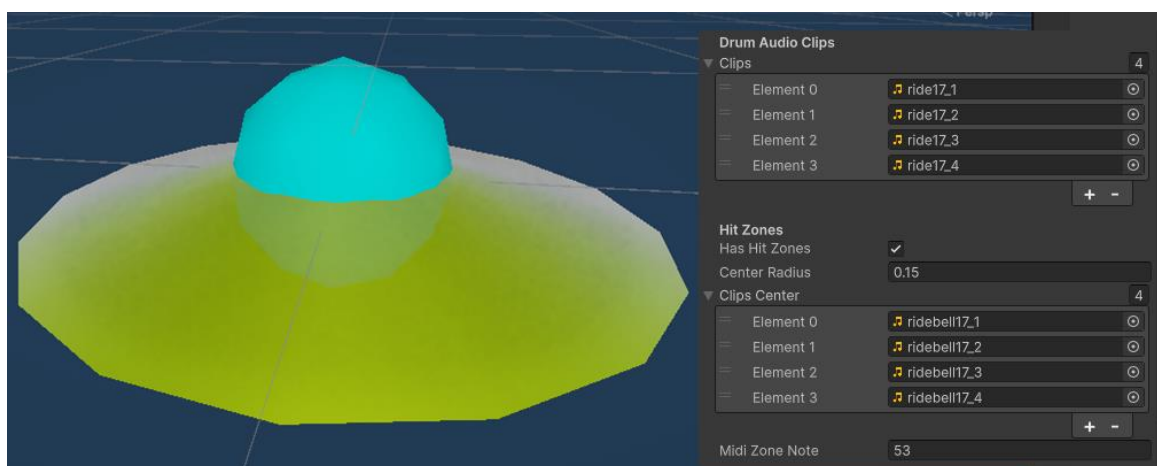
Per donar un toc més de realisme al so que generen els instruments no s'utilitza només un únic clip d'àudio, sinó diversos *samples* disponibles per a diferents intensitats i variacions. Aquesta és la pràctica que s'utilitza per a instruments MIDI reals i es el que té millors resultats.

A la Il·lustració 71 es pot veure com a l'instrument *Snare* hi ha 6 clips d'àudio diferents per elegir.



Il·lustració 71. Clips d'àudio del instrument *Snare*

A més, per als instruments que tenen més d'una zona d'impacte amb sons diferents, com son els *Ride*, es pot afegir la opció *HasHitZones* i afegir sons extrems per a una zona d'impacte especifica. A la Il·lustració 72 es pot veure la zona d'impacte del *bell* en color blau i la configuració amb el *HasHitZones* i els clips d'àudio extra.



Il·lustració 72. *Ride* amb els seus clips d'àudio i zones diferents

Llavors, un cop rebut l'impacte es fa la crida al `DrumHit()` com ja hem vist als apartats anteriors i en aquesta funció es on es fa la elecció del so que es reproduceix.

A la Il·lustració 73 es pot veure la secció de codi que fa l'elecció del clip que es reproduirà. Per decidir quin clip concret reproduir, primer es calcula la velocitat normalitzada del cop (*normalized*), que es un valor entre 0 i 1 que reflecteix amb quina força s'ha tocat. Aquest valor es multiplica pel nombre total de clips disponibles per obtenir l'índex aproximat del clip que s'utilitzarà. Després, aquest índex es limita amb *Mathf.Clamp* per assegurar que sempre estigui dins dels marges vàlids de la llista de clips.

```
float normalized = Mathf.InverseLerp(minSpeed, maxSpeed, speed);
float volume = Mathf.Clamp(normalized * maxVolume * volumeMultiplier, 0f, maxVolume);
int midiVelocity = Mathf.Clamp(Mathf.RoundToInt(normalized * 127f), 1, 127);

AudioClip clipToPlay = null;
if (hasHitZones)
{
    float distance = Vector3.Distance(transform.position, hitPoint);
    if (distance <= centerRadius)
    {
        clipVel = Mathf.FloorToInt(normalized * (clipsCenter.Count - 1));
        clipVel = Mathf.Clamp(clipVel, 0, clipsCenter.Count - 1);
        clipToPlay = clipsCenter[clipVel];
    }
    else
    {
        clipVel = Mathf.FloorToInt(normalized * (clips.Count - 1));
        clipVel = Mathf.Clamp(clipVel, 0, clips.Count - 1);
        clipToPlay = clips[clipVel];
    }
}
else
{
    clipVel = Mathf.FloorToInt(normalized * (clips.Count - 1));
    clipVel = Mathf.Clamp(clipVel, 0, clips.Count - 1);
    clipToPlay = clips[clipVel];
}
```

Il·lustració 73. Fragment de codi que escull l'àudio a reproduir

En el cas que l'instrument tingui activades les zones d'impacte (*hasHitZones*) es calcula la distància entre el punt exacte del cop (*hitPoint*) i el centre de l'instrument i depenent de si es troba dins del radi o fora, s'escull quins clips es reproduiran.

En el cas de que l'input hagi sigut a través de teclat o de midi, la comprovació de les zones d'impacte no son necessàries.

Per a que hi hagi una variació extra de so, també s'aplica una variació del pitch de forma aleatòria (*Random.Range*) i després es fa la reproducció del so amb *PlayOneShot()* tal i com es pot veure en la Il·lustració 74.

```
if (clipToPlay != null)
{
    float randomPitch = Random.Range(1f - pitchVariation/2, 1f + pitchVariation/2);
    source.pitch = randomPitch;
    source.PlayOneShot(clipToPlay, volume);
    source.pitch = 1f;
}
```

Il·lustració 74. Secció de codi de reproducció del so.

Això funciona igual per a tots els instruments a excepció dels *hi-hats* que tenen funcionalitats extres com ja hem vist en altres apartats, per més informació vegeu l'apartat 5.3.7.

5.3.5 Animació i partícules

A més del so, cada instrument genera una resposta visual cada cop que rep un cop. Aquesta resposta està composta per dos parts: una animació d'escala i una partícula que simula una ona d'impacte.

Al final de les funcions *DrumHit()* i *DrumHitFromMidi()*, ja mencionades als apartats anteriors, es fa la crida a la funció de *AnimateEscale()* del component *DrumHitAnimation()* per fer l'animació d'escala i es fa un *EmitWave()*, es a dir, es fa la crida de una partícula, tal com es pot veure a la Il·lustració 75.

```
drumHitAnimation?.AnimateScale(normalized);
MidiOutManager.Instance.SendNote(midiNote, midiVelocity);
EmitWave(normalized);
```

Il·lustració 75. Fragment final de les funcions *DrumHit()* i *DrumHitFromMidi()*

En primer lloc, la animació es calcula a partir de la intensitat de el cop per a que s'escali segons la força o intensitat del volum d'entrada. Això es fa amb el codi de *AnimateScale* que es mostra a la Il·lustració 76, on es crida a la corutina *ScaleAnimation*.

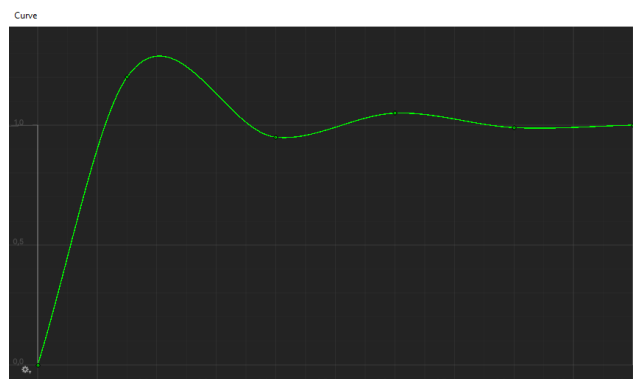
```
public void AnimateScale(float value)
{
    float scaleAmount = 1f + (maxScaleIncrease * value);

    if (currentAnim != null)
        StopCoroutine(currentAnim);

    currentAnim = StartCoroutine(ScaleAnimation(scaleAmount));
}
```

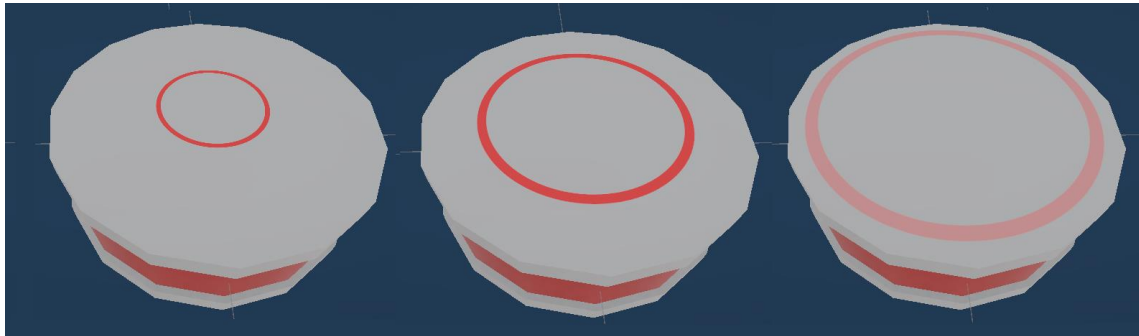
Il·lustració 76. Funció *AnimateScale*

A dins de la corutina es fa servir una *AnimationCurve*, vegeu Il·lustració 77, per interpolar el canvi d'escala: primer l'instrument creix ràpidament, després baixa i finalment torna a la mida original. Això dona la sensació que el tambor vibra o ressona quan és colpejat, reforçant visualment la sensació de cop.



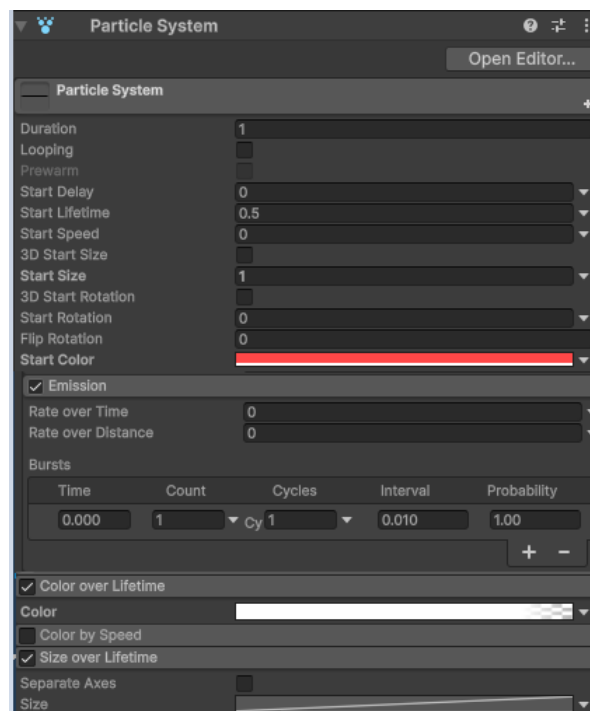
Il·lustració 77. Corba de l'animació dels instruments

Per altra banda, les partícules s'activen gràcies al `Emit()`, com s'ha comentat anteriorment. A la Il·lustració 78 es pot veure un exemple de una partícula en l'instrument *Snare*,



Il·lustració 78. Partícula d'un instrument

Per implementar aquestes partícules s'ha creat un *ParticleSystem* en el que s'ha configurat la base i s'ha fet un *prefab* a partir d'aquest. La configuració del *ParticleSystem*, mostrada a la Il·lustració 79, s'ha dissenyat per simular una ona circular expansiva que acompanya l'impacte.



Il·lustració 79. Configuració del ParticleSystem

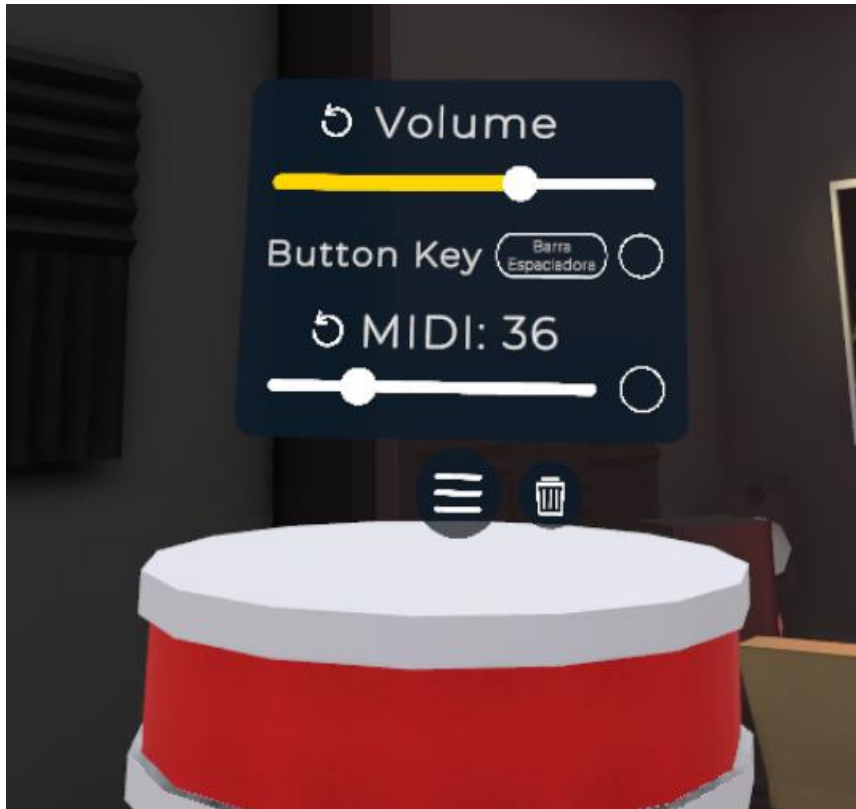
Els paràmetres clau de la configuració són:

- Start Lifetime a 0.5 segons. Per a que l'ona tingui aquesta duració.
- Size over Lifetime. Fa créixer progressivament el cercle des de 0 fins a 1 de forma normalitzada al llarg de la durada de la partícula.
- Color over Lifetime. Configurat per a que el color passi a transparent cap el final de l'animació, aconseguint un efecte de *fade out* quan l'ona desapareix.

Amb aquest feedback visual, a més del so corresponent, es com es reforça a l'usuari que s'ha produït un impacte a un instrument.

5.3.6 Menú de l'instrument

Cada instrument, sense excepció, disposa de un menú individual des de que es pot configurar els paràmetres principals d'aquests. El menú es pot obrir i tancar gràcies a una icona de tres ratlles horitzontals que es troba a sobre de cada instrument.



Il·lustració 80. Menú individual dels instruments

Com es pot veure a la Il·lustració 80, el menú està format per tres seccions principals: el volum, l'assignació de tecla (*KeyBinder*) i l'assignació de la nota MIDI. També al costat de la icona de menú hi ha un botó que permet eliminar l'instrument de l'escena.

Per al **control del volum** trobem un *Slider* que permet canviar el volum específic del instrument gràcies a que canvia el valor de *volumeMultiplier* del component *DrumHit()* i, tal com es pot veure a la Il·lustració 81, quan es fa el càlcul del volum es multiplica pel valor i dona com a resultat el volum ajustat.

```
float normalized = Mathf.InverseLerp(minspeed, maxspeed, speed);
float volume = Mathf.Clamp(normalized * maxVolume * volumeMultiplier, 0f, maxVolume);
int midiVelocity = Mathf.Clamp(Mathf.RoundToInt(normalized * 127f) - 1, 1, 127);
```

Il·lustració 81. Volum del instrument multiplicat pel valor *volumeMultiplier*

També es pot reiniciar al volum base gràcies al boto de *reset* que es troba al costat esquerra del text de *Volume*.

Per a l'assignació de tecla (**KeyBinder**) es mostra la tecla que es té seleccionada com entrada i al costat un *Toggle* que quan està activat, permet que el *ButtonKey* pugui activar a l'instrument.

Quan es prem la tecla assignada, s'entra al mode "espera de nova tecla" i el següent *input* de tecla que es registra s'assigna com a nova tecla per activar al instrument. Això es gestionat pel component *DrumKeyBinder*.

A la Il·lustració 82 es pot veure com la funció *StartRebind()* del *DrumKeyBinder* es on es fa la reassignació de tecla.

```
public void StartRebind()
{
    keyText.text = " ";
    drumAction.Disable();

    drumAction.PerformInteractiveRebinding()
        .WithControlsExcluding("Mouse")

        .WithControlsExcluding("<XRController>{RightHand}/triggerTouched")
        .WithControlsExcluding("<XRController>{RightHand}/gripTouched")
        .WithControlsExcluding("<XRController>{RightHand}/thumbrestTouched")
        .WithControlsExcluding("<XRController>{RightHand}/thumbstickTouched")
        .WithControlsExcluding("<XRController>{RightHand}/primaryTouched")
        .WithControlsExcluding("<XRController>{RightHand}/secondaryTouched")
        .WithControlsExcluding("<XRController>{RightHand}/thumbstick/x")
        .WithControlsExcluding("<XRController>{RightHand}/thumbstick/y")

        .WithControlsExcluding("<XRController>{LeftHand}/triggerTouched")
        .WithControlsExcluding("<XRController>{LeftHand}/gripTouched")
        .WithControlsExcluding("<XRController>{LeftHand}/thumbrestTouched")
        .WithControlsExcluding("<XRController>{LeftHand}/thumbstickTouched")
        .WithControlsExcluding("<XRController>{LeftHand}/primaryTouched")
        .WithControlsExcluding("<XRController>{LeftHand}/secondaryTouched")
        .WithControlsExcluding("<XRController>{LeftHand}/thumbstick/x")
        .WithControlsExcluding("<XRController>{LeftHand}/thumbstick/y")

        .OnMatchWaitForAnother(0.1f)
        .OnComplete(operation =>
        {
            drumAction.Enable();
            keyText.text = drumAction.GetBindingDisplayString();
            Debug.Log($"{drumAction.GetBindingDisplayString()}");
            operation.Dispose();
        })
        .Start();
}
```

Il·lustració 82. Funció *StartRebind()* per l'assignació de tecles

Aquest mètode *StartRebind()* primer neteja el text de la UI i desactiva l'acció perquè no es pugui executar mentre es canvia, després inicia un procés de *rebinding* en el que s'espera que l'usuari premi una nova tecla. Durant aquest procés s'exclouen controls no desitjats com el ratolí o els sensors tàctils dels controladors VR per evitar assignacions incorrectes. Quan finalment es detecta una tecla vàlida es torna a activar l'acció i s'assigna com a nova tecla.

Per últim per a la **assignació de nota MIDI** s'utilitza un *slider* que canvia el valor de *targetMidiNote* per a que més endavant el *MidiReceiver* respondrà a la nota assignada.



Il·lustració 83. Funció *OnValueChanged* del slider de canvi de *MIDI Note*

5.3.7 Cas especial: hi-hat

El *hi-hat* és un instrument que té un complexitat extra respecte als altres instruments del sistema, ja que no només depèn dels cops rebuts sinó també de l'estat d'un pedal que pot estar obert, tancat o fer un *splash*. Aquesta combinació d'*inputs* fa que el seu comportament requereixi components específics.

En el cas de la detecció de inputs, el funcionament és semblant als altres instruments però amb la diferència de que tenen en compte l'estat actual del pedal. Això és possible gràcies a l'script *PedalState* que conté el *HiHat*.

A la Il·lustració 84 es pot veure com aquest script llegeix contínuament el valor de l'entrada del pedal, detecta si el pedal s'ha premut i guarda el moment en *value*.

Quan s'allibera el pedal calcula la durada de la pressió. Si aquesta és molt curta (per sota del valor *splashThreshold*), activa l'esdeveniment *OnSplashTriggered*, que correspon a l'efecte *splash* del pedal (tancar ràpidament els plats). Si la pressió és més llarga, activa *OnClosedTriggered* i posa el valor *isPressed* a true mentre el pedal està pressionat. A més, exposa en públic el mètode *IsPedalDown()* per comprovar fàcilment des d'altres scripts si el pedal està actualment premut o no.

```
Mensaje de Unity | 0 referencias
private void Update()
{
    if (pedalInput == null) return;

    float value = pedalInput.action.ReadValue<float>();

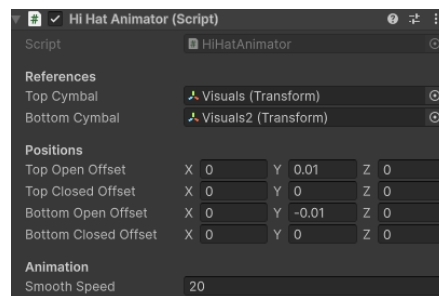
    if (!isPressed && value > 0.5f)
    {
        isPressed = true;
        pressStartTime = Time.time;
    }
    else if (isPressed && value <= 0.5f)
    {
        isPressed = false;
        float duration = Time.time - pressStartTime;

        if (duration < splashThreshold)
        {
            OnSplashTriggered?.Invoke();
        } else
        {
            OnClosedTriggered?.Invoke();
        }
    }
}

3 referencias
public bool IsPedalDown()
{
    return isPressed;
}
```

Il·lustració 84. Script *PedalState*

Un cop l'estat del pedal s'ha determinat, l'script *HiHatAnimator* (Il·lustració 85) és l'encarregat de comprovar aquest valor a cada *frame* i reflectir-lo visualment en el moviment dels plats. Els atributs d'aquest component són els dos plats del *hihat*, les posicions dels plats oberts i tancats i el paràmetre *smoothSpeed* que determina la velocitat de tancament dels plats



Il·lustració 85. Configuració del component *HiHatAnimator*

A la Il·lustració 86 es poden veure les línies principals del script *HiHatAnimator*. Aquest comprova en l'*Update()* en quina posició està el pedal. Amb això calcula una posició objectiu per als plats superior i inferior depenent de si el pedal està premut o obert.

```

Mensaje de Unity | 0 referencias
void Start()
{
    if (topCymbal != null) initialTopPos = topCymbal.localPosition;
    if (bottomCymbal != null) initialBottomPos = bottomCymbal.localPosition;
}

Mensaje de Unity | 0 referencias
void Update()
{
    if (PedalState.Instance == null) return;

    bool isDown = PedalState.Instance.IsPedalDown();

    Vector3 targetTop = initialTopPos + (isDown ? topClosedOffset : topOpenOffset);
    Vector3 targetBottom = initialBottomPos + (isDown ? bottomClosedOffset : bottomOpenOffset);

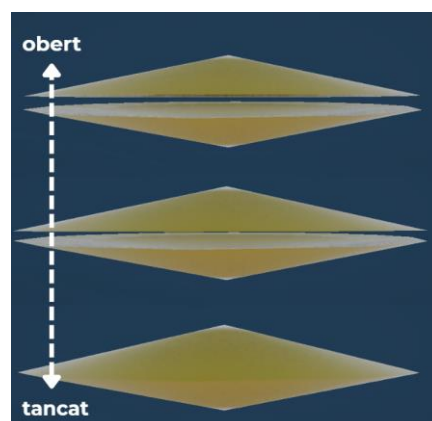
    if (topCymbal != null)
        topCymbal.localPosition = Vector3.Lerp(topCymbal.localPosition, targetTop, Time.deltaTime * smoothSpeed);

    if (bottomCymbal != null)
        bottomCymbal.localPosition = Vector3.Lerp(bottomCymbal.localPosition, targetBottom, Time.deltaTime * smoothSpeed);
}

```

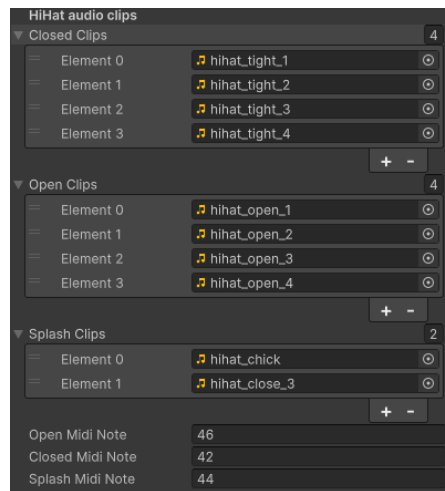
Il·lustració 86. Script *HiHatAnimator*

També interpola la posició dels plats amb *Vector3.Lerp* per a que quan el pedal es prem, els plats es tanquen a la velocitat de *smoothSpeed*, i quan es deixa de prémer el pedal tornen a la posició oberta, aconseguint una animació fluida del *hi-hat*.



Il·lustració 87. Estats del *hi-hat* amb animació

La resposta sonora del *hi-hat* també té particularitats ja que al tenir el valor del pedal, l'script *HiHatHitDetector* elegeix quin clip ha de reproduir depenent de l'estat que rep del *PedalState* gràcies a *IsPedalDown()*. A la Il·lustració 88 es pot veure els diferents clips d'àudio per cada cas.



Il·lustració 88. Clips d'àudio del *hi-hat* i els diferents MIDI notes

A la Il·lustració 89 s'escull cada clip d'àudio depenent de la variable *isPedalDown*.

```
bool isPedalDown = PedalState.Instance.IsPedalDown();
AudioClip clipToPlay;

if (isPedalDown)
{
    clipVel = Mathf.FloorToInt(Mathf.InverseLerp(1f, 127f, midiVelocity) * (closedClips.Count - 1));
    clipVel = Mathf.Clamp(clipVel, 0, closedClips.Count - 1);

    clipToPlay = closedClips[clipVel];
    midiNote = closedMidiNote;
}
else
{
    clipVel = Mathf.FloorToInt(Mathf.InverseLerp(1f, 127f, midiVelocity) * (openClips.Count - 1));
    clipVel = Mathf.Clamp(clipVel, 0, openClips.Count - 1);

    clipToPlay = openClips[clipVel];
    midiNote = openMidiNote;
}
```

Il·lustració 89. *HiHatHitDetector*

El *HiHatHitDetector* també té dos mètodes que es criden quan el pedal es tanca tant ràpidament per fer l'*splash* quan reb *OnSplashTriggered* (*SplashHit()*), i quan es tanca *OnClosedTriggered* (*PlayFootChick()*).

```
2 referencias
public void SplashHit()
{
    if (splashClips[1] == null) return;

    source.PlayOneShot(splashClips[1], 1f * volumeMultiplier);
    drumHitAnimation?.AnimateScale(1f * volumeMultiplier);
    MidiOutManager.Instance.SendNote(splashMidiNote, 100);
    EmitWave(0.5f);
}

2 referencias
private void PlayFootChick()
{
    if (splashClips[0] == null) return;
    source.PlayOneShot(closedClips[0], 0.7f * volumeMultiplier);

    MidiOutManager.Instance.SendNote(splashMidiNote, 100);
    EmitWave(0.5f);
}
```

Il·lustració 90. Funcions *SplashHit* i *PlayFootChick()*

Finalment, quan s'escull un clip per a tocar, el *HiHatHitDetector* funciona com el *DrumHitDetector*. Es a dir que fa la animació del *DrumHitAnimator* per a que el *hi-hat* tingui animació de escala, s'activa una partícula d'ona per a reforçar visualment i s'envia la nota MIDI que correspon a cada estat del pedal.

```
if (clipToPlay != null)
{
    float randomPitch = Random.Range(1f - pitchVariation / 2, 1f + pitchVariation / 2);
    source.pitch = randomPitch;
    source.PlayOneShot(clipToPlay, volume);
    source.pitch = 1f;
}

drumHitAnimation?.AnimateScale(normalized);

EmitWave(normalized);

MidiOutManager.Instance.SendNote(midiNote, midiVelocity);
```

Il·lustració 91. Secció de script que envia els outputs del *hi-hat*

5.3.8 Mapa de notes MIDI dels instruments

Cada instrument quan és activat ja sigui per un cop físic, una entrada MIDI externa o una tecla assignada, acaba enviant un esdeveniment MIDI. Aquesta sortida global fa possible connectar l'instrument virtual a qualsevol DAW o dispositiu compatible amb MIDI.

Per defecte, a cada instrument se li assigna un número de *MIDI Note* concret que segueix l'estàndard General MIDI per a kits de percussió. Aquest mapeig s'ha triat per tal que els sons siguin compatibles amb la majoria de bateries electròniques i DAWs. [29] No obstant això, totes aquestes notes es poden reassignar fàcilment des del menú individual de cada instrument (vegeu apartat 5.3.6).

A la Il·lustració 92 es mostra el mapa de correspondència entre els instruments implementats i les notes MIDI assignades per defecte.

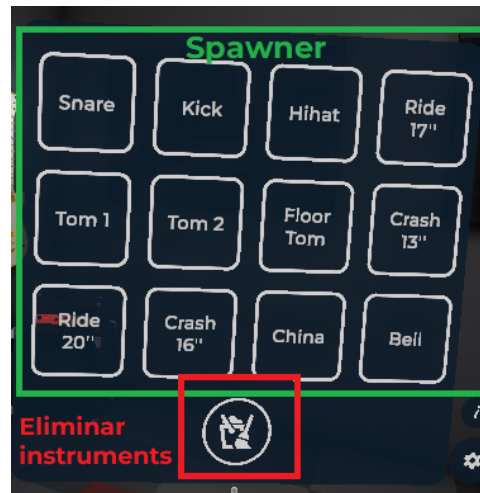
Instrument	MIDI Note	Instrument	MIDI Note
Snare	38	Ride (vora)	59
Kick	36	Ride (centre)	53
Tom 1	48	China	52
Tom 2	47	Hi-hat (tancat)	42
Floor Tom	43	Hi-hat (obert)	46
Crash	57	Hi-Hat (splash)	44

Il·lustració 92. Mapa de correspondència dels instruments i la seva nota MIDI

5.4 Funcions implementades

5.4.1 Spawn dels instruments

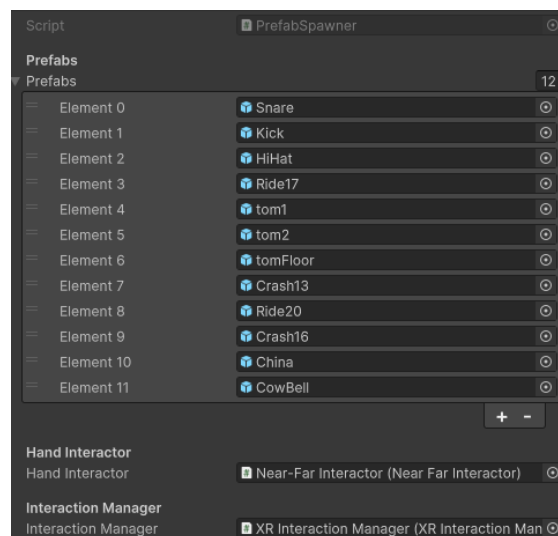
El *spawn* o creació d'instruments dins l'escena és una de les funcionalitats bàsiques ja que es el que permet a l'usuari construir i personalitzar el seu propi kit de bateria. Aquesta funció es gestiona des del menú principal, a la Il·lustració 93. Pàgina del menú d'*spawn*. es pot veure en la secció *spawner* on l'usuari pot seleccionar quin instrument vol afegir.



Il·lustració 93. Pàgina del menú d'*spawn*.

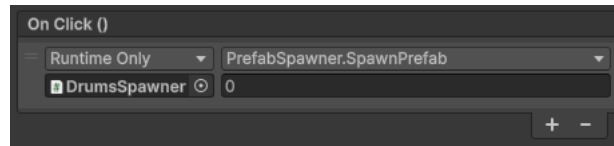
També es pot fer una neteja dels instruments instanciats en la escena amb el botó de sota de l'*spawner* a més de poder eliminar cada instrument per separat des de el menú individual.

A nivell tècnic, el procés es realitza mitjançant l'script *PrefabSpawner*. En aquest es seleccionen tots els *prefabs* dels instruments que es vulguin i s'afegeixen en la llista de *prefabs* tal i com mostra la Il·lustració 94. Component *PrefabSpawner*.

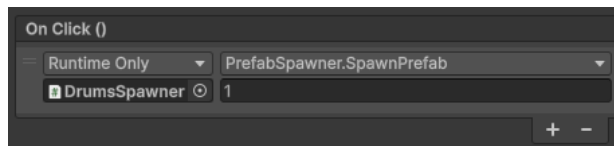


Il·lustració 94. Component *PrefabSpawner*

Un cop afegits tots els instruments al *PrefabSpawner*, en els botons del panell d'*spawn* s'assigna en el seu *OnClick()*, que s'executa cada cop que s'interacciona amb el botó, l'índex del *prefab* que es vol fer *spawn*. Tal i com es pot veure a la Il·lustració 95 i la Il·lustració 96, a cada botó se li assigna el seu índex fent així una llista que correspon a cada *prefab*.



Il·lustració 95. Mètode *OnClick()* del botó del *spawner* amb índex 0



Il·lustració 96. Mètode *OnClick()* del botó del *spawner* amb índex 1

A la Il·lustració 97 es pot observar la funció *SpawnPrefab* que es la que es crida quan es fa *click* en els botons del *spawner*. Depenent de quin índex se li ha assignat, busca quin *prefab* ha de instanciar i fa la instància amb *Instantiate()*.

```
public void SpawnPrefab(int index)
{
    if (index < 0 || index >= prefabs.Length)
    {
        return;
    }

    if (handInteractor.hasSelection)
    {
        return;
    }

    GameObject spawned = Instantiate(prefabs[index]);

    Transform attachTransform = handInteractor.attachTransform;
    spawned.transform.position = attachTransform.position;
    spawned.transform.rotation = attachTransform.rotation;

    XRGrabInteractable grabInteractable = spawned.GetComponent<XRGrabInteractable>();
    if (grabInteractable == null)
    {
        return;
    }

    interactionManager.SelectEnter(
        (IXRSelectInteractor)handInteractor,
        (IXRSelectInteractable)grabInteractable
    );
}
```

Il·lustració 97. Funció *SpawnPrefab*.

Un aspecte important és que els instruments es creen sempre en el controlador de l'usuari per a que sigui més senzilla la seva col·locació. Un cop col·locat l'usuari pot agafar-los amb els controladors i reubicar-los lliurement a l'escena.

Aquest sistema de *spawn* és flexible i permet afegir fàcilment nous instruments en el futur ja que només cal crear el *prefab* corresponent, afegir-lo al llistat de l'script i assigna-li un botó.

5.4.2 Drumsets

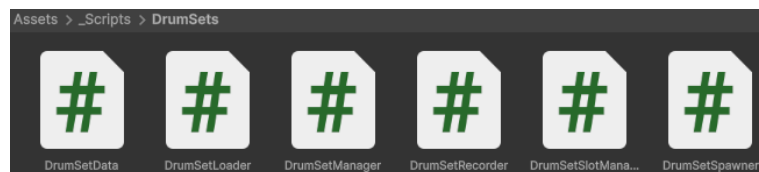
Una altra funcionalitat que té el sistema és la possibilitat de guardar i carregar kits de bateria complets, al que ens referim com a *drumsets*. A més el programa disposa de quatre kits predefinits disponibles.



Il·lustració 98. Panell de gestió dels DrumSets.

Al menú principal, tal i com es pot veure a la Il·lustració 98, es disposa d'un apartat dedicat als *drumsets*. L'usuari pot seleccionar qualsevol dels botons, que es divideixen en quatre kits fixos i sis *slots* per a kits personalitzables.

A nivell tècnic, aquest és un sistema més complex, aquesta funcionalitat es gestiona mitjançant diversos scripts que treballen conjuntament:



Il·lustració 99. Conjunt de *scripts* necessaris per al maneig dels *drumsets*.

- *DrumSetData*: és la classe que defineix l'estructura bàsica d'un kit. Guarda informació com els instruments que el componen, la seva posició i rotació dins de l'escena.
- *DrumSetManager*: és el script que té un registre global dels instruments actius a l'escena per a poder fer el maneig dels kits.
- *DrumSetRecorder*: s'encarrega de recollir la informació del kit actual present a l'escena i convertir-la en un objecte *DrumSetData*.
- *DrumSetSlotManager*: és el punt d'entrada des de la UI per carregar i guardar tots els kits, diferenciant entre *presets* i ranures personalitzables.
- *DrumSetLoader*: és l'encarregat d'agafar les dades desades i reconstruir el kit dins de l'escena instanciant cada instrument i col·locant-lo en la seva posició tant si són kits dels sistema com creats per l'usuari.
- *DrumSetSpawner*: gestiona el procés d'instanciar els instruments quan es carrega un kit complet.

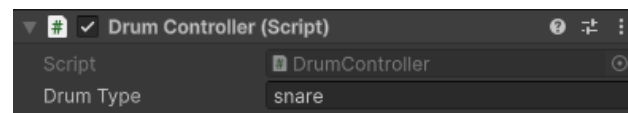
DrumData es la classe que guarda el tipus de instrument, la seva posició i la rotació. DrumsetData es el conjunt de DrumData de un sol kit.

```
[Serializable]
6 referencias
public class DrumData
{
    public string drumType;
    public Vector3 position;
    public Vector3 rotation;
}

[Serializable]
10 referencias
public class DrumsetData
{
    public string name;
    public List<DrumData> drums = new List<DrumData>();
}
```

Il·lustració 100. Classes DrumData i DrumsetData.

El DrumType s'agafa del DrumController de cada instrument tal i com es pot veure a la Il·lustració 101.



Il·lustració 102. DrumController del prefab Snare amb el seu DrumType.

Amb el DrumSetManager es manté una llista global dels instruments en la escena guardant una llista de DrumControllers. Com es pot veure a la Il·lustració 103, el DrumSetManager fa gestió de la llista gràcies als mètodes de Register, Unregister i ClearDrums, aquest últim també fa una neteja global per a que no quedin restes del kit anterior.

```
public class DrumSetManager : MonoBehaviour
{
    private static readonly List<DrumController> drums = new List<DrumController>();

    1 referencia
    public static void Register(DrumController drum)
    {
        if (!drums.Contains(drum))
            drums.Add(drum);
    }

    1 referencia
    public static void Unregister(DrumController drum)
    {
        if (drums.Contains(drum))
            drums.Remove(drum);
    }

    2 referencias
    public static List<DrumController> GetAllDrums()
    {
        return new List<DrumController>(drums);
    }

    1 referencia
    public static void ClearDrums()
    {
        foreach (DrumController drum in drums)
        {
            if (drum != null)
            {
                GameObject.Destroy(drum.gameObject);
            }
        }
        drums.Clear();
    }
}
```

Il·lustració 103. DrumSetManager.

Gràcies a l'script *DrumSetRecorder* es com es passa de una llista de *DrumControllers* al *DrumSetManager* a una sola classe *DrumSetData*. Tenint així la informació justa i necessària per a la gestió i guardat dels DrumSets.

```

// Referencia
public DrumsetData CaptureCurrentDrumset(string name)
{
    DrumsetData data = new DrumsetData();
    data.name = name;
    data.drums = new List<DrumData>();

    List<DrumController> allDrums = DrumSetManager.GetAllDrums();

    foreach (DrumController drum in allDrums)
    {
        DrumData d = new DrumData();
        d.drumType = drum.drumType;
        d.position = drum.transform.position;
        d.rotation = drum.transform.rotation.eulerAngles;

        data.drums.Add(d);
    }

    return data;
}

```

Il·lustració 104. *CaptureCurrentDumset* del script *DrumSetRecorder*.

Tenint clara aquesta gestió de la informació, ara es pot passar a com es gestiona quan l'usuari interacciona amb el panell. Això es gràcies al *DrumSetSlotManager*. Aquest component té la funció *OnSlotButtonPressed* (Il·lustració 105) que s'encarrega de detectar quan es prem un botó d'*slot*. El mètode es fixa si se està esperant a guardar un kit en algun slot (gràcies a *awaitingSaveSlotSelection*). Si es així, guarda la informació a l'*slot* corresponent. Si no ha de guardar cap kit, decideix si cal carregar un *preset* o un kit personalitzat i crida al mètode encarregat de fer la gestió en cada cas.

```

// Referencia
public void OnSlotButtonPressed(int slotIndex)
{
    if (awaitingSaveSlotSelection)
    {
        if (slotIndex > 10)
        {
            foreach (Button button in presetButtons)
            {
                button.image.color = Color.white;
            }
            SaveDrumsetToSlot(slotIndex);
            awaitingSaveSlotSelection = false;
        }
        return;
    }

    DrumSetManager.ClearDrums();

    if (slotIndex < 10)
    {
        currentDrumset = DrumsetLoader.LoadPresetFromResources(slotIndex);
    }
    else
    {
        currentDrumset = DrumsetLoader.LoadDrumsetFromSlot(slotIndex);
    }

    if (currentDrumset != null)
    {
        currentLoadedSlot = slotIndex;
        drumsetSpawner.SpawnDrumset(currentDrumset);
    }
    else...
}

```

Il·lustració 105. Funció *OnSlotButtonPressed*.

Pel procés de guardat, té la funció *EnterSaveMode()* que registra quan es selecciona el botó de guardat i s'activa el mode de guardar canviant el color dels botons disponibles i posant el booleà en *awaitingSaveSlotSelection* a *true*. Així després quan es fa la crida a *SaveDrumSetToSlot()* fa la gestió de guardar la informació en forma de *.json* a la carpeta corresponent.

```

public void EnterSaveMode()
{
    if (DrumSetManager.GetAllDrums().Count == 0) {...}

    foreach (Button button in presetButtons)
    {
        button.image.color = Color.yellow;
    }

    awaitingSaveSlotSelection = true;
}

1 referencia
private void SaveDrumsetToSlot(int slotIndex)
{
    DrumsetData newSet = drumsetRecorder.CaptureCurrentDrumset($"Slot{slotIndex}");

    string folder = Path.Combine(Application.persistentDataPath, "Drumsets");
    Directory.CreateDirectory(folder);

    string path = Path.Combine(folder, $"Slot{slotIndex}.json");
    string json = JsonUtility.ToJson(newSet, true);

    File.WriteAllText(path, json);
}

```

Il·lustració 106. Funcions de guardat del *DrumSetSlotManager*.

Això de la gestió de la informació és una part important ja que no es guarden en el mateix lloc els kits que venen fets i els personalitzats. Els kits que venen al sistema es guarden a *Assets/Resources* per a que, al fer el *build*, es mantingui la informació (Il·lustració 107). Per altra banda quan l'usuari guarda un kit personalitzat, aquest es guarda a la carpeta de *Persistent Data* (*Application.persistentDataPath*) (Il·lustració 108). Aquesta ruta és pròpia de cada sistema operatiu i fa que els arxius es mantinguin entre sessions, fins i tot després de tancar o actualitzar el programa però no ve inclòs dins del propi programa.

Nombre	Fecha de modificación	Tipo	Tamaño
pres1.json	12/09/2025 16:27	Archivo JSON	2 KB
pres1.json.meta	12/09/2025 18:25	Archivo META	1 KB
pres2.json	12/09/2025 16:27	Archivo JSON	2 KB
pres2.json.meta	12/09/2025 18:25	Archivo META	1 KB
pres3.json	12/09/2025 16:27	Archivo JSON	2 KB
pres3.json.meta	12/09/2025 18:25	Archivo META	1 KB
pres4.json	12/09/2025 16:27	Archivo JSON	2 KB
pres4.json.meta	12/09/2025 18:25	Archivo META	1 KB

Il·lustració 107. Kits prefets dins de la carpeta *Assets* del programa

Nombre	Fecha de modificación	Tipo	Tamaño
Slot11.json	12/09/2025 16:27	Archivo JSON	2 KB
Slot12.json	09/09/2025 17:06	Archivo JSON	5 KB
Slot13.json	09/09/2025 17:06	Archivo JSON	5 KB
Slot14.json	09/09/2025 17:06	Archivo JSON	5 KB

Il·lustració 108. Kits personalitzats dins de la carpeta de *Persistent Data*.

L'encarregat de carregar les dades depenent de si estan en els Assets (presets) o des de el Persistent Data (kits personalitzats), es el *DrumSetLoader*. Com es pot veure a la Il·lustració 109, les dues funcions llegeixen la informació del *.json* corresponent al *slot* i retornen la seva informació.

```

1 referencia
public static DrumsetData LoadDrumsetFromSlot(int slotIndex)
{
    string path = Path.Combine(Application.persistentDataPath, "Drumsets", $"Slot{slotIndex}.json");
    if (File.Exists(path))
    {
        string json = File.ReadAllText(path);
        return JsonUtility.FromJson<DrumsetData>(json);
    }
    return null;
}

1 referencia
public static DrumsetData LoadPresetFromResources(int index)
{
    TextAsset jsonFile = Resources.Load<TextAsset>("Drumsets/preset" + index);
    if (jsonFile != null)
    {
        return JsonUtility.FromJson<DrumsetData>(jsonFile.text);
    }
    return null;
}

```

Il·lustració 109. Funcions del *DrumSetLoader*.

Per últim, el *DrumSetSpawner*, és l'encarregat de instanciar els diferents instruments al lloc corresponent gràcies a la informació que li passen en forma de *DrumsetData*. A la Il·lustració 110 es pot veure com

```

public void SpawnDrumset(DrumsetData data)
{
    if (data == null)
    {
        Debug.LogWarning("DrumsetData es nulo.");
        return;
    }

    foreach (DrumData drum in data.drums)
    {
        if (!drumTypeToPrefab.TryGetValue(drum.drumType, out GameObject prefab)) { ... }

        GameObject instance = Instantiate(prefab, drum.position, Quaternion.Euler(drum.rotation));

        if (instance.GetComponent<DrumSpawnAnimation>() == null)
        {
            instance.AddComponent<DrumSpawnAnimation>();
        }
    }
}

```

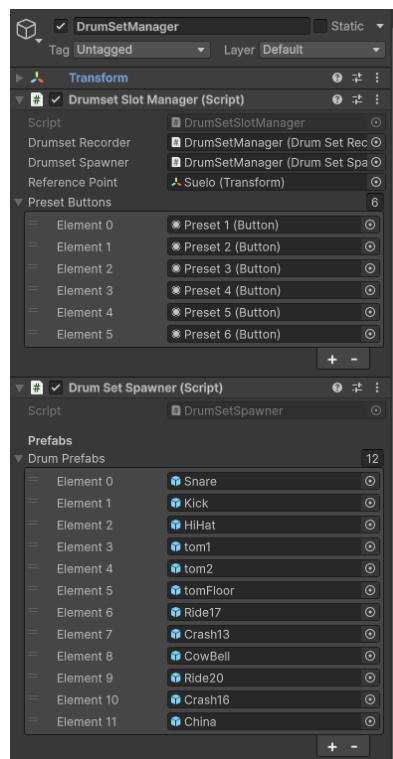
Il·lustració 110. Funció *SpawnDrumSet*.

També s'ha afegit una capa d'animació visual mitjançant l'script *DrumSpawnAnimation*, que fa que quan un instrument s'instancia fa una animació d'aparició de petit a gran amb un efecte de rebot.



Il·lustració 111. Animació d'aparició dels instruments

Un aspecte destacable d'aquest sistema es que està pensat per ser modular i escalable per poder afegir nous instruments. Tot el que s'ha de fer es afegir nous instruments al *DrumSetSpawner* i si es vol elegir algun slot més, agregar-lo al *DrumSetSlotManager* tal i com es pot veure a la Il·lustració 112.



Il·lustració 112. GameObject DrumSetManager dins de Unity amb els seus components

A la Il·lustració 113 es pot veure l'exemple del panell en acció, en aquest cas s'està en el procés de guardat i ara es farà l'elecció de en quin *slot* guardar la informació.



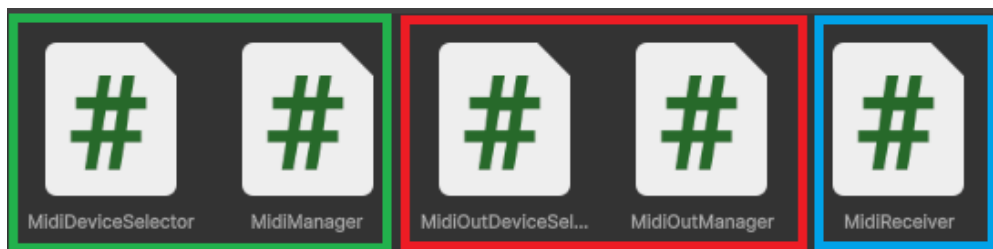
Il·lustració 113. Panell de gestió de *drumsets* esperant a guardar un *preset* de l'usuari

5.4.3 MIDI in/out

Aquesta funcionalitat consisteix en la integració completa amb dispositius MIDI, tant en entrada (MIDI In) com en sortida (MIDI Out). Aquest sistema permet que es pugui interactuar amb instruments reals, com una bateria electrònica (per exemple, l'Alesis Nitro Max), i també enviar senyals MIDI cap a programes externs com un DAW.

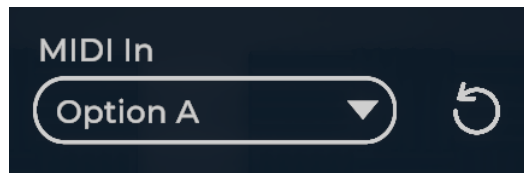
Per tal de fer-ho possible, s'ha utilitzat la llibreria DryWetMIDI de Melanchall (vegeu apartat 2.3.3), que proporciona totes les eines per detectar, llegir i enviar esdeveniments MIDI dins de Unity.

A la Il·lustració 114 es mostren els scripts que gestionen aquesta funcionalitat, dividits en tres blocs principals: MIDI In (verd), MIDI Out (vermell) i MIDI Receiver (blau).



Il·lustració 114. Scripts per a la gestió MIDI

El **MIDI In** s'encarrega de rebre els esdeveniments des d'un dispositiu d'entrada seleccionat al desplegable que pot veure a la Il·lustració 115. Això s'utilitza per tal que un cop de d'una bateria electrònica o algun altre instrument MIDI es tradueixi en un cop dins del sistema.



Il·lustració 115. Secció de MIDI In al panell del menú.

Això es possible gràcies al *MidiDeviceSelector* i al *MidiManager*. Primer, el *MidiManager* pot llegir quins són els dispositius d'entrada MIDI que existeixen en el sistema, tal i com es pot veure a la Il·lustració 116, llegeix tots els *InputDevice* (classe de *DryWetMidi*) i retorna els noms.

```
1 referencia
public List<string> GetAllInputDeviceNames()
{
    var devices = InputDevice.GetAll();
    var names = new List<string>();
    foreach (var device in devices)
    {
        if (device.Name.Contains("GS Wavetable")) continue;
        names.Add(device.Name);
    }
    return names;
}
2 referencia
```

Il·lustració 116. Funció per llegir tots els inputs MIDI connectats

Per a seleccionar quin *InputDevice* es vol tindre, el *MidiDeviceSelector* fa una llista de tots els dispositius disponibles i afegeix un *Listener* a cada opció per a que quan es seleccioni un *InputDevice* de la llista, el *MidiManager* pugui fer la selecció del Input. A la Il·lustració 117 es pot veure la funció que fa això. Un punt destacable es que es pot cridar cada cop que es prem el botó de *refresh* i així llistar un altre cop els *Inputs* actuals i actualitzar la llista.

```
2 referencias
public void RefreshDeviceList()
{
    dropdown.onValueChanged.RemoveAllListeners();
    dropdown.ClearOptions();

    List<string> deviceNames = MidiManager.Instance.GetAllInputDeviceNames();

    if (deviceNames.Count == 0)
    {
        dropdown.AddOptions(new List<string> {"No MIDI found"});
        prevDevice = null;
        return;
    }

    dropdown.AddOptions(deviceNames);

    int index = 0;
    if (!string.IsNullOrEmpty(prevDevice) && deviceNames.Contains(prevDevice))
    {
        index = deviceNames.IndexOf(prevDevice);
    }
    else
    {
        prevDevice = deviceNames[0];
    }

    dropdown.value = index;
    dropdown.RefreshShownValue();

    MidiManager.Instance.SelectDevice(prevDevice);

    dropdown.onValueChanged.AddListener(OnDropdownChanged);
}
```

Il·lustració 117. Funció *RefreshDeviceList*.

Quan es selecciona un input, el *MidiManager* fa la gestió de canviar el *InputDevice* gràcies a la funció *SelectDevice* que es pot veure a la Il·lustració 118.

```
2 referencias
public void SelectDevice(string deviceName)
{
    if (currentDevice != null)
    {
        currentDevice.EventReceived -= HandleMidiEvent;
        currentDevice.Dispose();
        currentDevice = null;
    }

    SelectedDeviceName = deviceName;

    try
    {
        currentDevice = InputDevice.GetByName(deviceName);
        currentDevice.EventReceived += HandleMidiEvent;
        currentDevice.StartEventsListening();

        Debug.Log($"MIDI Input set to: {deviceName}");
    }
    catch (Exception e)
    {
        Debug.LogError($"Error MIDI '{deviceName}': {e.Message}");
    }
}
```

Il·lustració 118. Funció *InputDevice*

Un cop es té seleccionat un *InputDevice*, ara el sistema escolta totes les entrades de senyals MIDI que genera aquest. Això es gràcies a la funció, del *MidiManager*, *HandleMidiEvent* que es pot veure a la Il·lustració 119.

```
2 referencias
private void HandleMidiEvent(object sender, MidiEventReceivedEventArgs e)
{
    if (e.Event is NoteOnEvent noteOn && noteOn.Velocity > 0)
    {
        Debug.Log($"NOTE ON: {noteOn.NoteNumber} Velocity: {noteOn.Velocity}");
        OnNoteOnReceived?.Invoke(noteOn);
    }
}
```

Il·lustració 119. Funció *HandleMidiEvent*.

Aquesta funció fa un *Invoke* a l'esdeveniment *OnNoteOnReceived* per a que pugui ser llegit pel MIDI desde qualsevol instrument. Que un instrument llegeixi les notes MIDI externes s'ha vist a l'apartat 5.3.2.

Per altra banda, el **MIDI Out** es el que permet enviar senyals des del sistema de realitat virtual cap a un dispositiu o programa extern.

Per fer això possible, s'ha de seleccionar un dispositiu de sortida. Aquest procés és igual al de la elecció de *InputDevice* però amb els scripts de *MidiOutDeviceSelector* i *MidiOutManager* que en comptes de funcionar amb *InputDevice*, funcionen amb *OutputDevice*.

També, el *MidiOutManager* té la funció *SendNote* que fa l'enviament al *OutputDevice* que està seleccionat a través de un *NoteOnEvent*. Aquesta classe de *DryWetMidi* es la que conté la informació de quina nota i a quina velocitat s'ha tocat per a que el dispositiu que rep la senyal sàpiga tota la informació que envia el programa.

```
7 referencias
public void SendNote(int noteNumber, int velocity = 100, int channel = 9)
{
    if (currentDevice == null) ...

    var noteOn = new NoteOnEvent((SevenBitNumber)noteNumber, (SevenBitNumber)velocity)
    {
        Channel = (FourBitNumber)channel
    };

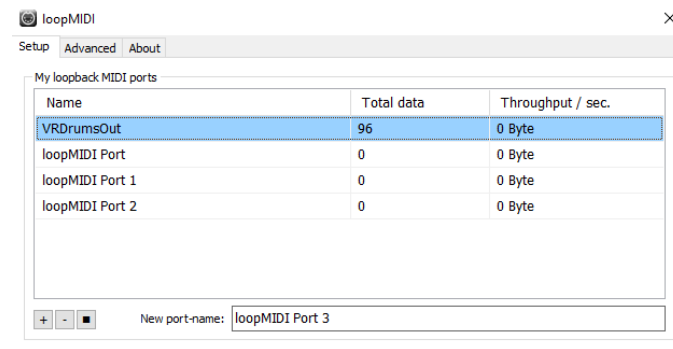
    currentDevice.SendEvent(noteOn);

    StartCoroutine(SendNoteOff(noteNumber, channel, 0.1f));
}
```

Il·lustració 120. Funció *SendNote*.

Al final de la funció es pot veure com s'envia una senyal de *SendNoteOff*. Això es per a que el dispositiu de sortida sàpiga que s'ha enviat una nota que ha acabat, ja que en el llenguatge de MIDI, si una nota no s'ha declarat com a *NoteOff*, aquesta segueix en encesa i tocant-se. La senyal de *NoteOff* sempre s'envia 0.1 segons després ja que les notes que es toquen a una bateria don cops ràpids i no tenen més duració que això.

Per establir un pont entre Unity i altres aplicacions, s'ha utilitzat i es recomana el programa LoopMIDI, que crea ports MIDI virtuals i permet redirigir les dades entre el projecte i programes externs. A la es pot veure l'exemple de la configuració amb LoopMIDI.



Il·lustració 121. Programa loopMidi configurat amb diferents ports MIDI virtuals

A la Il·lustració 122 es pot veure el panell de MIDI In / MIDI Out en acció.

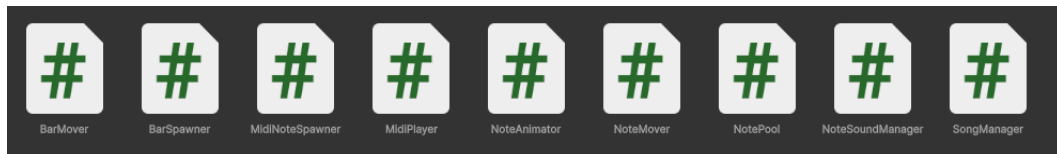


Il·lustració 122. Panell de Midiln / MidiOut configurat.

5.4.4 Cançons MIDI

Aquesta és la funcionalitat més avançada i complexa del sistema, que és la reproducció i gestió de cançons MIDI. Aquesta permet carregar arxius .mid, sincronitzar-los amb àudio, fer un control de la cançó gràcies al panell de reproducció i visualitzar les notes en el panell de visualització amb forma de pentagrama.

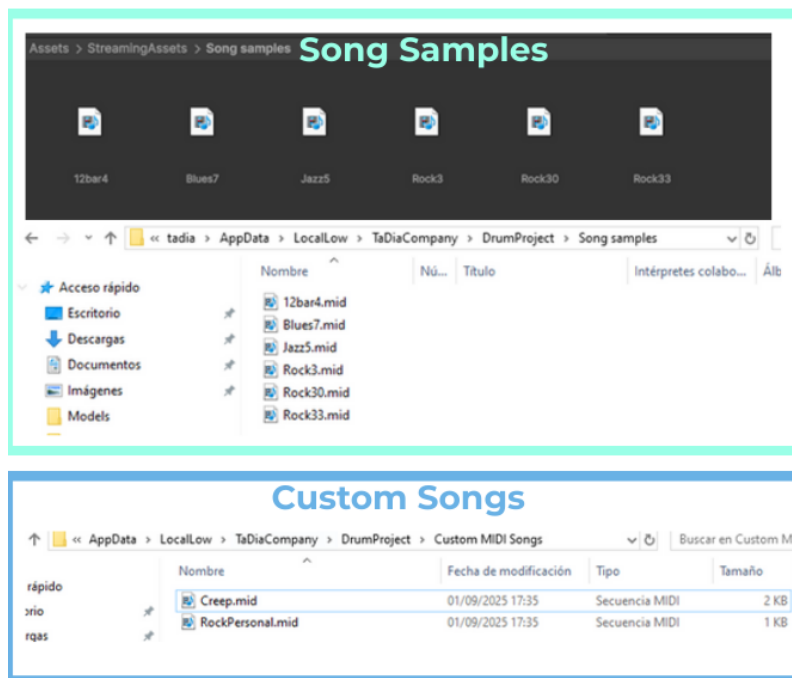
Aquesta funcionalitat també s'ha pogut gestionar gràcies a DryWetMidi (vegeu l'apartat 2.3.3). Aquests són els scripts que s'utilitzen per fer tota aquesta funcionalitat:



Il·lustració 123. Conjunt de scripts per a les cançons MIDI

En el cas de les cançons MIDI també hi ha de dos tipus tal i com hem vist amb els *drumkits* a l'apartat 5.4.2: les cançons que ja venen per defecte dins del programa i les cançons que l'usuari pot penjar amb arxius .mid.

La gestió es una mica diferent ja que les cançons MIDI no es poden guardar directament dins del programa sense que en el *build* es converteixin en arxius .bytes, que això faria que el reproductor de cançons no les pugui llegir. Llavors el procediment que es fa es que les cançons de mostra es guarden inicialment a la carpeta *StreamingAssets* i, en la primera execució del programa, el sistema les copia automàticament en format .mid a la carpeta de Persistent Data anomenada *SongSamples*. Les cançons personalitzades, en per altra banda, es desen a la carpeta Custom MIDI Songs de la carpeta de Persistent Data.



Il·lustració 124. Carpetes de guardat de les cançons

Per poder seleccionar les cançons disponibles s'utilitza la funció *RefreshSongsDropdown* de l'script *SongManager* que llista totes les cançons disponibles i les separa en dues seccions: *custom songs* i *song samples*.

```

2 Referències
public void RefreshSongsDropdown()
{
    options.Clear();
    songsDropdown.ClearOptions();

    var custom = Directory.Exists(customFolderPath)
        ? Directory.GetFiles(customFolderPath, "*.mid").OrderBy(Path.GetFileName).ToList()
        : new List<string>();

    var samples = Directory.Exists(samplesFolderPath)
        ? Directory.GetFiles(samplesFolderPath, "*.mid").OrderBy(Path.GetFileName).ToList()
        : new List<string>();

    options.Add(new OptionEntry { isHeader = true, label = "- Custom songs -" });
    foreach (var p in custom)
        options.Add(new OptionEntry { isHeader = false, label = Path.GetFileNameWithoutExtension(p), path = p });

    options.Add(new OptionEntry { isHeader = true, label = "- Song samples -" });
    foreach (var p in samples)
        options.Add(new OptionEntry { isHeader = false, label = Path.GetFileNameWithoutExtension(p), path = p });

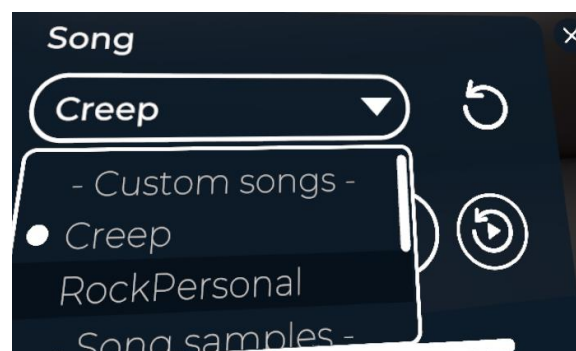
    songsDropdown.AddOptions(options.Select(o => o.label).ToList());

    suppressChange = true;
    songsDropdown.value = 0;
    suppressChange = false;
}

```

Il·lustració 125. Funció *RefreshSongsDropdown*.

Així es com queda la llista en el panell de cançons:



Il·lustració 126. Panell de selecció de cançons.

Quan es selecciona una cançó de la llista, es crida a *HandleSelection* que fa la comprovació de quina ha sigut la cançó seleccionada i fa el *Invoke* de l'esdeveniment *OnSongSelected* que podrà ser escoltat per altres components.

```

2 Referències
void HandleSelection(int index)
{
    if (suppressChange) return;
    if (index < 0 || index >= options.Count) return;

    var opt = options[index];
    if (opt.isHeader)
    {
        suppressChange = true;
        songsDropdown.SetValueWithoutNotify(lastValidIndex >= 0 ? lastValidIndex : 0);
        suppressChange = false;
        songsDropdown.Show();
        return;
    }

    lastValidIndex = index;
    if (!string.IsNullOrEmpty(opt.path))
    {
        if (songTitle != null)
        {
            songTitle.text = Path.GetFileNameWithoutExtension(opt.path);
        }
        OnSongSelected?.Invoke(opt.path);
    }
}

```

Il·lustració 127. Funció *HandleSelection*.

El component que rep l'esdeveniment de quina cançó està seleccionada i fa tota la gestió de la reproducció de les cançons es el *MidiPlayer*. Aquest mètode és el cor del sistema de reproducció. En el moment que rep l'esdeveniment *OnSongSelected*, es carrega el fitxer .mid corresponent i s'inicialitza un objecte Playback de la llibreria *DryWetMIDI* que permet avançar la cançó en temps real i accedir a tota la seva informació musical (notes, compàs, tempo, entre altres).

```
1 referencia
void LoadSong(string path)
{
    DisposePlayback();

    midiFile = MidiFile.Read(path);
    totalDuration = midiFile.GetDuration<MetricTimeSpan>();

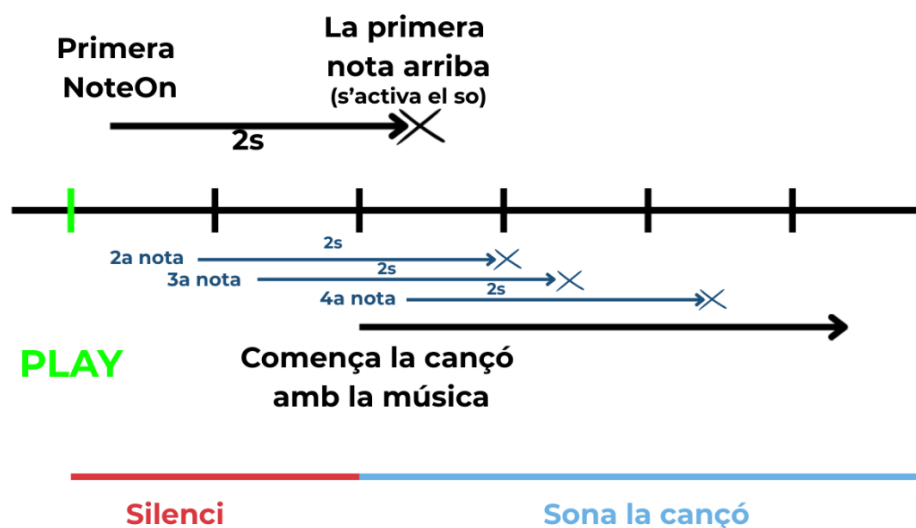
    var settings = new PlaybackSettings
    {
        ClockSettings = new MidiClockSettings { CreateTickGeneratorCallback = () => null }
    };
    playback = midiFile.GetPlayback(settings);
}
```

Il·lustració 128. Fragment del script *LoadSong*.

Aquest component també gestiona la sincronització amb l'àudio associat a la cançó. Si al mateix directori que el .mid hi ha un fitxer .wav o .mp3 amb el mateix nom, s'afegeix aquest àudio a l'*AudioSource* per a que pugui sonar al mateix temps que la lectura del MIDI.

Abans d'explicar el funcionament complet del reproductor de cançons MIDI, és important entendre el flux de temps que es segueix des del moment que es prem el botó Play fins que realment sona la música dins del sistema.

Per a que tot el sistema estigui sincronitzat i l'usuari pugui veure les notes que sonaran abans amb un temps de marge per a poder saber com segueix la cançó, a l'estil dels jocs de ritme, s'ha pensat la següent estratègia per a la sincronització:



Il·lustració 129. Esquema de la sincronització de les cançons

- En prémer Play, s'activa la cançó i es on es reproduceix el Playback de DryWetMidi i es van llegint les notes que van apareixent sense activar el so.
- Als dos segons de haver donat al Play, comença a sonar la musica de fons i es sincronitza.
- Al passar del temps, es van llegint les notes, les notes viatgen dos segons fins al seu destí on allà s'activen i s'activa el seu so pertinent.

A la pràctica, el panell de visualització (Il·lustració 130), que està fet en forma de pentagrama, es veu així, on les notes van de dreta a esquerra fins que desapareixen al cap de dos segons en el seu punt final:



Il·lustració 130. Temps aplicat al panell visual

Un aspecte tècnic rellevant és que els esdeveniments MIDI arriben a través del *thread* del dispositiu MIDI (gestionat per la llibreria DryWetMIDI), mentre que la lògica de joc i les actualitzacions visuals han de córrer dins del *thread* principal de Unity.

Per aquestes raons i per assegurar una experiència fluida i sincronitzada entre el que es veu i el que s'escolta, en el sistema s'utilitza el rellotge d'àudio intern de Unity (*AudioSettings.dspTime*). Aquest rellotge és d'alta precisió i no depèn dels *frames* per segon. Això vol dir que les notes que apareixen amb anticipació i es desplacen cap al punt final, segueixen el camí interpolat calculat amb *dspTime*. Així, encara que hi hagi petites variacions en l'Update loop de Unity, la posició i moment de la nota sempre coincideixen amb el rellotge d'àudio.

Això es pot veure reflectit en l'script de *NoteMover* (Il·lustració 131), encarregat de moure les notes cap al seu destí, on la seva posició depèn de *AudioSettings.dspTime*.

```

Mensaje de Unity | 0 referencias
void Update()
{
    double dspNow = AudioSettings.dspTime;
    float t = (float)((dspNow - dspSpawnTime) / (dspHitTime - dspSpawnTime));

    if (t >= 1f)
        HitTarget();
    else
        transform.position = Vector3.Lerp(start, target, t);
}

```

Il·lustració 131. Update del *NoteMover*.

En aquest mateix script es on s'activa el so cada cop que una nota arriba al seu destí:

```

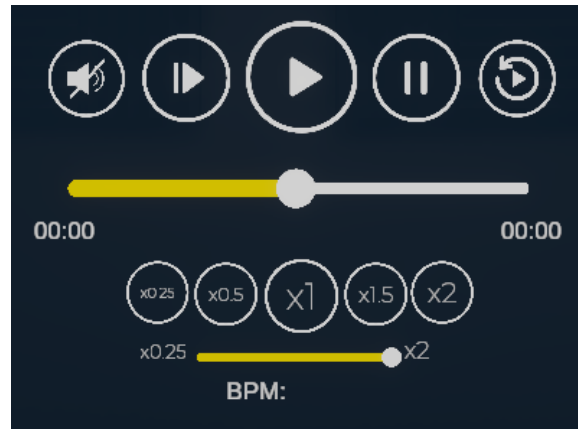
private void HitTarget()
{
    if (clips != null && clips.Length > 0)
    {
        float normalized = Mathf.InverseLerp(1f, 127f, noteVelocity);
        int index = Mathf.FloorToInt(normalized * (clips.Length - 1));
        index = Mathf.Clamp(index, 0, clips.Length - 1);
        var clip = clips[index];
        if (clip != null)
            NoteSoundManager.Instance.PlayClip(clip, normalized);
    }
    NotePool.Instance.Release(noteName, gameObject);
}

```

Il·lustració 132. Funció *HitTarget* del *NoteMover*

Un cop s'ha vist com funciona tota la sincronització de les notes amb la cançó i amb el sistema de Unity, podem passar al control que es proporciona de les cançons a través de la interfície.

A la Il·lustració 133 es pot veure el panell de reproducció amb les funcions que té el *MidiPlayer*.



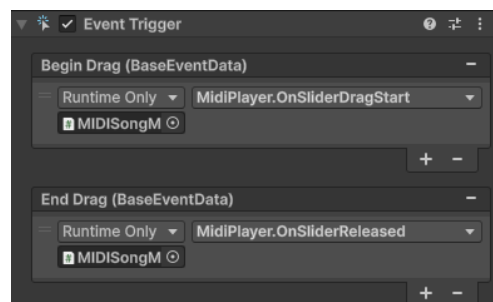
Il·lustració 133. Panell del reproductor de cançons

El *MidiPlayer* es qui proporciona aquests controls de reproducció:

- *PlaySong()*: reproduceix la cançó des del punt actual de la cançó.
- *PlayFromStart()*: reinicia la reproducció des del principi i reseteja els comptadors.
- *PauseSong()*: atura la reproducció i desa el temps actual per poder reprendre-la desde aquest punt.
- *Loop*: amb el Toggle de la UI es pot activar o desactivar la reproducció en bucle quan la cançó arriba al final.
- Canviar la posició en el temps de la cançó amb *OnSliderDragStart()* i *OnSliderReleased()*.
- *SetSpeed()*: permet canviar la velocitat de reproducció, amb opcions predefinides de 0.25x, 0.5x, 1x, 1.5x o 2x o a través d'un *slider*.

Aquestes funcions utilitzen mètodes que proporciona *DryWetMidi* per al control del Playback amb personalitzacions per a poder aplicar-los als botons.

El control més complex del panell de reproducció és el *slider* de temps, que permet a l'usuari avançar o retrocedir manualment dins de la cançó. A nivell tècnic, aquest control requereix més que simplement moure la posició del playback, ja que cal garantir que la música, els esdeveniments MIDI i les notes visuals quedin de nou totalment sincronitzats. Per això al *slider* se li agreguen els següents mètodes amb el component *EventTrigger*:



Il·lustració 134. Event trigger del slider de temps

Quan l'usuari comença a arrossegar el slider (*Begin Drag*) es crida a *OnSliderDragStart* (Il·lustració 135). Aquesta funció fa que es faci pausa el Playback i es marqui l'estat de *seeking* (buscant). Això evita que el reproductor segueixi avançant mentre l'usuari encara no ha decidit el nou punt.

```
public void OnSliderDragStart(BaseEventData data)
{
    isDraggingSlider = true;
    wasRunningBeforeDrag = playback != null && playback.IsRunning;
    if (playback != null && wasRunningBeforeDrag)
        playback.Stop();
    if (currentTimeText) currentTimeText.text = FormatMs(timelineSlider.value);
    isPaused = !wasRunningBeforeDrag;
}
```

Il·lustració 135. Funció *OnSliderDragStart*.

Quan l'usuari deixa anar el slider (*End Drag*), es crida a *OnSliderReleased* (Il·lustració 136). En aquesta funció es on es calcula el nou temps objectiu dins de la cançó a partir del percentatge de la barra. El Playback es mou a aquest punt (*MoveToTime*) i s'actualitzen els comptadors interns.

```
public void OnSliderReleased(BaseEventData data)
{
    if (playback == null)
    {
        isDraggingSlider = false;
        return;
    }

    double ms = timelineSlider.value;
    var targetTime = new MetricTimeSpan(0, 0, (int)(ms / 1000), (int)(ms % 1000));
    playback.MoveToTime(targetTime);
    pausedTime = targetTime;

    if (currentTimeText) currentTimeText.text = FormatMs(ms);
    isDraggingSlider = false;

    if (wasRunningBeforeDrag)
    {
        isPaused = false;
        playback.Start();
    }
    else
    {
        isPaused = true;
    }
}
```

Il·lustració 136. Funció *OnSliderReleased*.

Un cop reposicionat, es recalcula el *startDspTime* perquè el rellotge intern de Unity torni a estar alineat amb el nou punt de la cançó. Si hi ha una pista d'àudio externa associada, es programa de nou amb *audioSource.PlayScheduled()* perquè soni exactament a temps.

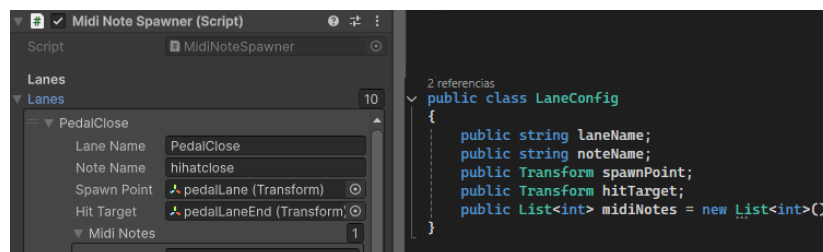
Quan el *MidiPlayer* reproduïx la cançó, cada vegada que detecta un *NoteEvent* envia el senyal *OnNoteSpawn*. Aquest esdeveniment és rebut pel component *MidiNoteSpawner*, que s'encarrega de crear les notes visuals per a que es vegi en el pentagrama.

Cada nota sap quin recorregut ha de fer gràcies a saber quin és el seu punt inicial d'*spawn* i quin és el seu punt final. A la Il·lustració 137, es pot veure com dins del *canvas* del pentagrama es troben dins de les *Lanes* els punts d'inici i de final que determinen el recorregut de cada nota.



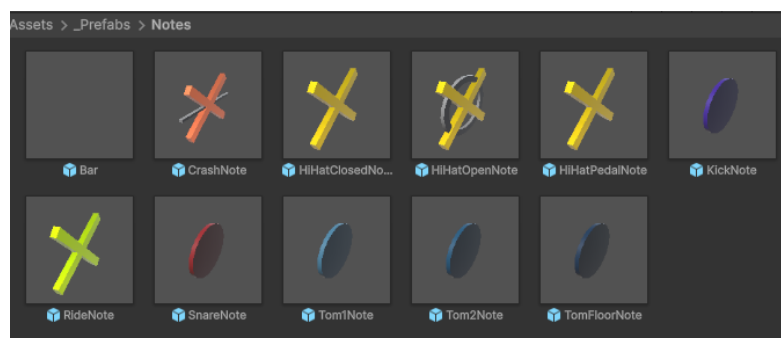
Il·lustració 137. Pentagrama amb les seves lanes

El procés consisteix a determinar en quina línia s'ha de generar la nota segons el valor MIDI. Això es fa gràcies a que a la definició del component *MidiNoteSpawner*, es determina la informació necessària amb la classe *LaneConfig* passant els punts del recorregut, el nom de la nota i a quina nota MIDI correspon:



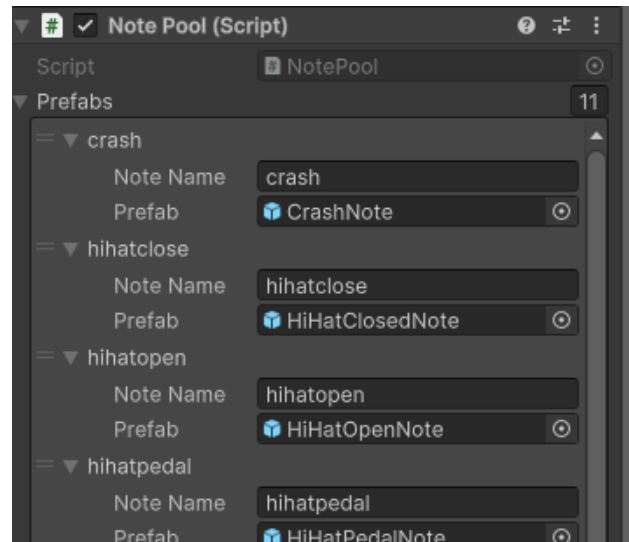
Il·lustració 138. Component *MidiNoteSpawner* i classe *LaneConfig*

També cal mencionar que cada nota té el seu propi *prefab* amb el que s'identifica per la seva forma i color, que està relacionat amb els instruments, tal i com es pot veure a la Il·lustració 139.



Il·lustració 139. Prefabs de cada una de les notes

La creació de noves notes afecta al rendiment de tot el projecte per això s'ha utilitzat la tècnica de *object pooling* que ajuda a que no es sobrecarregui tot el sistema. Gràcies a l'ús de *object pooling*, les notes no es creen i destrueixen constantment, sinó que es reciclen, cosa que millora el rendiment i evita pics de memòria. A la Il·lustració 140 es pot veure el component *Note Pool* que s'encarrega d'això mateix:



Il·lustració 140. Component *Note Pool*

A la Il·lustració 141, es poden veure les dues funcions que fan possible el maneig del *pool* de notes. Amb *Get()* s'agafa la nota que es vol utilitzar i amb *Release()* es deixa anar i deixa la nota disponible per a ser agafada en un altre moment.

```
2 referencias
public GameObject Get(string noteName, Vector3 position)
{
    if (!pools.ContainsKey(noteName))...

    GameObject obj = pools[noteName].Count > 0
        ? pools[noteName].Dequeue()
        : Instantiate(prefabLookup[noteName], transform);

    obj.transform.position = position;
    obj.SetActive(true);
    return obj;
}

2 referencias
public void Release(string noteName, GameObject obj)
{
    obj.SetActive(false);
    if (!pools.ContainsKey(noteName))
        pools[noteName] = new Queue<GameObject>();

    pools[noteName].Enqueue(obj);
}
```

Il·lustració 141. Funció *Get* i *Release* del *Note Pool*

El *Note Pool* s'inicialitza amb un nombre de notes en concret, però si la cançó requereix de més notes, el *Note Pool* instancia una nota més i s'afegeix al *pool* de notes.

Com ja s'ha vist, quan una nota arriba al seu punt final, el *NoteMover* executa la funció *HitTarget()*. En aquest moment s'activa el *NoteSoundManager* (Il·lustració 142), que s'encarrega de reproduir el clip d'àudio associat a aquella nota. Per permetre que sonin múltiples notes simultàniament sense tallar-se entre elles, aquest component gestiona un pool d'*AudioSource* i va alternant-los de manera cíclica.



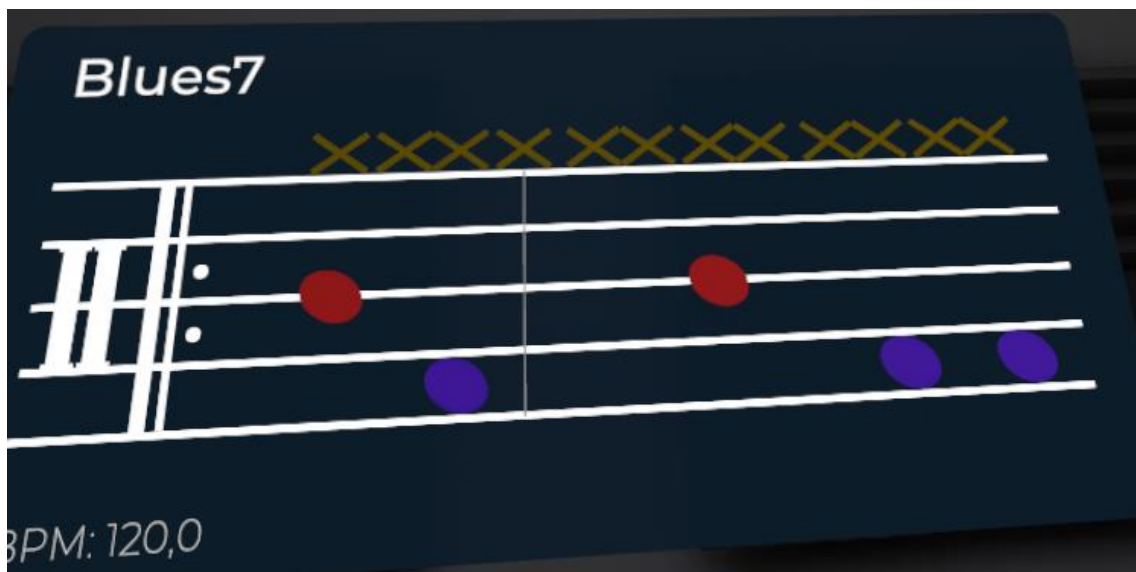
Il·lustració 142. Component *NoteSoundManager*.

A més, des de la interfície s'ofereix un Toggle per silenciar la sortida d'àudio de les cançons MIDI, de manera que l'usuari pot decidir si vol escoltar només la base musical o també els sons generats per les notes visuals.

L'objectiu era fer que la visualització de les cançons sigui la més propera a llegir un pentagrama per a que pugui ser d'ajuda a l'aprenentatge del instrument d'una forma realista. Per això s'utilitzen els símbols de les notes com en una partitura i també s'ha volgut separar cada compàs per una barra.

Per això s'aprofita que el *MidiPlayer* també emet l'esdeveniment *OnBarChanged* cada vegada que es detecta un canvi de compàs a la cançó. Aquest esdeveniment és rebut pel *BarSpawner*, que genera una barra de compàs dins del panell visual. A continuació, el component *BarMover* s'encarrega de desplaçar aquesta barra fins al punt final, seguint el mateix rellotge de sincronització que les notes.

A la Il·lustració 143, es pot observar com es veu el pentagrama en el moment de reproducció d'una cançó, amb les seves notes i barres de compàs.

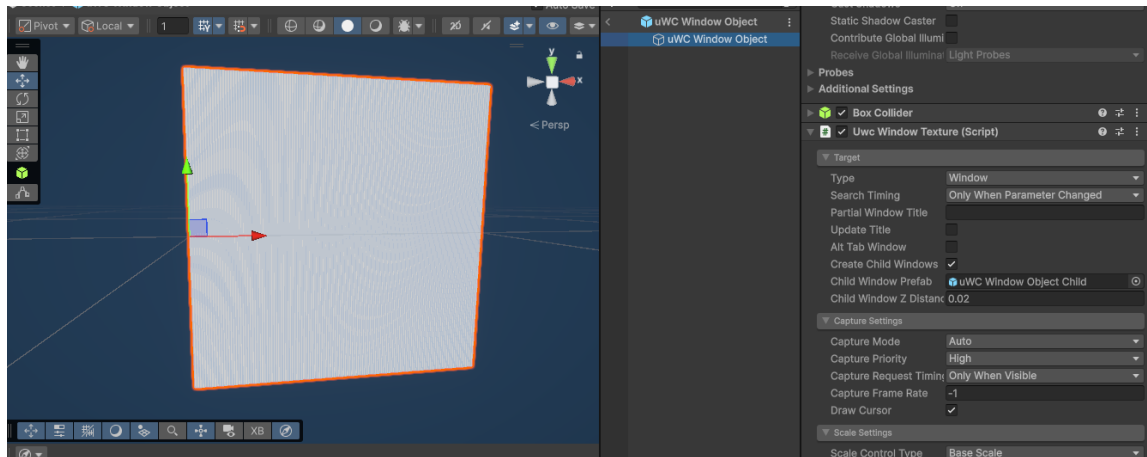


Il·lustració 143. Pentagrama en acció

5.4.5 Captura de finestres

Per a la funcionalitat de la captura de finestres dins de l'entorn de realitat virtual s'ha utilitzat la llibreria *uWindowCapture* (vegeu l'apartat 2.2.5.1), que permet capturar en temps real finestres de Windows i mostrar-les com a textures dins de Unity.

Per a que les finestres es puguin veure dins de la realitat virtual, s'utilitza de base el *prefab* creat per la llibreria anomenat *uWCWindowObject*. Aquest té contingut un *UwcWindowTexture*, el component encarregat de generar la textura capturant en temps real la finestra.



Il·lustració 144. Prefab del uWC Window Object

Amb el component *UwcWindowTextureManager* es fa la gestió central de les finestres, on es manté un registre amb totes les finestres actives i emet dos esdeveniments:

- *onWindowTextureAdded* : Quan es detecta una nova finestra.
- *onWindowTextureRemoved*: Quan la finestra es tanca o deixa d'estar disponible.

Aquests esdeveniments criden a les funcions que fan possible que s'agreguin o s'eliminen les finestres. A la Il·lustració 145. Funcions d'afegir i treure finestres es poden veure aquestes funcions: *AddWindowTexture* i *RemoveWindowTexture*.

```
public UwcWindowTexture AddWindowTexture(UwcWindow window)
{
    if (!windowPrefab) {
        Debug.LogError("windowPrefab is null.");
        return null;
    }

    var obj = Instantiate(windowPrefab);
    var windowTexture = obj.GetComponent<UwcWindowTexture>();
    Assert.IsNotNull(windowTexture, "Prefab must have UwcWindowTexture component.");
    windowTexture.window = window;
    windowTexture.manager = this;

    windows_.Add(window.id, windowTexture);
    onWindowTextureAdded.Invoke(windowTexture);

    return windowTexture;
}

2 referencias
public void RemoveWindowTexture(UwcWindow window)
{
    UwcWindowTexture windowTexture;
    windows_.TryGetValue(window.id, out windowTexture);
    if (windowTexture) {
        onWindowTextureRemoved.Invoke(windowTexture);
        windows_.Remove(window.id);
        Destroy(windowTexture.gameObject);
    }
}
```

Il·lustració 145. Funcions d'afegir i treure finestres

Per integrar la captura de finestres dins del projecte s'ha creat una adaptació anomenada *UwcModifiedSpawn*. Aquest script s'encarrega de col·locar les finestres que es volen activar en *slots* predefinits al costat del menú principal, fent que no hi hagi solapament de finestres i fent possible activar tantes finestres com l'usuari vulgui.

Aquest script utilitza els esdeveniments *onWindowTextureAdded* i *onWindowTextureRemoved* que genera el *UwcWindowTextureManager* per a fer aquesta gestió i tal com es pot veure a la Il·lustració 146, es criden aquestes funcions:

```

1 referencia
void OnWindowAdded(UwcWindowTexture wt)
{
    int slotIndex = slots.FindIndex(s => s == null);
    if (slotIndex == -1) slotIndex = slots.Count;

    if (slotIndex < slots.Count)
        slots[slotIndex] = wt;
    else
        slots.Add(wt);

    Vector3 offset = offsetPerWindow * slotIndex;
    wt.transform.position = transform.TransformPoint(offset);
    wt.transform.rotation = transform.rotation;
    wt.scalePer1000Pixel = 0.5f;

    var grab = wt.GetComponent<UnityEngine.XR.Interaction.Toolkit.Interactables.XRGrabInteractable>();
    if (grab != null)
    {
        grab.selectEntered.AddListener(_ => OnGrabbed(wt));
    }
}

1 referencia
void OnWindowRemoved(UwcWindowTexture wt)
{
    int index = slots.IndexOf(wt);
    if (index >= 0) slots[index] = null;
}

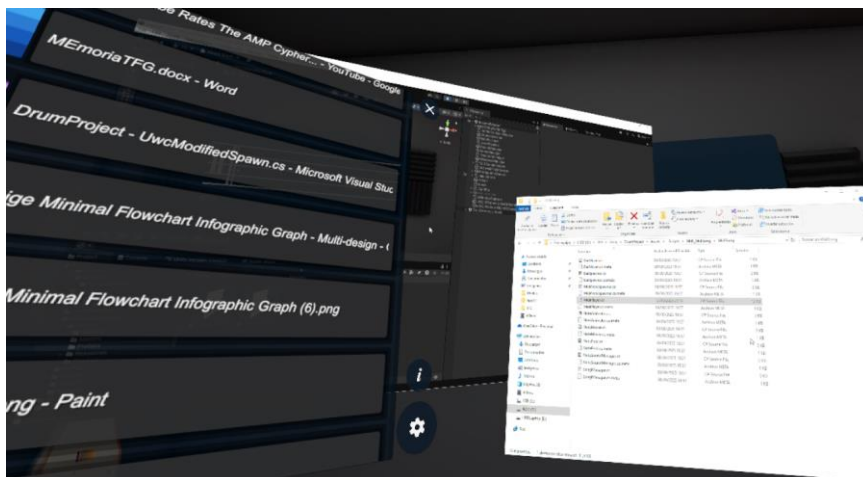
```

Il·lustració 146. Funcions de *OnWindowsAdded* i *OnWindowsRemoved* de *UwcModifiedSpawn*

UwcModifiedSpawn fa el següent:

- Cada finestra que apareix ocupa el primer *slot* disponible.
- Si l'usuari agafa la finestra (gràcies al component *XRGrabInteractable*), aquesta es desvincula de l'*slot* i passa a ser filla del món (*detachedParent*). Això permet col·locar-la lliurement dins de l'escena.
- Quan una finestra es tanca, l'*slot* torna a quedar buit i disponible.

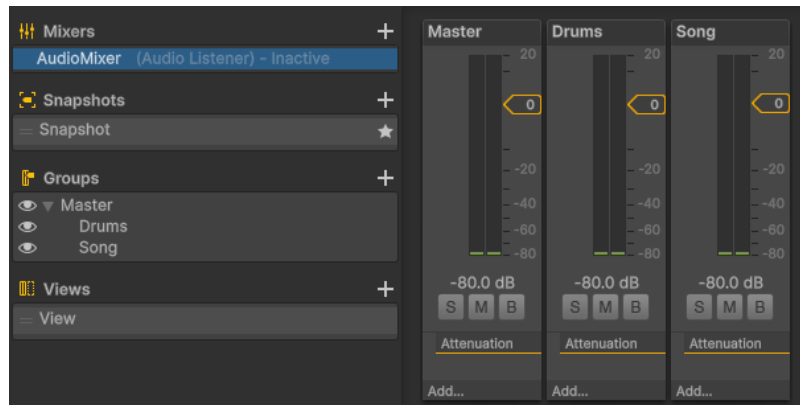
Finalment, per a que les finestres s'instanciïn sempre al costat del menú s'ha fet l'script *FollowScript*. Aquest script fa que un objecte segueixi la posició i rotació d'un altre amb un offset configurable. A la es pot veure un exemple de la funció de captura de finestres en acció:



Il·lustració 147. Captura de finestres en acció

5.4.6 Control de volum i mescla

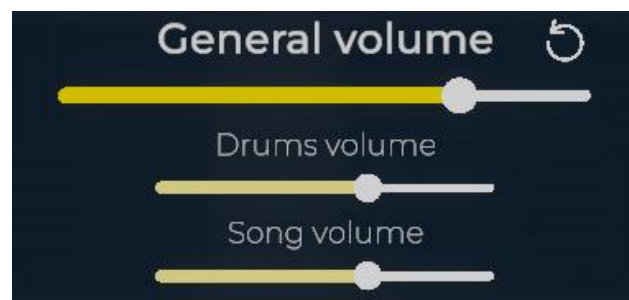
Per tal de donar a l'usuari control total sobre el so del sistema, el sistema disposa d'un panell de mescla d'àudio implementat amb el component *AudioMixer* de Unity. Aquest mesclador està dividit en tres canals principals (Il·lustració 148):



Il·lustració 148. *AudioMixer* del sistema

- Master: controla el volum general de tot el sistema.
- Drums: regula específicament el volum dels instruments.
- Song: ajusta el volum de les cançons MIDI o pistes d'àudio de fons.

Aquests tres canals es vinculen a la UI mitjançant *sliders* (Il·lustració 149), de manera que l'usuari pot modificar fàcilment els nivells durant l'execució.



Il·lustració 149. Panell de volum general amb *sliders* de *Master*, *Drums* i *Song*.

Els valors del volum es desen a *PlayerPrefs*, assegurant que es mantinguin entre sessions. A la Il·lustració 150 es pot veure l'ús dels *PlayerPrefs* per a iniciar el volum a el valor que s'ha deixat l'últim cop.

```
void Start()
{
    float masterVol = PlayerPrefs.GetFloat("Volume_Master", 1f);
    float drumsVol = PlayerPrefs.GetFloat("Volume_Drums", 1f);
    float songVol = PlayerPrefs.GetFloat("Volume_Song", 1f);
}
```

Il·lustració 150. S'agafen els valors de *PlayerPrefs* per posar el volum inicial.

El controlador que s'encarrega d'això i es el principal component del sistema d'àudio es l'script *SoundManager* que es pot veure a la Il·lustració 151.

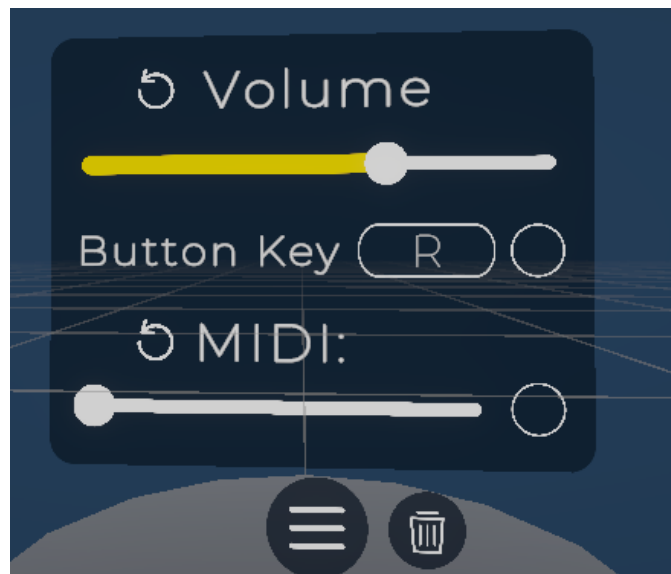
```
1 referencia
public void SetSongVolume(float value)
{
    mixer.SetFloat("SongVolume", Mathf.Log10(value) * 20);
    PlayerPrefs.SetFloat("Volume_Song", value);
}

0 referencias
public void resetMasterVolume()
{
    masterSlider.value = 1f;
}
```

Il·lustració 151. Funcions del *SoundManager*

A més, el *SoundManager* inclou opcions de reinici, com *resetMasterVolume()*, que retorna el volum general al seu valor per defecte (1.0).

Per a un control més detallat, cada instrument també disposa del seu *slider* individual dins del seu menú propi, com ja s'ha vist en l'apartat 5.3.6. Aquest *slider* està vinculat al *HitDetector* de l'instrument i ajusta el paràmetre *volumeMultiplier*, que escala la intensitat del so generat per aquell tambor en particular.



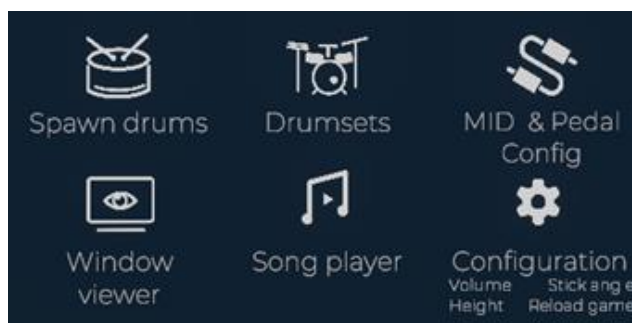
Il·lustració 152. Menú individual de cada instrument

5.5 User Interface

Un dels objectius del projecte ha estat aconseguir que totes les funcionalitats es poguessin gestionar de manera senzilla i accessible dins de l'entorn de realitat virtual. Per aquest motiu es va desenvolupar un menú principal que actua el punt central des d'on l'usuari pot controlar el sistema.

Inicialment, la idea del menú va sorgir com una eina bàsica per poder crear i col·locar peces de bateria dins de l'escena. Però, a mesura que el projecte va créixer i es van anar afegint noves funcionalitats, el menú es va anar expandint fins a convertir-se en una interfície que permet tant configurar el sistema com accedir a opcions més avançades.

A la Il·lustració 153 es pot veure les diferents icones que es troben en el menú principal que corresponen a les pantalles de les seccions d'aquest.

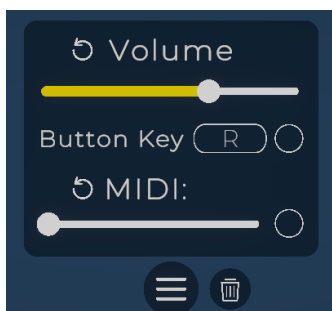


Il·lustració 153. Icones dels diferents panells del menú

El disseny final del menú permet navegar per les funcionalitats implementades (vegeu 5.4):

- Crear i *spawnear* peces de bateria per personalitzar el kit.
- Guardar i carregar *drumsets* complets per reutilitzar configuracions.
- Seleccionar dispositius MIDI *in/out*, que permeten connectar controladors externs i enviar notes cap a un DAW.
- Obrir el *Desktop Viewer*, que projecta aplicacions de l'escriptori dins de l'entorn VR.
- Escollir i carregar cançons MIDI, que poden ser reproduïdes dins del sistema.
- Ajustar els paràmetres de so i volum mitjançant *sliders* i opcions d'àudio.

Com ja hem vist també trobem menús individuals en cada instrument que segueixen el mateix estil de disseny que el menú principal, per més detall vegeu 5.3.6.



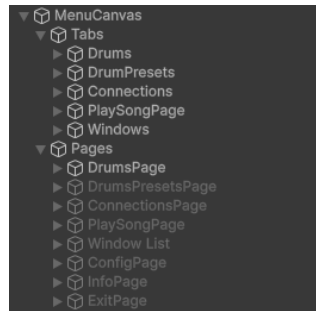
Il·lustració 154. Menú individual de l'instrument

Les interfícies, com s'ha vist en l'apartat 2.2.2.5, s'han posat en el mode de render *World Space* per a que existeixin i es puguin interactuar dins del món.



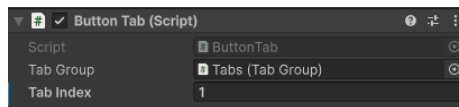
II·lustració 155. Configuració del *canvas* de les interfícies

El menú principal està implementat amb scripts específics que gestionen cadascuna de les funcionalitats d'aquest menú. Com es pot observar a la II·lustració 156. Estructura del menú156 el menú està conformat per els *Tabs* (botons de selecció) i les *Pages* (pàgines).



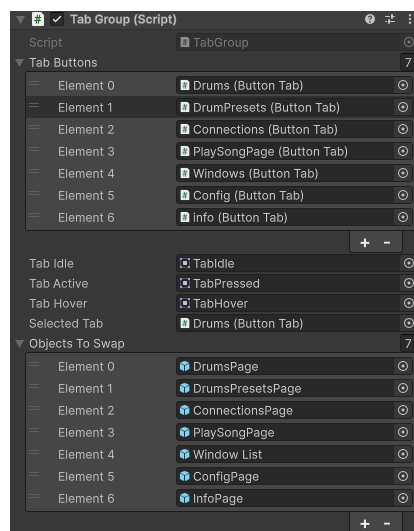
II·lustració 156. Estructura del menú

Per a que hi hagi un control de quin *tab* correspon a cada pàgina, a cada *ButtonTab* se li ha assignat un *Tab Index*:



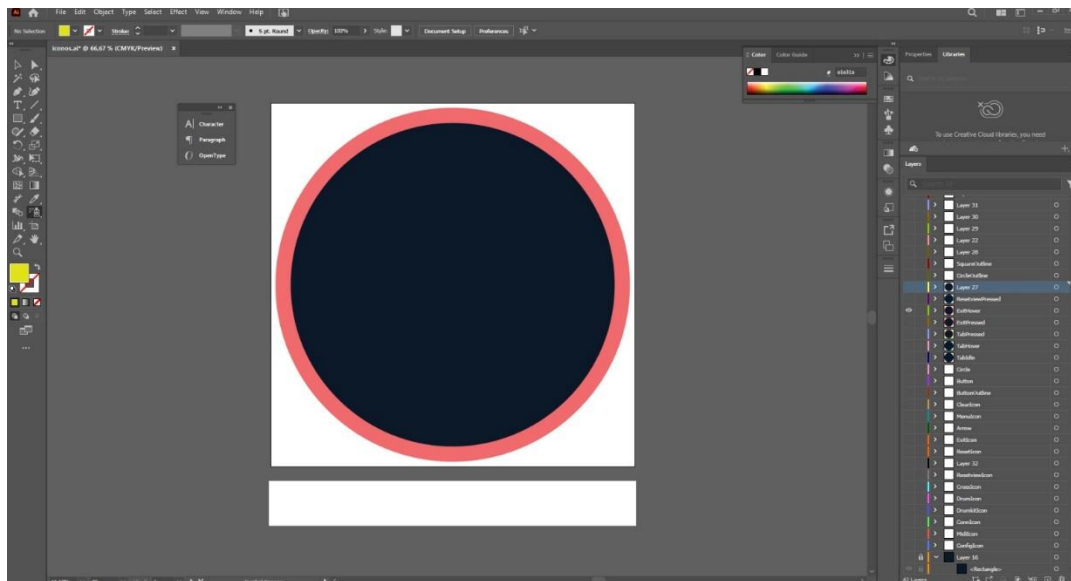
II·lustració 157. *ButtonTab* amb el seu índex assignat

Llavors, en el component de *TabGroup* de la II·lustració 158158, es on relaciona cada pàgina amb el seu botó del *tab*. A més aquest component porta el control de quin és la pàgina del menú que s'obre per defecte en *SelectedTab* i gestiona els diferents *Sprites* (visuals dels botons) per a quan un botó està sense seleccionar (*Tab Idle*), sent senyalat (*Tab Hover*) o seleccionat (*Tab Active*).



II·lustració 158. Component *TabGroup*.

A més de la seva implementació funcional, s'ha fet tot disseny gràfic de la interfície per a que resulti clara i intuïtiva. Tots els elements visuals s'han creat amb Adobe Illustrator, amb un estil senzill i minimalista.



Il·lustració 159. Programa Adobe Illustrator amb totes les capes amb els elements de les interfícies.

La paleta de colors escollida (**¡Error! No se encuentra el origen de la referencia.**160) es basa en tons foscos pels fons, complementats colors contrastants com el blanc i groc per ressaltar les accions principals i un color vermell per les accions crítiques:



Il·lustració 160. Paleta de colors de les interfícies.

Pel que fa a la tipografia, s'ha utilitzat la font Montserrat, amb un pes fi i estil net. Aquesta tria respon a la necessitat de tenir una lletra moderna i fàcilment llegible dins de la realitat virtual, sense carregar visualment els panells.

MONTSERRAT HAIRLINE	MONTSERRAT SEMI BOLD
MONTSERRAT ULTRA LIGHT	MONTSERRAT BOLD
MONTSERRAT LIGHT	MONTSERRAT EXTRA BOLD
MONTSERRAT REGULAR	MONTSERRAT BLACK

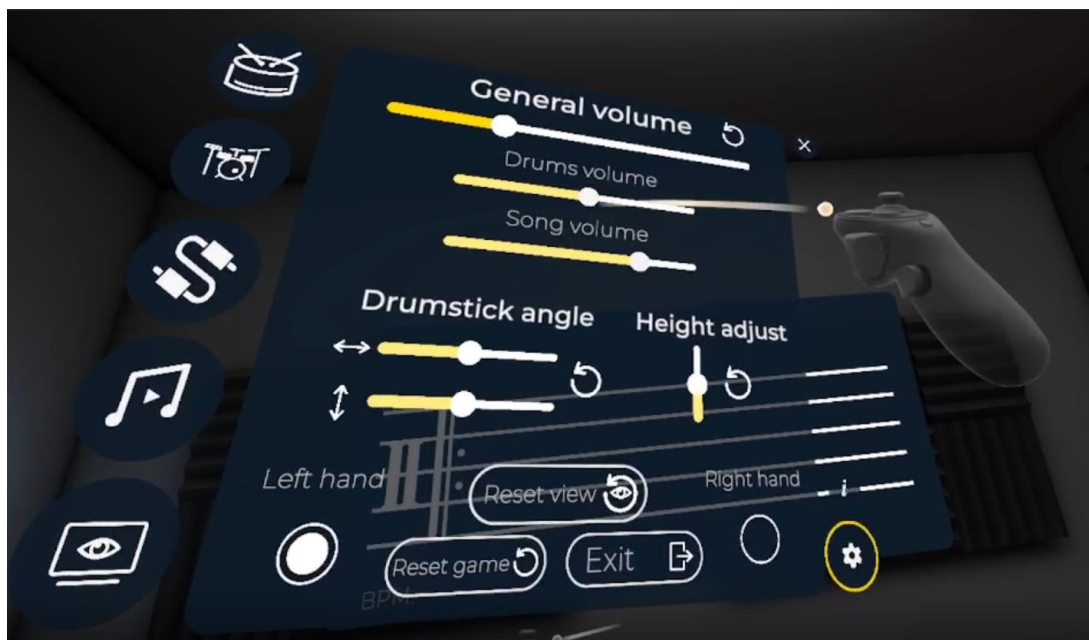
Il·lustració 161. Exemples de la font Montserrat. Font: <https://zsquarestudio.com/exploring-the-elegance-and-versatility-of-the-montserrat-font/>

Les icones del menú segueixen l'estil simple, de manera que cada secció del menú principal és fàcilment identificable. L'ús del color blanc i transparències fa que aquestes icones destaquin sense necessitat de textos addicionals, fent que la navegació sigui intuïtiva i ràpida.



Il·lustració 162. Icones del programa dins de Unity.

Amb tot això s'ha construït la tota la UI del sistema, que actuen com a punt central des d'on es poden gestionar totes les funcionalitats. A la Il·lustració 163 es pot veure el menú en el panell de configuració en acció.



Il·lustració 163. Menú principal en acció

5.6 Accessibilitat i usabilitat

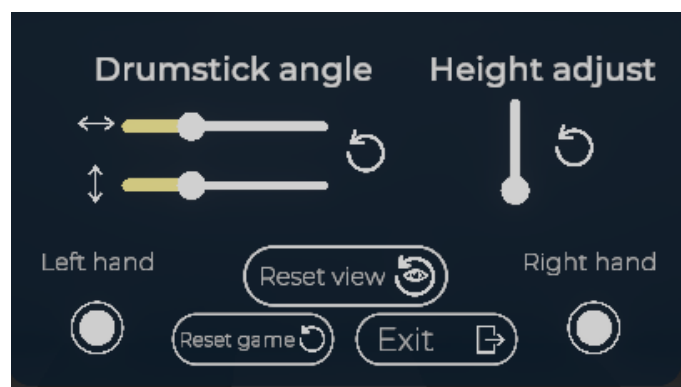
Com ja s'ha mencionat en els objectius del projecte, un d'aquests objectius és que es l'usuari tingui una experiència còmoda, senzilla i adaptable a diferents tipus d'usuaris independentment de les seves preferències o necessitats. Per aconseguir-ho, s'han implementat diferents funcionalitats orientades a millorar l'accessibilitat i la usabilitat del sistema.

En primer lloc, per facilitar a l'usuari, especialment per als nous usuaris o als que fa molt de temps que no entren a la aplicació, i que sàpiga com es pot moure pel sistema s'ha creat un panell d'informació que mostra de tots els controls principals i el significat de les icones que es poden trobar a l'hora de navegar pel programa.



Il·lustració 164. Panell d'informació amb controls i icones del sistema

Per altra banda, també s'han afegits algunes funcionalitats per adaptar la configuració a gust de cada usuari. A la Il·lustració 165 es pot veure el panell d'ajustos que conté les funcions de accessibilitat.



Il·lustració 165. Panell d'ajustos amb les seves funcions

5.6.1 Ajust de l'alçada

Per tal d'adaptar el sistema a usuaris de diferents estatures i preferències d'alçada del seient de la bateria, una preferència que es molt personal i s'ha parlat molt en el món de la bateria, s'ha desenvolupat el component *CameraHeightAdjuster*. Aquest script permet ajustar l'alçada de la càmera dins de l'*XROrigin* mitjançant un *slider*, tal i com es pot veure a la Il·lustració 166.



Il·lustració 166. Ajust de l'alçada en el panell del menú

Aquest script (Il·lustració 167) escolta al *slider*, quan aquest *slider* es mogut, no s'aplica l'alçada directament, sinó que s'espera a quan es deixa (*EndDrag*) i activa la funció *ApplyHeight()*.

```
endDrag.callback.AddListener((data) => { ApplyHeight(); });
trigger.triggers.Add(endDrag);
}

2 referencias
void ApplyHeight()
{
    if (xrOrigin == null) return;

    float newHeight = Mathf.Lerp(minHeight, maxHeight, heightSlider.value);

    xrOrigin.CameraYOffset = newHeight;

    if (xrOrigin.CameraFloorOffsetObject != null)
    {
        Vector3 pos = xrOrigin.CameraFloorOffsetObject.transform.localPosition;
        pos.y = newHeight;
        xrOrigin.CameraFloorOffsetObject.transform.localPosition = pos;
    }
}

2 referencias
public void ResetHeight()
{
    heightSlider.value = Mathf.InverseLerp(minHeight, maxHeight, baseHeight);
    ApplyHeight();
}
```

Il·lustració 167. Funcions del component *CameraHeightAdjuster*.

ResetHeight() retorna la càmera a una alçada base configurada.

Els valors que estan configurats i s'han establert com a estàndard gràcies al feedback i preferències generals, són els següents:

Config	
Min Height	1.35
Base Height	1.55
Max Height	1.75

Il·lustració 168. Configuració d'alçades

5.6.2 Ajust de les baquetes

Aquesta funció permet ajustar l'angle amb el que “s’agafen” les baquetes gràcies a dos *sliders* que canvien l'angle vertical i l'angle horitzontal. A la Il·lustració 169 es pot veure aquest panell d'ajust.



Il·lustració 169. Panell d'ajust dels angles de les baquetes

Durant l'ajust es mostren unes baquetes “fantasma” que serveixen de guia visual abans d'aplicar els canvis a les baquetes reals. Tot això es implementat gràcies a l'script *DrumstickAngleAdjust*. Com es pot veure a la Il·lustració 170, quan es comença a ajustar la baqueta, es crida a la funció *BeginAdjust()* que activa a les baquetes i gràcies a *UpdateGhosts()* les baquetes de recolzament s'actualitzen i es poden veure.

```
2 referències
private void UpdateGhosts()
{
    if (!adjusting) return;

    float offsetYaw = Mathf.Lerp(-maxOffsetY, maxOffsetY, sliderX.value);
    float offsetRoll = Mathf.Lerp(-maxOffsetZ, maxOffsetZ, sliderY.value);

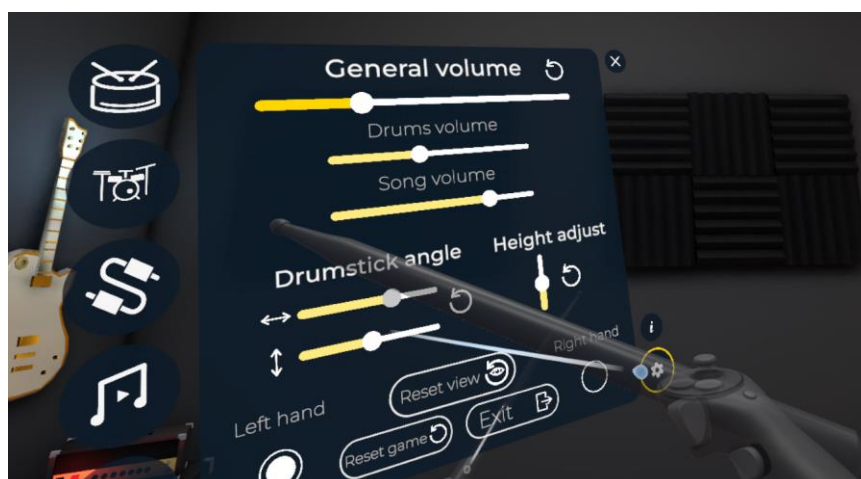
    Vector3 leftRot = baseLeftRotation + new Vector3(0, offsetYaw, offsetRoll);
    ghostLeft.localRotation = Quaternion.Euler(leftRot);

    Vector3 rightRot = baseRightRotation + new Vector3(0, -offsetYaw, offsetRoll);
    ghostRight.localRotation = Quaternion.Euler(rightRot);
}

1 referència
public void BeginAdjust()
{
    adjusting = true;
    ghostLeft.gameObject.SetActive(true);
    ghostRight.gameObject.SetActive(true);
    UpdateGhosts();
}
```

Il·lustració 170. Funcions de *DrumstickAngleAdjust*

A la Il·lustració 171 es pot veure l'exemple de l'ajust de les baquetes i les baquetes “fantasma”.



Il·lustració 171. Usuari ajustant les baquetes

5.6.3 ResetView

Per poder tornar al punt de partida per resoldre situacions on l'usuari es desplaça massa, es descalibra el tracking o senzillament vol tornar a aquest punt, fet la funció de ResetView:



Il·lustració 172. Botó de *reset view*

L'script que controla això es *ResetPosition*, que conté una referència a l'*XROrigin* i una *initialPosition* definida com a punt base dins l'escena. A la Il·lustració 173 es pot veure el *GameObject* de *Initial Position* a la escena.



Il·lustració 173. *GameObject InitialPosition* en el punt inicial d'aparició.

Quan es prem el botó de *ResetView*, es crida a la funció *ResetPlayerPosition()* (Il·lustració 174) que utilitza els mètodes:

- *MoveCameraToWorldLocation()*: la càmera es trasllada exactament a la posició de la *InitialPosition*.
- *MatchOriginUpCameraForward()*: ajusta la rotació perquè l'orientació de la càmera coincideixi amb la de *Initial Position*.

```
1 referencia
public void ResetPlayerPosition()
{
    if (xrOrigin == null || initialPosition == null)
    {
        Debug.LogWarning("[ResetPosition]");
        return;
    }

    xrOrigin.MoveCameraToWorldLocation(initialPosition.position);
    xrOrigin.MatchOriginUpCameraForward(initialPosition.up, initialPosition.forward);
}
```

Il·lustració 174. Funció *ResetPlayerPosition()*.

5.6.4 Menú ambidextre

Per poder adaptar-se als usuaris tant dretans com esquerrans s'ha implementat la funció de canviar de costat el menú. Això es perquè de manera predeterminada, el menú es troba seguint la mà esquerra per a que l'usuari pugui interactuar amb la mà dreta però això pot ser incòmode per a persones que prefereixen interactuar amb la mà esquerra.



Il·lustració 175. Panell del menú amb els botons de canvi de mà.

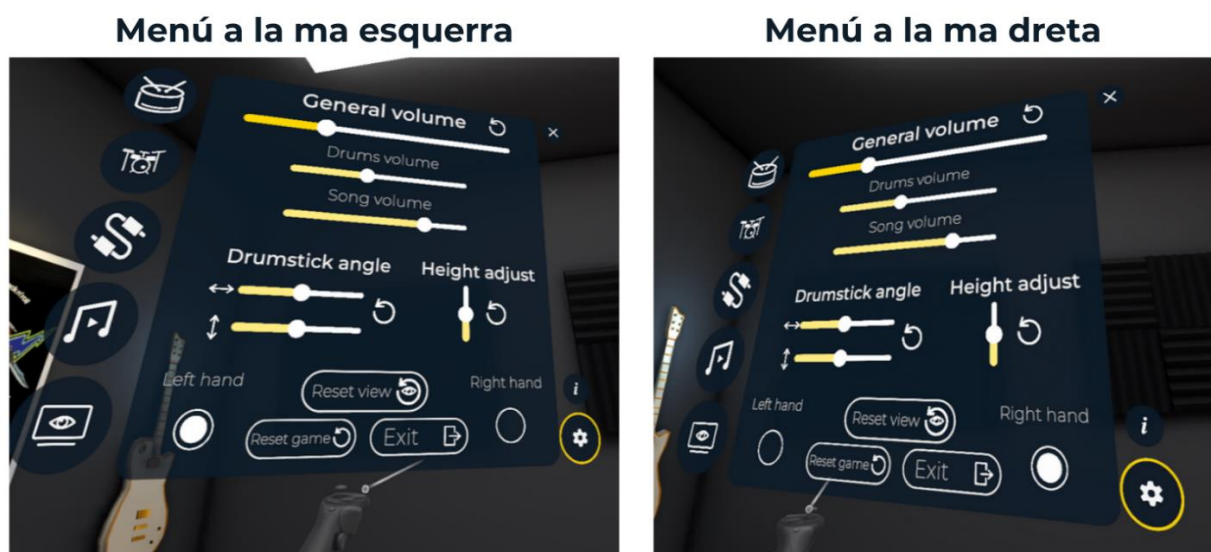
Aquesta funcionalitat s'ha implementat amb l'script *MenuHandToggle*, que vincula el menú a una de les dues àncores de les mans (*leftHandAnchor* i *rightHandAnchor*). Tal i com es veu a la Il·lustració 176, el component permet que el menú quedi fixat a la mà seleccionada, s'apliquen uns offsets de posició i rotació per ajustar la posició del menú en cada cas.

```
2 referencias
void SetMenuToLeft()
{
    menu.SetParent(leftHandAnchor, false);
    menu.localPosition = localPositionOffsetLeft;
    menu.localRotation = Quaternion.Euler(localRotationOffsetLeft);
}

1 referencia
void SetMenuToRight()
{
    menu.SetParent(rightHandAnchor, false);
    menu.localPosition = localPositionOffsetRight;
    menu.localRotation = Quaternion.Euler(localRotationOffsetRight);
}
```

Il·lustració 176. Funcions del MenuHandToggle

El canvi de mà es realitza mitjançant dos *toggles* a la UI, un per a cada mà. Per evitar inconsistències, quan un *toggle* està actiu l'altre es desactiva automàticament.



Il·lustració 177. Exemple del menú en cada mà

5.7 Selecció dels àudios i preparació

Per aconseguir un so el més realista possible dins del sistema, s'ha seguit uns passos de selecció, edició i normalització dels clips d'àudio dels instruments. Aquesta preparació garanteix que tots els sons tinguin un volum uniforme i que no hi hagi distorsions per a que es puguin integrar fàcilment dins del *mixer* d'àudio del projecte.

Els clips d'àudio utilitzats en el projecte han estat extrets de la llibreria pública de SMM Drums [30] on es poden trobar mostres d'instruments acústics reals gravades amb diferents intensitats de cop.

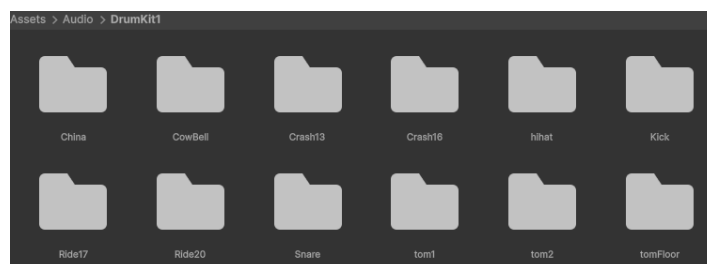
En aquesta llibreria es poden trobar fins a 30 o més mostres per instrument, amb molt detall en cada cop, però integrar tants clips d'àudio en el projecte podria sobrecarregar l'aplicació amb un nombre excessiu de mostres, per això es va fer una selecció de quants samples incloure per a cada instrument. A més ja que es compta amb tot el sistema de regulació de volums (vegeu 5.4.6), no era necessari comptar amb tantes mostres de diferents intensitats.

A la Il·lustració 178 es pot veure un exemple de les mostres d'àudio utilitzades pel *Ride* de 20 polzades on s'han escollit quatre mostres per la vora de l'instrument i es pot veure la carpeta de bell que conté els àudios de la part central de l'instrument.



Il·lustració 178. Clips d'àudio per al Ride de 20'

Els àudios estan organitzats per instrument dins de la carpeta d'Assets de Unity tal i com es pot veure a la Il·lustració 179.

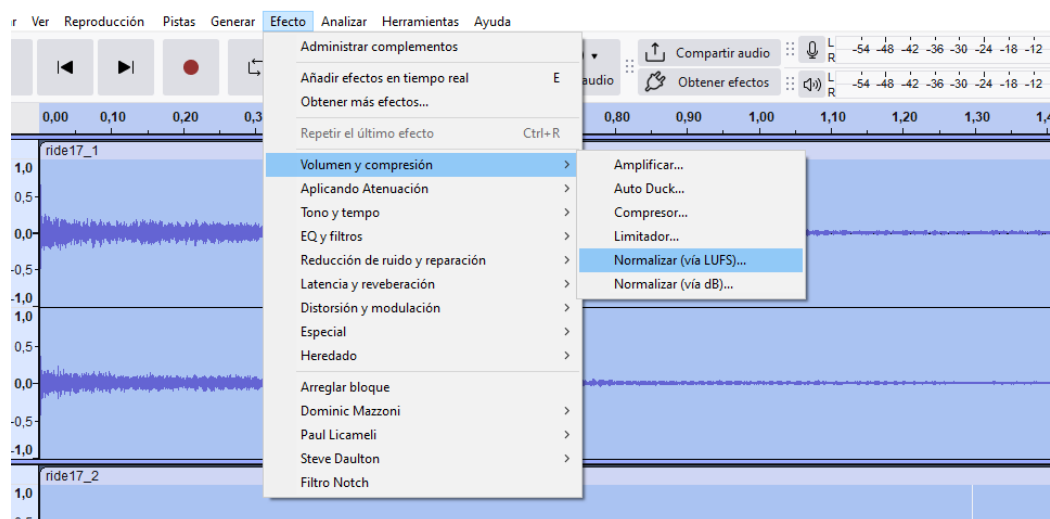


Il·lustració 179. Carpetes organitzades dels clips d'àudio

Les mostres originals de SMM Drums estan pensades per a que el volum percebut depengui exclusivament del volum de la intensitat del cop amb la què es van gravar.

Això, pel seu context d'ús on t'utilitzen totes les mostres és molt útil, però per a aquest projecte generava problemes, ja que la diferència de volum entre mostres feia que el control de volums integrat no funcionés correctament. Els sons de cops suaus s'escoltaven amb un volum baix pel propi clip i pel volum aplicat pel sistema, mentre que pel cops forts era tot el contrari i s'escoltava massa fort.

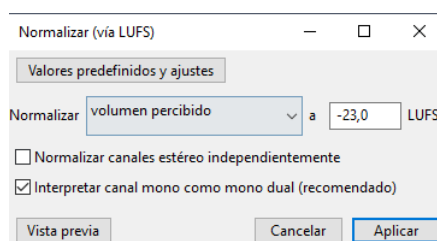
Per aquest motiu, tots els àudios s'han de normalitzar a un mateix volum per a que el sistema de volum pugui funcionar correctament. Per fer això s'ha utilitzat el programa de edició de àudio Audacity per utilitzar l'eina de normalització en LUFS (II·lustració 180. Efecte de normalització en LUFS de Audacity.).



II·lustració 180. Efecte de normalització en LUFS de Audacity.

Els LUFS (*Loudness Units relative to Full Scale*) són una unitat estàndard que mesura la sonoritat percebuda d'un àudio, no només el seu nivell de pic. A diferència del volum tradicional (dB), tenen en compte com l'oïda humana percep la intensitat del so en diferents freqüències. [31]

Després d'anar provant diferents intensitats de volums es va arribar a la conclusió de normalitzar totes les mostres seleccionades a -23 LUFS, tal i com es pot veure a la II·lustració 181. Aplicació de la normalització a -23 LUFS



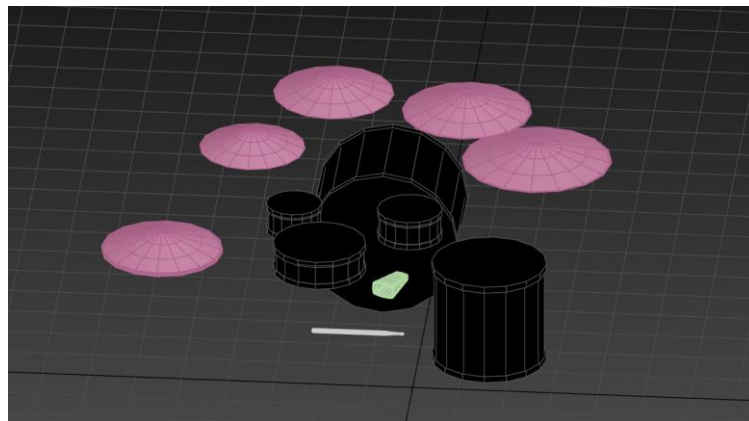
II·lustració 181. Aplicació de la normalització a -23 LUFS.

D'aquesta manera, tots els clips queden amb el mateix volum base i, un cop dins de l'aplicació, és el propi sistema qui controla la intensitat del so en funció de la velocitat del cop detectat o del paràmetre configurat pel jugador.

5.8 Modelatge i ambientació

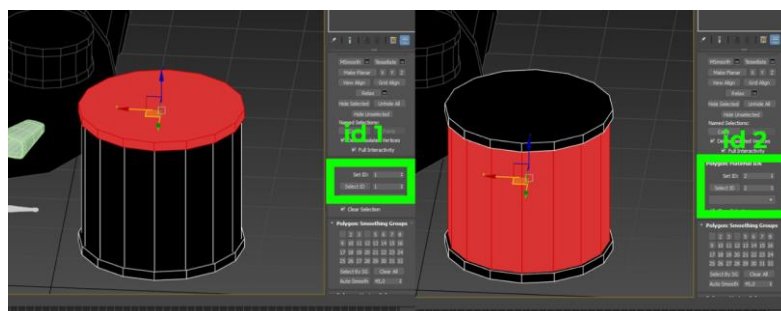
La estètica que s'ha seguit en el projecte ha estat l'estil del *low poly*, un estil de modelatge senzill però amb molta personalitat que ha donat un ambient amb una estètica clara i personal. Aquest estil també redueix la complexitat geomètrica, fent que el rendiment en entorns de realitat virtual pugui ser millor, alhora que manté una identitat visual pròpia.

Per a tot el procés de modelatge s'ha utilitzat 3DS Max. A la Il·lustració 182. Model dels instruments en 3DS Max. es pot veure els models dins del programa abans de importar-los a Unity.



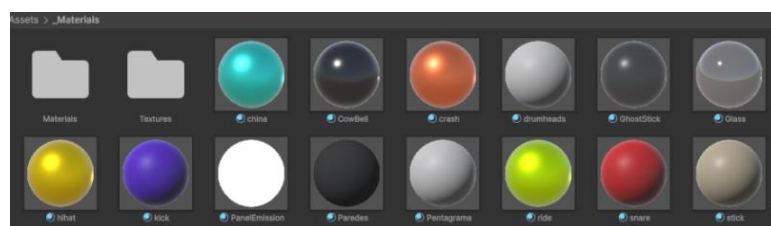
Il·lustració 182. Model dels instruments en 3DS Max.

Per la preparació dels models dins de Unity, els models dels instruments s'han separat en diferents IDs de material per després assignar-los fàcilment



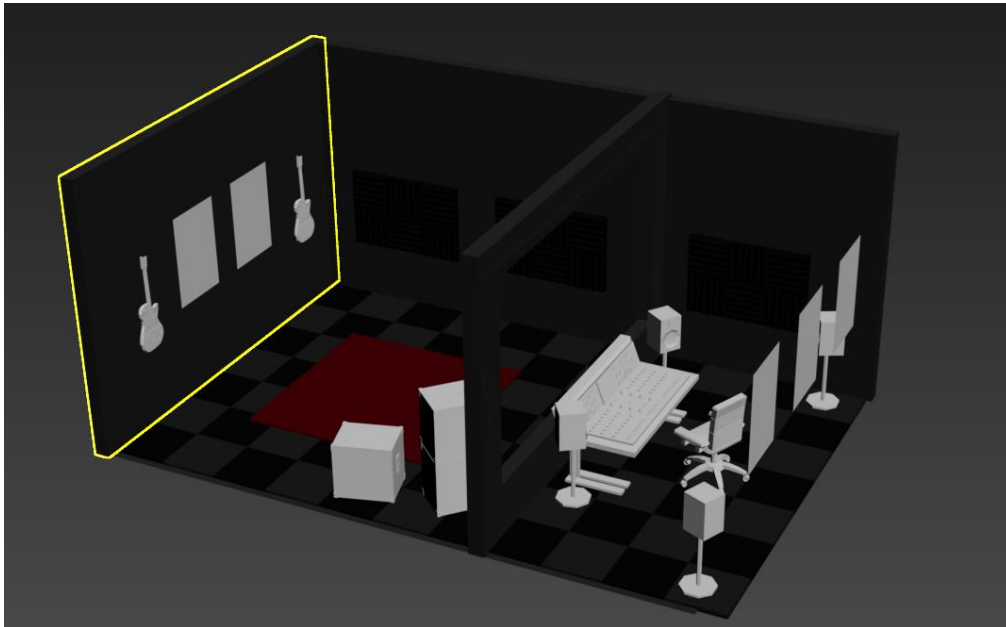
Il·lustració 183. Separació dels materials en ID's dins de 3DS Max

Dins de Unity s'han definit els materials finals. Els materials s'han aplicat amb colors plans i de diferents tipus com metalls per als plats i mate per als altres instruments (Il·lustració 184).



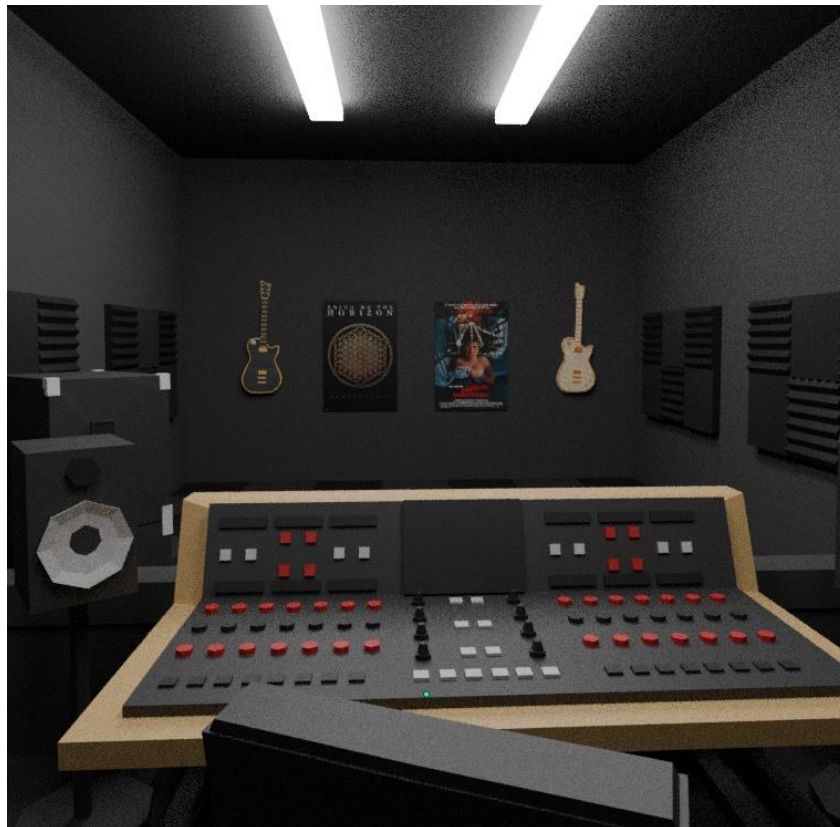
Il·lustració 184. Materials utilitzats dins de Unity.

Pel que fa l'ambient s'ha creat una sala d'estudi musical dividida en dos espais amb un panell central amb un material de vidre. A la Il·lustració 185 es pot veure com es veu l'estudi dins de 3dsmax.



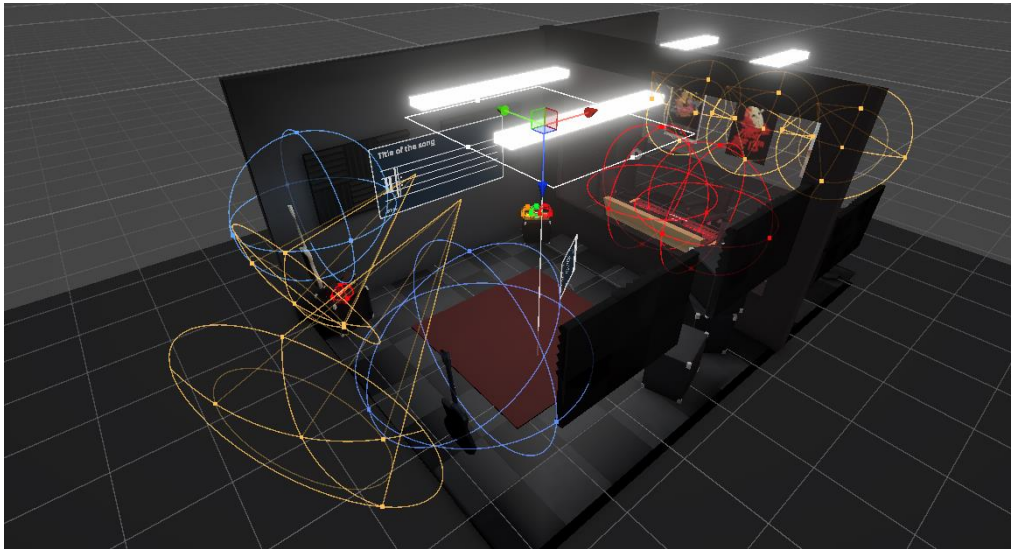
Il·lustració 185. Model del entorn dins de 3DS Max

Dins de 3DS Max es va afegir l'ambientació amb panells de llum amb *emission* que després s'han adaptat a Unity. A la Il·lustració 186 es pot veure un exemple d'un render dins de 3dsmax.

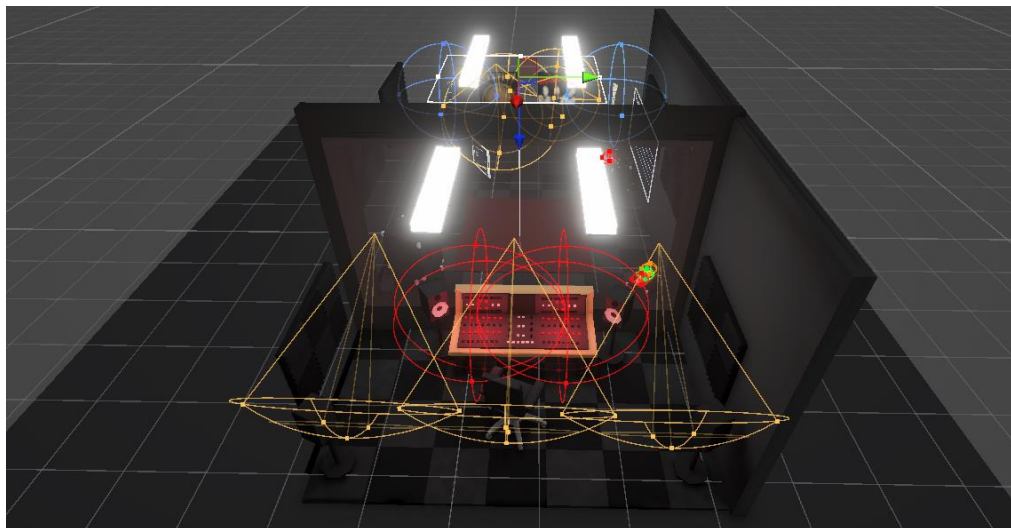


Il·lustració 186. Render de la sala dins de 3DS Max

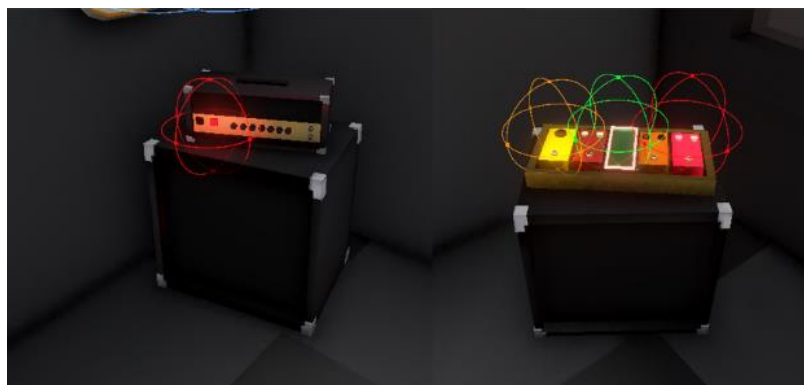
Per a la il·luminació de l'escena s'han utilitzat diversos tipus de llum: *point lights*, *directional lights* i *area lights* per aconseguir una atmosfera equilibrada. A les següents il·lustracions es pot veure les llums col·locades dins de l'escena.



Il·lustració 187. Llums col·locades dins de l'escena 1

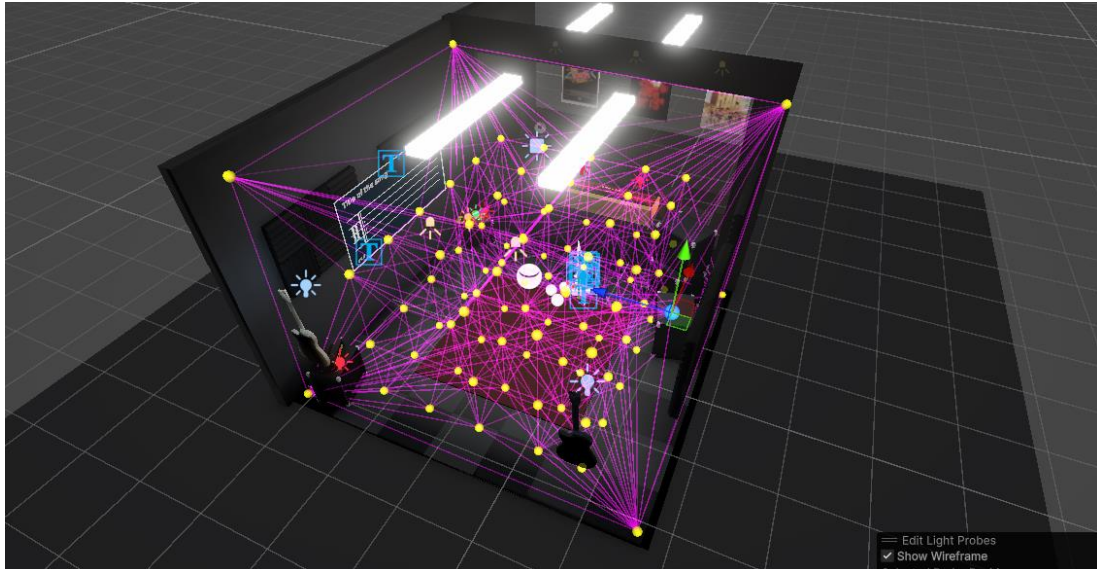


Il·lustració 188. Llums col·locades dins de l'escena 2



Il·lustració 189. Llums col·locades dins de l'escena 3

Per a la optimització del rendiment, totes aquestes llums s'han *bakejat*, de manera que queden fixes sobre les superfícies estàtiques de l'entorn. Aquest *bake* no afecta a els objectes dinàmics com son els instruments que es mouen per la escena. Per això s'ha utilitzat el sistema de *light probes* per optimitzar el rendiment i que permet als instruments rebre la il·luminació ambiental de forma realista.



Il·lustració 190. Light Probes dens de la sala de bateria.

També per a poder mantenir el reflexa de la ambientació per als plats o materials dinàmics amb reflexes, també s'ha utilitzat un *reflection probe* per a que aquests materials dinàmics rebin els reflexes de l'ambient.

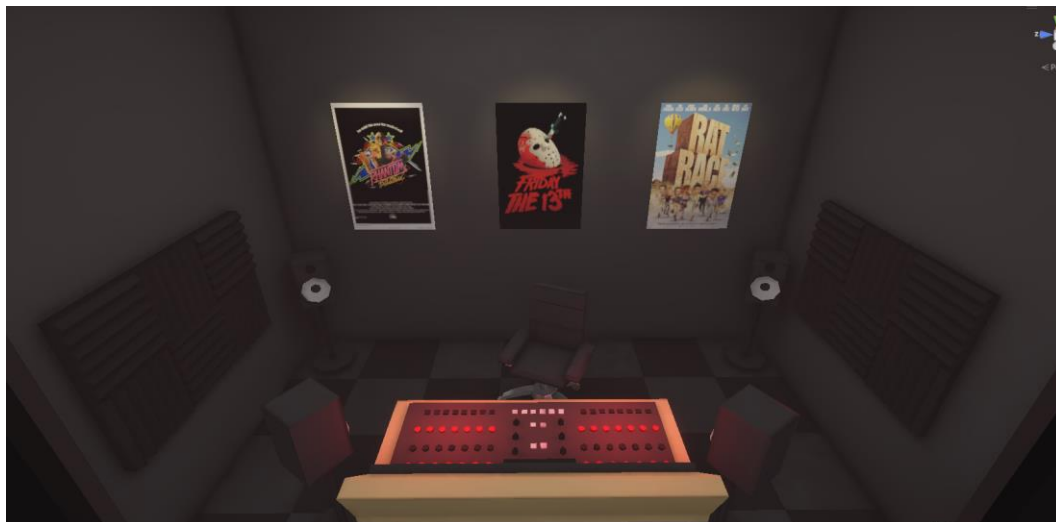


Il·lustració 191. Reflection probe dins de l'escena

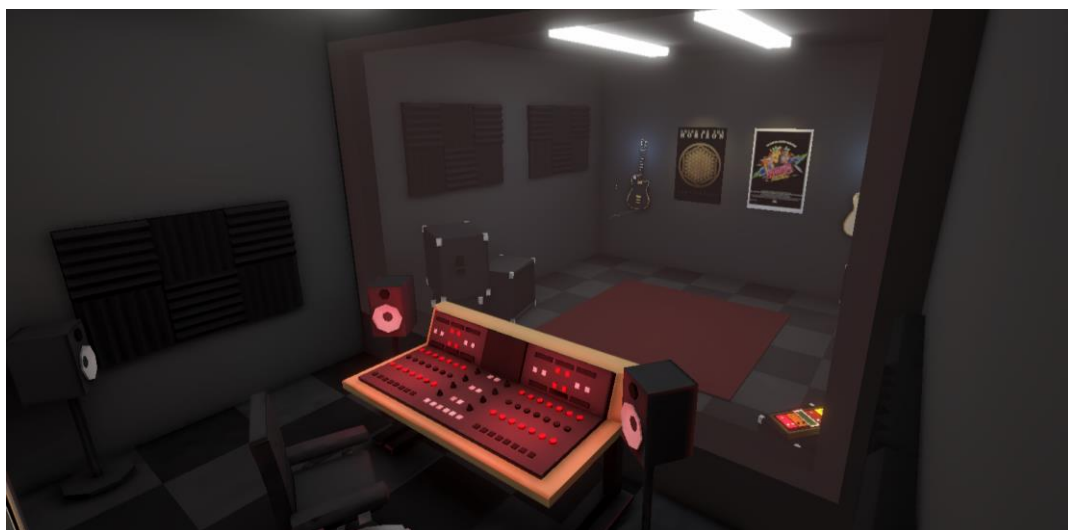
Després de tot el procés l'ambient ha quedat de la següent manera:



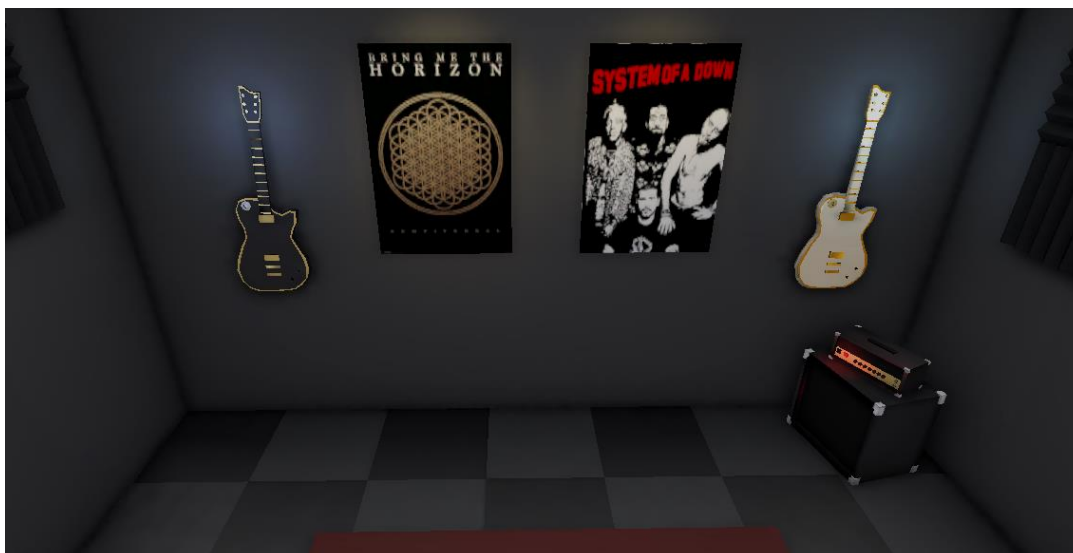
Il·lustració 192. Exemple ambient 1



Il·lustració 193. Exemple ambient 2



Il·lustració 194. Exemple ambient 3.



II·lustració 195. Exemple ambient 4.

El resultat ha sigut un entorn que combina el rendiment i l'estètica de *low poly* que s'ha volgut implementar. A la II·lustració 196 es pot veure un exemple de com queda l'ambient amb els instruments col·locats.



II·lustració 196. Exemple de la sala amb instruments

5.9 Rendiment i optimització

Un dels reptes d'aquest projecte ha estat el de mantenir un bon rendiment per aconseguir una experiència fluida i immersiva dins de la realitat virtual. Aquest repte al principi estava pensat per a que el programa s'executi a les ulleres Meta Quest 3, però des de que es va decidir que es necessitaria la connexió directe amb una plataforma d'escriptori per a connectar les diferents finestres i fer la connexió amb interfícies externes, finalment s'ha desenvolupat la versió final per a PC, que dona un marge de rendiment més ampli.

Tot i així s'ha mantingut aquesta filosofia d'optimització i fluïdesa pel projecte. Els punts principals de les decisions d'optimització han estat:

- Scripts i lògica:
 - L'ús de *coroutines* en alguns casos en comptes d'utilitzar la funció `Update()` en tot moment.
 - La utilització dels càlculs basats en temps DSP en lloc d'`Update()` per a reduir els error d'interpolació en cada *frame*.
 - El sistema de *cooldown* de les baquetes per a evitar cops múltiples i així reduir tant errors com executar lògica extra.
 - La utilització dels modes *kinematic* (sense físiques de Unity) per als objectes que no necessiten necessàriament físiques o canviant entre físiques i *kinematic* per així reduir la càrrega de càlculs de físiques.
- Gestió d'objectes:
 - Implementació de *pooling* per a les notes musicals de el visualitzador de les cançons per evitar costos elevats que es tindria d'*Instantiate* i *Destroy*.
 - Implementació de *pooling* per als diferents *Audio Sources* de les notes, amb el mateix efecte.
- Modelatge:
 - L'ús d'una estètica *low poly* per reduir el nombre de polígons.
- Il·luminació:
 - Aplicació de *bake* de llums per a les superfícies estàtiques.
 - L'ús de *light probes* i de *reflection probe* per aconseguir que els instruments dinàmics mantinguessin la mateixa il·luminació i reflex de l'ambient sense haver de estar calculant la llum dinàmicament en el sistema.
- Gràfics:
 - la configuració del Universal Render Pipeline (URP) amb opcions simples aplicats amb el sistema de *baked lighting* i l'*occlusion culling* de la càmera, com a exemples.
- Àudio:
 - L'ús del sistema *AudioMixer* de Unity per gestionar canals, càrrega prèvia dels clips i sincronització amb el rellotge DSP, per assegurar així una latència mínima.

Gràcies a aquest conjunt de decisions, el sistema ha pogut mantenir-se amb un funcionament fluid, tant a PC com en dispositius com les ulleres Meta Quest 3, fent que es pugui tindre una experiència immersiva sense comprometre l'estabilitat ni la qualitat visual.

6 Problemes i solucions

Durant el desenvolupament del projecte han anat sorgint diferents problemes i reptes que s'han hagut d'anar solucionant de diferents formes. Alguns d'aquestes solucions ja s'han pogut anar veient durant l'explicació del projecte, però en aquest apartat s'explicaran també els problemes concrets i es recull de forma més concreta en forma de llista.

SDK de Meta vs XR Interaction Toolkit

Inicialment es va plantejar utilitzar l'SDK oficial de Meta per a Quest com a base pel desenvolupament de la aplicació però presentava una estructura molt més complexa i estava molt orientat a les necessitats específiques de l'ecosistema de Meta, pel que li treia molta flexibilitat i personalitat al projecte.

Per això, es va migrar a XR Interaction Toolkit (XRI), que proporciona un sistema modular i escalable, a més d'integrar-se millor amb Unity Input System. Aquesta elecció va permetre desenvolupar un sistema més flexible i personalitzat, on es podien implementar les interaccions necessàries des de zero i adaptar-les a les necessitats del projecte. Amés que amb aquesta base es manté també la compatibilitat amb futures versions de Unity i la possibilitat d'utilitzar altres dispositius XR.

Detecció amb *colliders* als instruments

Als primer prototips es van utilitzar *colliders* i *OnTriggerEnter* per detectar els cops de baqueta, però apareixien errors molt freqüents: cops múltiples, deteccions inconsistentes i problemes amb la velocitat.

Com s'ha parlat en profunditat en l'apartat 5.1, es va substituir per un sistema basat en *Physics.Linecast*, on a cada *frame* es dibuixa una línia entre la posició actual i l'anterior de la baqueta. Si aquesta línia travessa un instrument, es considera un impacte. Això dona resultats molt més coherents i precisos, eliminant les deteccions falses que es creaven amb la versió anterior.

Gestió del volum dels instruments

Inicialment, cada instrument tenia un volum independent i quan es combinaven múltiples sons a l'hora s'escoltava un àudio incoherent. A més, es va veure que quan es reproduïen sons molt llargs (per exemple el so d'un plat que durava mes de 5 segons), el sistema es col·lapsava i el resultat era una saturació de l'àudio que impedia escoltar correctament els clips.

Per solucionar-ho es va implementar un *AudioMixer* amb grups separats (música, bateria i efectes), centralitzat amb un *SoundManager* que permet ajustar volums globals i per instrument. També ara cada nota generada agafa un *AudioSource* d'un pool per a que no hi hagi saturacions

A més, per garantir un comportament uniforme del sistema de volums, tots els clips d'àudio es van normalitzar a -23 LUFS mitjançant Audacity (vegeu l'apartat 5.7 per més informació).

Latència MIDI i gestió de threads

Els esdeveniments MIDI arriben en un *thread* extern a Unity configurat per la llibreria DryWetMIDI, i això provocava errors i problemes de sincronització amb les animacions i el so que es produïen al *main thread*.

Per solucionar-ho es va utilitzar un *UnityMainThreadDispatcher* per passar les crides MIDI al *main thread* (vegeu l'apartat 5.4.3).

A més, per la sincronització amb l'àudio per a les cançons també s'ha utilitzat el rellotge *AudioSettings.dspTime*, que manté la precisió i evita desincronitzacions encara que hi hagi fluctuacions el *main thread*.

Rendiment i *object pooling*

Les notes de les cançons MIDI es generaven amb *Instantiate* i *Destroy*, provocant grans pics de rendiment i problemes amb la visualització del programa. Això era causat perquè es generaven moltes notes alhora i s'anaven destruint, fent que es creï tot el sistema que conté cada nota a cada moment que es produïa una i destruint-la al poc temps.

La solució ha estat la implementació d'un sistema de *object pooling* (*NotePool*), que crea prèviament un conjunt de notes i les reutilitza (vegeu l'apartat 5.4.4).

Problema de doble volum en la sortida d'àudio

Durant el procés de desenvolupament del projecte es va detectar que alguns sons s'escoltaven duplicats. Al principi va ser difícil de trobar el motiu de el perquè d'aquest problema fins que es va descobrir que el motiu era que el sistema detectava per defecte com a *MIDI Out* la sortida MIDI de *Windows WaveSynth*, reproduint els mateixos *MIDI Events* que es produïen en el propi escriptori de Windows.

La solució final va ser excloure aquesta sortida en el procés de detecció de inputs i outputs de MIDI i així assegurar que només s'utilitzés el sistema d'àudio propi del projecte, eliminant completament el problema del doble volum.

7 Conclusions i futur

7.1 Conclusions

Es pot concloure que els resultats obtinguts d'aquest projecte de TFG han estat satisfactoris ja que s'han pogut assolir els objectius principals. Finalment, s'ha construït una sala de pràctica en VR al voltant d'una bateria modular amb integració de MIDI bidireccional (*in/out*), importació i reproducció de cançons MIDI, motor d'àudio propi amb dinàmica de volum i un visor per projectar pantalles del PC dins la VR. El resultat no ha estat només un simulador, sinó un entorn amb les eines per poder practicar, aprendre i crear música.

L'aplicació s'ha dissenyat perquè la puguin utilitzar tant persones noves en la bateria com usuaris amb experiència. Per això s'ofereixen drumsets preconfigurats llestos per tocar i, alhora, la possibilitat de guardar configuracions pròpies. La UI s'ha plantejat perquè sigui intuïtiva i permeti una posada en marxa ràpida sense renunciar al control fi quan l'usuari el necessita.

En la interacció, s'ha aconseguit realitzar una experiència immersiva amb baquetes virtuals i un sistema de detecció capaç de captar intensitat (dinàmica), velocitat i zona d'impacte de els instruments.

L'objectiu MIDI bidireccional s'ha pogut complir satisfactòriament. El sistema admet dispositius MIDI externs (per exemple, una bateria electrònica) com a entrada dins de la VR i, al mateix temps, pot actuar com a instrument de sortida, enviant notes a un DAW. Això converteix l'aplicació en una eina que no només es troba en l'àmbit lúdic sinó que es pot endinsar més enllà.

Com a suport a l'aprenentatge i la pràctica lliure, el reproductor MIDI permet importar i interpretar qualsevol cançó en aquest format, combinat amb la projecció de partitures i la visualització de pantalles del PC dins la VR (per controlar un DAW o reproduir música de fons), l'usuari pot estudiar, practicar i enregistrar sense sortir de l'entorn immersiu.

També s'ha complert l'objectiu estètic de la aplicació de una bona manera.

A nivell personal, aquest projecte ha estat ***ACABAR***

7.2 Línies de futur

Integració de hand tracking

- Adoptar Unity XR Hands / OpenXR (o l'API de Meta Hand Tracking) per agafar les baquetes directament amb les mans.
- Gestos (pinça, agafar, deixar) i calibratge de la posició de la baqueta respecte a la mà.

Ampliació d'instruments i biblioteques de so

- Aprofitar l'arquitectura modular per afegir nous tambors, plats i percussions.
- Procés de mostreig i edició amb normalització a -23 LUFS per mantenir coherència de volum.

Millora del catàleg i dels "samples" de cançons

- Crear noves cançons midi per a la pràctica del instrument

Publicació de la aplicació

- Adaptar el programa per a poder publicar-ho en plataformes
- Nou logo
- notes de llançament, correccions, noves funcionalitats
- Preparació de fitxes de botiga, trailer curt i documentació d'usuari.

7.3 Costos

Material:

- Meta Quest 3 (segona mà): 433 €
- Pedal USB: 5 €
- Cable USB 3.0: 5 €
- Total maquinari: 443 €

Programes (gratis)

- Unity (licència gratuïta/personal).
- Audacity (codi obert).
- 3ds Max i Illustrator (ús amb llicència educativa/accés sense cost durant el TFG).
- Canva (pla gratuït per a gràfics de la memòria).

Temps

5–6 h diàries durant uns 3 mesos.

Tram final s'hi han dedicat més hores, mes o menys 500–600 h totals

8 Referències

- [1] Xiaorui Guo. (2023). *The Evolution of the Music Industry in the Digital Age: From Records to Streaming* [Online]
https://www.clausiuspress.com/assets/default/article/2023/11/27/article_1701102221.pdf
- [2] "frommaplesteeze". (2018, Febrer). Still too loud for my neighbors. I hate this instrument sometimes. [Online]
https://www.reddit.com/r/drums/comments/8ynxly/still_too_loud_for_my_neighbors_i_hate_this
- [3] Qingtian Li. (2024, Març). From Virtual to Reality: How VR, AR, XR, MR are Reshaping Our Lives and Work. [Online]
<https://drpress.org/ojs/index.php/ijeh/article/view/24901/24393>
- [4] Wikipedia. (2020). Beat Saber [Online]
https://es.wikipedia.org/wiki/Beat_Saber
- [5] Steam store. (2022, Novembre). Paradiddle [Online]
<https://store.steampowered.com/app/685240/Paradiddle/?l=spanish>
- [6] Steam store. (2021, Juliol). Ragnarock. [Online]
<https://store.steampowered.com/app/1345820/Ragnarock/>
- [7] Steam Store. (2024, Maig). KORG Gadget VR. [Online]
https://store.steampowered.com/app/2298070/KORG_Gadget_VR/
- [8] Steam Store. (2022, Novembre). Maestro VR. [Online]
https://store.steampowered.com/app/1327920/Maestro_VR/
- [9] Wikipedia. (2018). VRChat [Online]
<https://es.wikipedia.org/wiki/VRChat>
- [10] Meta. (2025, September). ¿Qué es el metaverso?. [Online]
<https://www.meta.com/es-es/metaverse/what-is-the-metaverse>
- [11] Unity. (2025). Características del motor. [Online]
<https://unity.com/es/products/unity-engine>
- [12] Unity Documentation. Aprendiendo de la Interfaz. [Online]
<https://docs.unity3d.com/es/530/Manual/LearningtheInterface.html>
- [13] Unity Documentation. Game Objects. [Online]
<https://docs.unity3d.com/es/530/Manual/class-GameObject.html>
- [14] Unity Documentation. Usando Componentes. [Online]
<https://docs.unity3d.com/es/530/Manual/UsingComponents.html>
- [15] Unity Documentation. Prefabs. [Online]
<https://docs.unity3d.com/es/530/Manual/Prefabs.html>
- [16] Unity Documentation. Creando y usando scripts. [Online]
<https://docs.unity3d.com/es/current/Manual/CreatingAndUsingScripts.html>
- [17] Unity Documentation. UI (Interfaz de Usuario). [Online]
<https://docs.unity3d.com/es/2018.4/Manual/UISystem.html>
- [18] Unity Documentation. Audio. [Online]
<https://docs.unity3d.com/es/530/Manual/Audio.html>
- [19] Unity Documentation. Introduction to materials. [Online]
<https://docs.unity3d.com/6000.1/Documentation/Manual/materials-introduction.html>
- [20] Unity Documentation. Sistema de partículas. [Online]
<https://docs.unity3d.com/es/530/Manual/class-ParticleSystem.html>

- [21] FistFullofShrimp. How To Make A VR Game in an Hour Using Unity 6! (2024) [Online]
https://www.youtube.com/watch?v=kbBYcVrGZus&t=2026s&ab_channel=FistFullofShrimp
- [22] Unity Manual. XR Interaction Toolkit. (2021, Octubre). [Online]
<https://docs.unity.cn/Packages/com.unity.xr.interaction.toolkit%401.0/manual/index.html>
- [23] GitHub. uWindowCapture. (2021, Desembre). [Online]
<https://github.com/hecomi/uWindowCapture>
- [24] AssetStore. LeanTween. (2021, Desembre) [Online]
<https://assetstore.unity.com/packages/tools/animation/leantween-3595>
- [25] Wikipedia. MIDI. (2025, Setembre) [Online]
<https://ca.wikipedia.org/wiki/MIDI>
- [26] MIDI Association. MIDI. (2025). [Online]
<https://midi.org/>
- [27] Wikipedia. Digital Audio Workstation. (2025, Juny) [Online]
https://en.wikipedia.org/wiki/Digital_audio_workstation
- [28] DryWetMidi – Overview. (2025) [Online]
<https://melanchall.github.io/drywetmidi/>
- [29] General MIDI Standard Percussion Set Key Map (2025). [Online]
<https://musescore.org/sites/musescore.org/files/General%20MIDI%20Standard%20Percussion%20Set%20Key%20Map.pdf>
- [30] SM Drums - Deeply Sampled Free Drums for the Masses. (2025). [Online]
<https://smmdrums.wordpress.com/>
- [31] Wikipedia. LUFs. (2025, agost) [Online]
<https://ca.wikipedia.org/wiki/LUFs>
- [32] Canal de youtube – Fist Full of Shrimp. (2025) [Online]
<https://www.youtube.com/@FistFullofShrimp>
- [33] Canal de youtube – Valem Tutorials. (2025) [Online]
<https://www.youtube.com/@ValemTutorials>
- [34] Canal de youtube – Mike VR. (2025) [Online]
<https://www.youtube.com/@MikeVR>
- [35] Unity Manual. [Online]
<https://docs.unity3d.com/es/530/Manual/UnityManual.html>

9 Il·lustracions

Il·lustració 1. Representació visual de la indústria musical. Font: https://www.royaltyexchange.com/blog/influence-of-streaming-services-on-global-music-consumption	13
Il·lustració 2. La bateria i les seves parts Font: https://www.drumeo.com/beat/parts-of-a-drum-set/	13
Il·lustració 3. Bateria acústica insonoritzada.....	14
Il·lustració 4. Implementació de la VR en laboratori de Ciències de la Salut. Font: https://simzine.news/es/experience-es/sim-voices-es/utilizar-la-vr-en-el-aula-o-laboratorio-de-ciencias-de-la-salud-con-un-gran-numero-de-estudiantes/	15
Il·lustració 5. Fotograma del joc Beat Saber. Font: https://www.meta.com/es-es/experiences/beat-saber/2448060205267927/	15
Il·lustració 6. Fotograma d'una cover feta en ParadiddleVR. Font: https://paradiddleapp.com/	16
Il·lustració 7. Exemple del joc KORG Gadget VR amb espai de producció musical. Font: https://store.steampowered.com/app/2298070/KORG_Gadget_VR/?l=spanish	16
Il·lustració 8. Diagrama de Vann de les motivacions per al projecte	17
Il·lustració 9. Diagrama de com funciona el sistema MIDI bidireccional	18
Il·lustració 10. Fotograma de gameplay del joc Beat Saber. Font: https://store.steampowered.com/app/620980/Beat_Saber/	19
Il·lustració 11. Banner oficial de ParadiddleVR.	20
Il·lustració 12. Kit modular del joc ParadiddleVR.....	20
Il·lustració 13. Fotograma del joc RagnaRock.	21
Il·lustració 14. Fotograma de KORG Gadget VR.	22
Il·lustració 15. Fotograma del joc Maestro VR.	23
Il·lustració 16. Consola Virtual Boy de Nintendo. Font: https://es.wikipedia.org/wiki/Virtual_Boy	24
Il·lustració 17. Fotograma del videojoc Half-Life: Alyx. Font: https://store.steampowered.com/app/546560/HalfLife_Alyx	24
Il·lustració 18. VRChat. Font: https://www.businessinsider.com/vrchat-explained-2018-2	25
Il·lustració 19. La realitat virtual aplicada en entorns professionals. Font: https://centroformacionprofesorado.castillalamancha.es/comunidad/crpf/recurso/realidad-virtual-y-su-aplicacion-en-fp-a-traves/a5a471b6-b629-40cb-80fa-a8e216d256d7	25
Il·lustració 20. Aplicació UnityHub amb els seus diferents templates.	26
Il·lustració 21. Interfície de Unity i les seves parts.....	27
Il·lustració 22. Llistat de GameObjects de Unity.	28
Il·lustració 23. Exemple de un GameObject i els seus components.....	28
Il·lustració 24. <i>Component Browser</i> de Unity.....	29
Il·lustració 25. Exemple de un component Script amb les seves propietats.....	29
Il·lustració 26. Un prefab i la seva configuració desde l'editor.	30
Il·lustració 27. Nou Script de Unity.	31
Il·lustració 28. Componentes de un canvas per a VR.	32
Il·lustració 29. Menú d'elements de la UI de Unity.....	32

Il·lustració 30. Audio Mixer de Unity.....	33
Il·lustració 31. Audio Source de Unity.....	33
Il·lustració 32. Material vist des de l'inspector de Unity.....	34
Il·lustració 33. Partícula i el sistema de partícules	34
Il·lustració 34. Build Settings de Unity	35
Il·lustració 35. XR Plug-in Management.....	35
Il·lustració 36. Pestanya de Project Validation sense errors.....	36
Il·lustració 37. Prefab XR Origin del XR Interaction Toolkit.....	37
Il·lustració 38. Component XR Grab Interactable.	37
Il·lustració 39. uWindowCapture en acció. Font: https://github.com/hecomi/uWindowCapture	38
Il·lustració 40. Exemples de animacions que pot fer LeanTween.....	38
Il·lustració 41. <i>MIDI Events</i> registrats per la aplicació MIDI-OX.....	39
Il·lustració 42. Connectors MIDI físics	39
Il·lustració 43. Configuració MIDI del DAW FLStudio 2024	40
Il·lustració 44. Ports MIDI virtuals creats amb loopMIDI	40
Il·lustració 45. Logo de DryWetMIDI. Font: https://melanchall.github.io/drywetmidi/	41
Il·lustració 46. Apartat d'informació del menú principal	42
Il·lustració 47. Pantalla de <i>spawner</i> d'instruments.....	42
Il·lustració 48. Menú d' <i>spawn</i> de <i>drumkits</i>	43
Il·lustració 49. Reproductor de cançons MIDI amb la partitura de fons.....	43
Il·lustració 50. Menú de configuració de MIDI i Pedal	44
Il·lustració 51. Exemple de el <i>window viewer</i> amb diferents pestanyes obertes	44
Il·lustració 52. Exemple de la configuració del menú principal i un menú d'un instrument obert	45
Il·lustració 53. Escena de el programa	45
Il·lustració 54. Primer prototip de una bateria amb <i>colliders</i>	47
Il·lustració 55. Fragment de l'script d'activació del so <i>OnTriggerEnter</i>	47
Il·lustració 56. Diagrama d'arquitectura del sistema	48
Il·lustració 57. <i>Prefabs</i> dels diferents instruments implementats	49
Il·lustració 58. Snare.prefab	49
Il·lustració 59. Exemple del <i>prefab</i> Ride17 amb un <i>ConfigurableJoint</i> i <i>CymbalKinematicToggle</i>	50
Il·lustració 60. Prefab hi-hat.....	50
Il·lustració 61. Baqueta virtual col·locada en l'escena amb el <i>HeadPosition</i> seleccionat.....	51
Il·lustració 62. Esquema de la trajectòria de la baqueta entre dos frames i la detecció a través de linecast	51
Il·lustració 63. Update de <i>DrumStickCollision</i>	52
Il·lustració 64. Script <i>DrumStickCooldown</i>	52
Il·lustració 65. Fragment de la funció <i>DrumHit</i>	53
Il·lustració 66. Fragment de <i>DrumHit()</i> on s'aplica la força si l'instrument té físiques	53
Il·lustració 67. <i>HandleMidiEvent</i> del script <i>MidiManager</i>	54
Il·lustració 68. Codi del <i>MidiReceiver</i>	54
Il·lustració 69. Funció <i>DrumHitFromMidi</i>	55
Il·lustració 70. Funció <i>SetAction</i> del component <i>KeyTrigger</i>	55

Il·lustració 71. Clips d'àudio del instrument <i>Snare</i>	56
Il·lustració 72. <i>Ride</i> amb els seus clips d'àudio i zones diferents.....	56
Il·lustració 73. Fragment de codi que escull l'àudio a reproduir.....	57
Il·lustració 74. Secció de codi de reproducció del so.	57
Il·lustració 75. Fragment final de les funcions <i>DrumHit()</i> i <i>DrumHitFromMidi()</i>	58
Il·lustració 76. Funció <i>AnimateScale</i>	58
Il·lustració 77. Corba de l'animació dels instruments	58
Il·lustració 78. Partícula d'un instrument.....	59
Il·lustració 79. Configuració del <i>ParticleSystem</i>	59
Il·lustració 80. Menú individual dels instruments.....	60
Il·lustració 81. Volum del instrument multiplicat pel valor <i>volumeMultiplier</i>	60
Il·lustració 82. Funció <i>StartRebind()</i> per l'assignació de tecles	61
Il·lustració 83. Funció <i>OnValueChanged</i> del <i>slider</i> de canvi de <i>MIDI Note</i>	61
Il·lustració 84. Script <i>PedalState</i>	62
Il·lustració 85. Configuració del component <i>HiHatAnimator</i>	63
Il·lustració 86. Script <i>HiHatAnimator</i>	63
Il·lustració 87. Estats del <i>hi-hat</i> amb animació	63
Il·lustració 88. Clips d'àudio del <i>hi-hat</i> i els diferents MIDI notes.....	64
Il·lustració 89. <i>HiHatHitDetector</i>	64
Il·lustració 90. Funcions <i>SplashHit</i> i <i>PlayFootChick()</i>	64
Il·lustració 91. Secció de script que envia els outputs del <i>hi-hat</i>	65
Il·lustració 92. Mapa de correspondència dels instruments i la seva nota MIDI	65
Il·lustració 93. Pàgina del menú d' <i>spawn</i>	66
Il·lustració 94. Component <i>PrefabSpawner</i>	66
Il·lustració 95. Mètode <i>OnClick()</i> del botó del <i>spawner</i> amb índex 0	67
Il·lustració 96. Mètode <i>OnClick()</i> del botó del <i>spawner</i> amb índex 1	67
Il·lustració 97. Funció <i>SpawnPrefab</i>	67
Il·lustració 98. Panell de gestió dels <i>DrumSets</i>	68
Il·lustració 99. Conjunt de <i>scripts</i> necessaris per al maneig dels <i>drumsets</i>	68
Il·lustració 100. Classes <i>DrumData</i> i <i>DrumsetData</i>	69
El <i>DrumType</i> s'agafa del <i>DrumController</i> de cada instrument tal i com es pot veure a la Il·lustració 101.....	69
Il·lustració 102. <i>DrumController</i> del <i>prefab Snare</i> amb el seu <i>DrumType</i>	69
Il·lustració 103. <i>DrumSetManager</i>	69
Il·lustració 104. <i>CaptureCurrentDumset</i> del script <i>DrumSetRecorder</i>	70
Il·lustració 105. Funció <i>OnSlotButtonPressed</i>	70
Il·lustració 106. Funcions de guardat del <i>DrumSetSlotManager</i>	71
Il·lustració 107. Kits prefets dins de la carpeta <i>Assets</i> del programa	71
Il·lustració 108. Kits personalitzats dins de la carpeta de <i>Persistent Data</i>	71
Il·lustració 109. Funcions del <i>DrumSetLoader</i>	72
Il·lustració 110. Funció <i>SpawnDrumSet</i>	72
Il·lustració 111. Animació d'aparició dels instruments	72
Il·lustració 112. <i>GameObject DrumSetManager</i> dins de <i>Unity</i> amb els seus components	73
Il·lustració 113. Panell de gestió de <i>drumsets</i> esperant a guardar un <i>preset</i> de l'usuari	73
Il·lustració 114. Scripts per a la gestió MIDI	74

Il·lustració 115. Secció de MIDI In al panell del menú.	74
Il·lustració 116. Funció per llegir tots els inputs MIDI connectats	74
Il·lustració 117. Funció <i>RefreshDeviceList</i>	75
Il·lustració 118. Funció <i>InputDevice</i>	75
Il·lustració 119. Funció <i>HandleMidiEvent</i>	76
Il·lustració 120. Funció <i>SendNote</i>	76
Il·lustració 121. Programa <i>loopMidi</i> configurat amb diferents ports MIDI virtuals.....	77
Il·lustració 122. Panell de <i>MidiIn / MidiOut</i> configurat.....	77
Il·lustració 123. Conjunt de scripts per a les cançons MIDI	78
Il·lustració 124. Carpetes de guardat de les cançons.....	78
Il·lustració 125. Funció <i>RefreshSongsDropdown</i>	79
Il·lustració 126. Panell de selecció de cançons.	79
Il·lustració 127. Funció <i>HandleSelection</i>	79
Il·lustració 128. Fragment del script <i>LoadSong</i>	80
Il·lustració 129. Esquema de la sincronització de les cançons	80
Il·lustració 130. Temps aplicat al panell visual	81
Il·lustració 131. Update del <i>NoteMover</i>	81
Il·lustració 132. Funció <i>HitTarget</i> del <i>NoteMover</i>	81
Il·lustració 133. Panell del reproductor de cançons.....	82
Il·lustració 134. Event trigger del slider de temps.....	82
Il·lustració 135. Funció <i>OnSliderDragStart</i>	83
Il·lustració 136. Funció <i>OnSliderReleased</i>	83
Il·lustració 137. Pentagrama amb les seves lanes.....	84
Il·lustració 138. Component <i>MidiNoteSpawner</i> i classe <i>LaneConfig</i>	84
Il·lustració 139. Prefabs de cada una de les notes	84
Il·lustració 140. Component <i>Note Pool</i>	85
Il·lustració 141. Funció Get i Release del <i>Note Pool</i>	85
Il·lustració 142. Component <i>NoteSoundManager</i>	86
Il·lustració 143. Pentagrama en acció	86
Il·lustració 144. Prefab del <i>uWC Window Object</i>	87
Il·lustració 145. Funcions d'afegir i treure finestres.....	87
Il·lustració 146. Funcions de <i>OnWindowsAdded</i> i <i>OnWindowsRemoved</i> de <i>UwcModifiedSpwan</i>	88
Il·lustració 147. Captura de finestres en acció	88
Il·lustració 148. <i>AudioMixer</i> del sistema	89
Il·lustració 149. Panell de volum general amb <i>sliders</i> de <i>Master, Drums i Song</i>	89
Il·lustració 150. S'agafen els valors de <i>PlayerPrefs</i> per posar el volum inicial.	89
Il·lustració 151. Funcions del <i>SoundManager</i>	90
Il·lustració 152. Menú individual de cada instrument	90
Il·lustració 153. Icones dels diferents panells del menú	91
Il·lustració 154. Menú individual de l'instrument	91
Il·lustració 155. Configuració del <i>canvas</i> de les interfícies	92
Il·lustració 156. Estructura del menú	92
Il·lustració 157. <i>ButtonTab</i> amb el seu índex assignat.....	92
Il·lustració 158. Component <i>TabGroup</i>	92

Il·lustració 159. Programa Adobe Illustrator amb totes les capes amb els elements de les interfícies.....	93
Il·lustració 160. Paleta de colors de les interfícies.	93
Il·lustració 161. Exemples de la font Montserrat. Font: https://zsquarestudio.com/exploring-the-elegance-and-versatility-of-the-montserrat-font/	93
Il·lustració 162. Icones del programa dins de Unity.....	94
Il·lustració 163. Menú principal en acció	94
Il·lustració 164. Panell d'informació amb controls i icones del sistema	95
Il·lustració 165. Panell d'ajustos amb les seves funcions.....	95
Il·lustració 166. Ajust de l'alçada en el panell del menú	96
Il·lustració 167. Funcions del component CameraHightAdjuster.	96
Il·lustració 168. Configuració d'alçades.....	96
Il·lustració 169. Panell d'ajust dels angles de les baquetes	97
Il·lustració 170. Funcions de <i>DrumstickAngleAdjust</i>	97
Il·lustració 171. Usuari ajustant les baquetes	97
Il·lustració 172. Botó de <i>reset view</i>	98
Il·lustració 173. <i>GameObject InitialPosition</i> en el punt inicial d'aparició.....	98
Il·lustració 174. Funció <i>ResetPlayerPosition()</i>	98
Il·lustració 175. Panell del menú amb els botons de canvi de mà.	99
Il·lustració 176. Funcions del MenuHandToggle.....	99
Il·lustració 177. Exemple del menú en cada mà.....	99
Il·lustració 178. Clips d'àudio per al Ride de 20'	100
Il·lustració 179. Carpetes organitzades dels clips d'àudio	100
Il·lustració 180. Efecte de normalització en LUFS de <i>Audacity</i>	101
Il·lustració 181. Aplicació de la normalització a -23 LUFS.	101
Il·lustració 182. Model dels instruments en 3DS Max.	102
Il·lustració 183. Separació dels materials en ID's dins de 3DS Max	102
Il·lustració 184. Materials utilitzats dins de Unity.....	102
Il·lustració 185. Model del entorn dins de 3DS Max.....	103
Il·lustració 186. Render de la sala dins de 3DS Max.....	103
Il·lustració 187. Llums col·locades dins de l'escena 1	104
Il·lustració 188. Llums col·locades dins de l'escena 2	104
Il·lustració 189. Llums col·locades dins de l'escena 3	104
Il·lustració 190. Light Probes dens de la sala de bateria.	105
Il·lustració 191. Reflection probe dins de l'escena.....	105
Il·lustració 192. Exemple ambient 1	106
Il·lustració 193. Exemple ambient 2	106
Il·lustració 194. Exemple ambient 3.....	106
Il·lustració 195. Exemple ambient 4.....	107
Il·lustració 196. Exemple de la sala amb instruments.....	107